



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Computación

Análisis del rendimiento y consumo de modelos de IA para detección de intrusiones

Performance and consumption analysis of AI models for intrusion detection

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, June 8, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Mención en Computación

**Análisis del rendimiento y consumo de modelos de
IA para detección de intrusiones**

**Performance and consumption analysis of AI
models for intrusion detection**

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JUNE 8, 2026

Resumen

El proyecto se enfoca en el análisis, evaluación, selección de hiperparámetros y configuración de modelos de IA para comparar el rendimiento y eficacia computacional (CPU, latencia) de los distintos modelos aplicados a los sistemas de detección de intrusiones. Se busca determinar que modelos son más robustos, pero también eficientes desde el punto de vista computacional.

El proyecto se estructura en varias fases. Inicialmente, obtenemos y analizamos conjuntos de datos realistas proporcionados por expertos en ciberseguridad y altamente utilizados en tareas de investigación. Posteriormente, desarrollamos sistemas de detección de intrusiones, haciendo una búsqueda de los mejores hiperparámetros. Los modelos que usaremos son modelos ensembles de árboles, SVM, perceptrón multicapa y redes convolucionales 1-D.

Finalmente, miramos cómo se comportan estos modelos entrenados ante mecanismos de ataques de evasión como PGD o FGSM y compararlos para ver su degradación.

Palabras clave: sistemas de detección de intrusiones (IDS), ataques adversarios, Internet de las Cosas (IoT), eficiencia computacional, frontera de pareto, aprendizaje automático

Abstract

The project focuses on the analysis, evaluation, hyperparameter selection and configuration of AI models to compare the performance and computational efficiency (CPU, latency) of the various models applied to intrusion detection systems. The aim is to determine which models are not only more robust but also computationally efficient.

The project is structured in several phases. Initially, we obtain and analyse realistic *datasets* provided by cybersecurity experts and widely used in research tasks. Subsequently, we develop intrusion detection systems, searching for the optimal hyperparameters. The models we will use are ensemble models comprising decision trees, SVMs, multilayer perceptrons and 1-D convolutional networks.

Finally, we examine how these trained models perform against evasion attack mechanisms such as PGD or FGSM and compare them to assess their degradation.

Keywords: intrusion detection systems (IDS), cyberattacks, the Internet of Things (IoT), computational efficiency, pareto frontier, machine learning

Agradecimientos

A mis padres. Este trabajo lleva mi nombre en la portada, pero la historia que lo ha hecho posible es la vuestra. Habéis sacrificado los sueños que no os pudisteis permitir para que yo pudiera construir los míos. Gracias por enseñarme a no rendirme y a perseguir mis metas con pasión. Este logro es tan vuestro como mío, y os lo dedico con todo mi corazón. Prometo algún día ser tan fuerte como vosotros.

A mi hermana. Literalmente, mi compañera de vida. La mejor mitad que me pudo tocar. Venir al mundo acompañado tiene la ventaja de que nunca caminas solo, y tú me lo has demostrado cada día de mi vida. Gracias por ser la persona que mejor me conoce en el mundo. Luci, hemos compartido todo desde el minuto cero, y no entendería la vida, ni mucho menos este logro, si no es teniéndote a mi lado.

A mis abuelos. El amor más grande y puro que tengo. Gracias a los que me acompañáis de cerca, porque ilumináis todo el esfuerzo que me ha costado llegar hasta aquí y vuestra cara de orgullo es mi mayor recompensa. Y a los que se fueron, pero que nunca me han dejado del todo porque viven en mi corazón. Cada paso que he dado lleva vuestra huella, y sé que hoy celebraríais esta meta tanto como yo.

A mis amigos. Los que llevan conmigo toda la vida. Los que me han visto crecer. La familia que se elige. Gracias por ser la constante en este mundo de cambios y por no fallar nunca cuando más os he necesitado. Y gracias también a las personas increíbles que se han cruzado en mi camino durante estos años universitarios. Este trabajo pone fin a una etapa académica, pero mi mayor victoria ha sido conocerlos.

A mis tutores, Fran y Rodrigo. Gracias por acompañarme en este proceso, por guiarme, por enseñarme y por confiar en mí. Este trabajo no habría sido posible sin vuestra ayuda, y me siento muy afortunado de haber podido aprender de vosotros.

Y por último, **a mí mismo.** Por todo lo lejos que hemos llegado, y por las grandes cosas que quedan por venir. Porque a veces el mayor reto es creer en uno mismo. Gracias por no rendirte, por seguir adelante incluso cuando las cosas se ponían difíciles, y por demostrarme que soy capaz de lograr lo que me proponga. A seguir luchando por mis sueños, que esto es solo el primer paso.

Gracias a todos.

Resumen	1
Abstract	3
Agradecimientos	5
Lista de acrónimos	11
1 Introducción	11
1.1 Motivación	11
1.2 Objetivos	11
1.3 Metodología de trabajo	12
1.4 Estructura del documento	12
2 Investigación Previa	13
2.1 Machine Learning	13
2.1.1 Random Forest (RF)	13
2.1.2 Extreme Gradient Boosting (XGBoost)	14
2.1.3 Light Gradient Boosting Machine (LightGBM)	15
2.1.4 CatBoost	15
2.1.5 Support Vector Machine (SVM)	16
2.2 Deep Learning	17
2.2.1 Multilayer Perceptron (MLP)	17
2.2.2 Convolutional Neural Networks (CNN)	18
2.3 Justificación del enfoque en la fase de inferencia	19
2.4 Métricas Utilizadas	19
3 Tecnologías utilizadas	21
3.1 Lenguaje de Programación	21
3.2 Librerías Usadas	21
3.3 Entorno de pruebas	21
3.4 Optuna	22
4 Obtención y Análisis de Datos	25
4.1 UNSW-NB15 <i>DataSet</i>	25
4.1.1 Motivación para el uso del <i>dataset</i>	25
4.1.2 Estructura del <i>dataset</i>	25
4.1.3 Distribución de ataques	26
4.2 ToN_IoT <i>DataSet</i>	26
4.2.1 Motivación para el uso del <i>dataset</i>	26
4.2.2 Estructura del <i>dataset</i>	27
4.2.3 Distribución de ataques	27
4.3 IoT-23 <i>DataSet</i>	28

4.3.1	Motivación para el uso del <i>dataset</i>	28
4.3.2	Estructura del <i>dataset</i>	28
4.3.3	Distribución de ataques	28
4.4	Relación entre los <i>datasets</i>	29
5	Análisis de Modelos y Búsqueda de Hiperparámetros	31
5.1	Metodología experimental	31
5.1.1	Flujo de trabajo experimental	31
5.1.2	Optimización con Optuna	32
5.2	Optimización de hiperparámetros estructurales	33
5.2.1	Random Forest	33
5.2.2	XGBoost	34
5.2.3	LightGBM	34
5.2.4	CatBoost	34
5.2.5	Support Vector Machine (SVM)	34
5.2.6	Multilayer Perceptron (MLP)	35
5.2.7	Convolutional Neural Networks (CNN 1D)	35
5.3	Modelos entrenados con UNSW-NB15	35
5.4	Modelos entrenados con IOT-23	39
5.5	Modelos entrenados con ToN-IoT	42
5.6	Potencial de Análisis en Tiempo Real	45
5.6.1	Resultados UNSW-NB15	45
5.6.2	Resultados IOT-23	46
5.6.3	Resultados TON-IOT	46
5.6.4	Comparación con cargas de red de referencia	46
6	Ataques de Evasión	49
6.1	Evaluación de robustez frente a ataques adversarios	49
6.1.1	Uso de Adversarial Robustness Toolbox	49
6.1.2	Análisis de Resultados en UNSW-NB15	51
6.1.3	Análisis de Resultados en IOT-23	55
6.1.4	Análisis de Resultados en TON-IOT	58
6.2	Análisis Límite de Perturbación	62
6.2.1	Resultados UNSW-NB15	62
6.2.2	Resultados IOT-23	64
6.2.3	Resultados TON-IOT	65
6.2.4	Conclusiones del análisis del límite de perturbación	67
7	Conclusiones y Líneas Futuras	69
7.1	Conclusiones	69
7.2	Líneas Futuras	70
	Apéndice A. Resultados completos de la optimización de hiperparámetros	73

A.1	Modelos entrenados con UNSW-NB15	73
A.1.1	Random Forest	73
A.1.2	XGBoost	74
A.1.3	LightGBM	75
A.1.4	CatBoost	76
A.1.5	SVM	77
A.1.6	MLP	78
A.1.7	CNN-1D	79
A.2	Modelos entrenados con IoT-23	80
A.2.1	Random Forest	80
A.2.2	XGBoost	81
A.2.3	LightGBM	82
A.2.4	CatBoost	83
A.2.5	SVM	84
A.2.6	MLP	85
A.2.7	CNN-1D	86
A.3	Modelos entrenados con TON-IoT	87
A.3.1	Random Forest	87
A.3.2	XGBoost	88
A.3.3	LightGBM	89
A.3.4	CatBoost	90
A.3.5	SVM	92
A.3.6	MLP	93
A.3.7	CNN-1D	94
Apéndice B. Manual de instalación, configuración y ejecución		97
B.1	Contenido del repositorio	97
B.2	Requisitos previos	98
B.3	Obtención del código fuente	98
B.4	Preparación del entorno Python	98
B.5	Preparación de los datos	99
B.6	Ejecución de Jupyter	100
B.7	Reproducción de los entrenamientos	100
B.8	Uso de modelos ya entrenados	102
B.9	Reproducción de los ataques adversarios	103
B.10	Compilación de la memoria	103
B.11	Comprobaciones finales	104

5.1	Fronteras de Pareto para XGBoost y SVM	32
5.2	Curva ROC - Modelos	37
5.3	MC - Random Forest	37
5.4	MC - XGBoost	37
5.5	MC - LightGBM	38
5.6	MC - CatBoost	38
5.7	MC - SVM	38
5.8	MC - MLP	38
5.9	MC - CNN	38
5.10	Curva ROC - Modelos (IOT-23)	40
5.11	MC - Random Forest (IOT-23)	40
5.12	MC - XGBoost (IOT-23)	40
5.13	MC - LightGBM (IOT-23)	41
5.14	MC - CatBoost (IOT-23)	41
5.15	MC - SVM (IOT-23)	41
5.16	MC - MLP (IOT-23)	41
5.17	MC - CNN (IOT-23)	41
5.18	Curva ROC - Modelos (ToN-IoT)	43
5.19	MC - Random Forest (ToN-IoT)	44
5.20	MC - XGBoost (ToN-IoT)	44
5.21	MC - LightGBM (ToN-IoT)	44
5.22	MC - CatBoost (ToN-IoT)	44
5.23	MC - SVM (ToN-IoT)	44
5.24	MC - MLP (ToN-IoT)	44
5.25	MC - CNN (ToN-IoT)	45
6.1	F1-PGD-UNSW-NB15	52
6.2	Accuracy-PGD-UNSW-NB15	52
6.3	Recall-PGD-UNSW-NB15	53
6.4	F1-FGSM-UNSW-NB15	54
6.5	Accuracy-FGSM-UNSW-NB15	54
6.6	Recall-FGSM-UNSW-NB15	54
6.7	F1-PGD-IOT-23	55
6.8	Accuracy-PGD-IOT-23	56
6.9	Recall-PGD-IOT-23	56
6.10	F1-FGSM-IOT-23	57

6.11	Accuracy-FGSM-IOT-23	57
6.12	Recall-FGSM-IOT-23	58
6.13	F1-PGD-TON-IOT	59
6.14	Accuracy-PGD-TON-IOT	59
6.15	Recall-PGD-TON-IOT	59
6.16	F1-FGSM-TON-IOT	60
6.17	Accuracy-FGSM-TON-IOT	61
6.18	Recall-FGSM-TON-IOT	61
7.1	Frontera de Pareto - Random Forest	73
7.2	Frontera de Pareto - XGBoost	74
7.3	Frontera de Pareto - LightGBM	75
7.4	Frontera de Pareto - CatBoost	76
7.5	Frontera de Pareto - SVM	77
7.6	Frontera de Pareto - MLP	78
7.7	Frontera de Pareto - CNN	79
7.8	Frontera de Pareto - Random Forest (IOT-23)	80
7.9	Frontera de Pareto - XGBoost (IOT-23)	81
7.10	Frontera de Pareto - LightGBM (IOT-23)	82
7.11	Frontera de Pareto - CatBoost (IOT-23)	83
7.12	Frontera de Pareto - SVM (IOT-23)	84
7.13	Frontera de Pareto - MLP (IOT-23)	85
7.14	Frontera de Pareto - CNN-1D (IOT-23)	86
7.15	Frontera de Pareto - Random Forest (ToN-IoT)	87
7.16	Frontera de Pareto - XGBoost (ToN-IoT)	88
7.17	Frontera de Pareto - LightGBM (ToN-IoT)	89
7.18	Frontera de Pareto - CatBoost (ToN-IoT)	91
7.19	Frontera de Pareto - SVM (ToN-IoT)	92
7.20	Frontera de Pareto - MLP (ToN-IoT)	93
7.21	Frontera de Pareto - CNN-1D (ToN-IoT)	94

2.1	Principales hiperparámetros de Random Forest y su significado.	14
2.2	Principales hiperparámetros de XGBoost y su significado.	15
2.3	Principales hiperparámetros de LightGBM y su significado.	15
2.4	Principales hiperparámetros de CatBoost y su significado.	16
2.5	Principales hiperparámetros de SVM y su significado.	17
2.6	Principales hiperparámetros de MLP y su significado.	18
2.7	Principales hiperparámetros de CNN 1D y su significado.	19
2.8	Definición de métricas de evaluación de rendimiento	20
3.1	Librerías utilizadas durante el desarrollo experimental	21
3.2	Especificaciones del entorno experimental	22
3.3	Características técnicas del equipo local utilizado para la inferencia	22
4.1	Distribución de clases y tipos de ataque en el <i>dataset</i> UNSW-NB15	26
4.2	Distribución de clases y tipos de ataque en el <i>dataset</i> ToN_IoT	27
4.3	Distribución de clases y tipos de tráfico en el <i>dataset</i> IoT-23	28
4.4	Correspondencia principal entre características de UNSW-NB15	29
4.5	Correspondencia principal entre características de ToN_IoT e IoT-23	30
5.1	Hiperparámetros ganadores de los modelos evaluados con UNSW-NB15	35
5.2	Comparativa global de rendimiento de modelos - UNSW-NB15	36
5.3	Hiperparámetros ganadores de los modelos evaluados con IOT-23	39
5.4	Comparativa global de rendimiento de modelos - IOT-23	39
5.5	Hiperparámetros ganadores de los modelos evaluados con ToN-IoT	42
5.6	Comparativa global de rendimiento de modelos - IOT-23	42
5.7	Comparativa de F1-score y throughput de inferencia pura para UNSW-NB15	45
5.8	Comparativa de F1-score y throughput de inferencia pura para IoT-23	46
5.9	Comparativa de F1-score y throughput de inferencia pura para TON-IoT	46
5.10	Cargas de red de referencia expresadas en paquetes por segundo	47
6.1	F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo MLP en UNSW-NB15	62
6.2	F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo CNN en UNSW-NB15	63
6.3	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP en UNSW-NB15	63

6.4	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN en UNSW-NB15	63
6.5	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	64
6.6	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	64
6.7	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	65
6.8	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	65
6.9	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	66
6.10	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	66
6.11	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	66
6.12	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	67
7.1	Comparativa final de modelos Random Forest en el set de test	74
7.2	Comparativa final de modelos XGBoost en el set de test	75
7.3	Comparativa final de modelos LightGBM en el set de test	76
7.4	Comparativa final de modelos CatBoost en el set de test	77
7.5	Comparativa final de modelos SVM en el set de test	78
7.6	Comparativa final de modelos MLP en el set de test	78
7.7	Comparativa final de modelos CNN en el set de test	80
7.8	Comparativa final de modelos Random Forest en el set de test (IOT-23)	81
7.9	Comparativa final de modelos XGBoost en el set de test (IOT-23)	82
7.10	Comparativa final de modelos LightGBM en el set de test (IOT-23)	83
7.11	Comparativa final de modelos CatBoost en el set de test (IOT-23)	84
7.12	Comparativa final de modelos SVM en el set de test (IOT-23)	85
7.13	Comparativa final de modelos MLP en el set de test (IOT-23)	86
7.14	Comparativa final de modelos CNN-1D en el set de test (IOT-23)	87
7.15	Comparativa final de modelos Random Forest en el set de test (ToN-IoT)	88
7.16	Comparativa final de modelos XGBoost en el set de test (ToN-IoT)	89
7.17	Comparativa final de modelos LightGBM en el set de test (ToN-IoT)	90
7.18	Comparativa final de modelos CatBoost en el set de test (ToN-IoT)	91
7.19	Comparativa final de modelos SVM en el set de test (ToN-IoT)	93
7.20	Comparativa final de modelos MLP en el set de test (ToN-IoT)	94
7.21	Comparativa final de modelos CNN-1D en el set de test (ToN-IoT)	95

1	Optimización multiobjetivo con validación cruzada estratificada	23
---	---	----

1

Introducción

En esta sección se presenta la motivación para la elaboración de este proyecto, los objetivos a alcanzar, la metodología seguida y la estructura del documento.

1.1 Motivación

En el ámbito de la ciberseguridad, los sistemas de detección de intrusiones (*Intrusion Detection Systems*, IDS) desempeñan un papel fundamental en la identificación temprana de ataques y actividades maliciosas en redes informáticas. En los últimos años, el uso de técnicas de inteligencia artificial y aprendizaje automático (*Machine Learning*) ha permitido mejorar significativamente la precisión y la capacidad de adaptación de estos sistemas frente a amenazas cada vez más complejas y dinámicas.

Sin embargo, gran parte de la investigación existente se ha centrado principalmente en mejorar la tasa de detección o reducir los falsos positivos, sin considerar en la misma medida el impacto computacional que conllevan estos modelos. En entornos donde el procesamiento debe realizarse en tiempo real, como redes corporativas, sistemas embebidos, dispositivos IoT o infraestructuras críticas, el consumo de recursos, la memoria utilizada y la latencia de inferencia se convierten en factores determinantes para la viabilidad de la solución, relacionado con la tendencia *Green AI*, que aboga por el desarrollo de modelos de inteligencia artificial más eficientes y sostenibles desde el punto de vista energético [green_ai].

Por este motivo, resulta necesario analizar no solo la capacidad predictiva de los modelos, sino también su eficiencia y su comportamiento ante escenarios adversos. En este contexto, este Trabajo Fin de Grado plantea una evaluación experimental de distintos modelos de aprendizaje automático y aprendizaje profundo aplicados a la detección de intrusiones, considerando tanto su rendimiento en condiciones normales como su robustez frente a perturbaciones adversarias.

1.2 Objetivos

El objetivo principal de este Trabajo de Fin de Grado es evaluar de forma integral el rendimiento, la robustez y la viabilidad operativa de diferentes modelos de aprendizaje automático y profundo aplicados a Sistemas de Detección de Intrusiones (IDS). Se busca analizar el compromiso (trade-off)

existente entre la capacidad predictiva de los algoritmos frente a amenazas camufladas y el coste computacional requerido para su ejecución en entornos con recursos limitados.

1.3 Metodología de trabajo

Para la elaboración de este TFG, se ha seguido una metodología estructurada siguiendo un enfoque iterativo y experimental. Dada la naturaleza exploratoria del proyecto, se ha seguido una metodología mixta, combinando el modelo CRISP-DM con un enfoque experimental orientado al análisis de rendimiento. Esta metodología se divide en las siguientes fases:

- Estudio del estado del arte sobre aprendizaje automático y sistemas de detección de intrusiones.
- Obtención y análisis de los *datasets*, evaluando su viabilidad para el proyecto.
- Implementación, evaluación y búsqueda de hiperparámetros de modelos de Machine Learning y Deep Learning buscando un equilibrio óptimo entre detección y uso de recursos.
- Aplicación de ataques de evasión a los modelos ganadores
- Análisis y conclusiones de los resultados obtenidos

1.4 Estructura del documento

Esta memoria está estructurada en los siguientes capítulos:

- **Capítulo 1. Introducción:** se presenta una introducción del proyecto, a través de su motivación, los objetivos y la metodología de trabajo a seguir.
- **Capítulo 2. Investigación previa:** se tratan los conceptos básicos para entender el objetivo del proyecto. Además, se explica el contexto y estado actual de la detección de anomalías a través de aprendizaje automático.
- **Capítulo 3. Tecnologías utilizadas:** se describen las tecnologías utilizadas por el proyecto.
- **Capítulo 4. Obtención y Análisis de Datos:** se detallan los *datasets* elegidos con un análisis de ellos.
- **Capítulo 5. Análisis de Modelos y Búsqueda de Hiperparámetros:** se explica el proceso seguido para la búsqueda de los mejores hiperparámetros de cada modelo, junto con sus métricas y resultados.
- **Capítulo 6. Ataques de Evasión:** se tratan los modelos elegidos y se exponen ante ataques adversarios para ver la degradación de la eficacia de los modelos.
- **Capítulo 7. Conclusiones y líneas futuras:** se reúnen conclusiones obtenidas tras la realización del trabajo, además de caminos de interés a seguir para trabajos futuros.

2

Investigación Previa

2.1 Machine Learning

El *Machine Learning* (aprendizaje automático) es una rama de la inteligencia artificial que permite a los sistemas aprender y mejorar automáticamente a partir de la experiencia, sin ser programados explícitamente para cada tarea. Utiliza algoritmos que identifican patrones en grandes volúmenes de datos y toman decisiones o realizan predicciones basadas en esos patrones.

En el contexto de los Sistemas de Detección de Intrusos (IDS), el *Machine Learning* se emplea para analizar el tráfico de red y detectar comportamientos anómalos o maliciosos de manera más eficiente y precisa que los métodos tradicionales basados en firmas. Además, la relación con la *Green AI* surge porque el uso de técnicas de aprendizaje automático más eficientes puede reducir el consumo energético, promoviendo soluciones más sostenibles.

2.1.1 Random Forest (RF)

Random Forest es un modelo de aprendizaje supervisado que combina múltiples árboles de decisión para obtener una predicción más robusta que la de un único árbol. En lugar de entrenar todos los árboles con exactamente los mismos datos, cada uno se construye a partir de una muestra diferente del conjunto de entrenamiento, generada mediante *bootstrap sampling* (muestreo con reemplazo). Además, en cada división se selecciona un subconjunto aleatorio de características. Esta combinación de aleatoriedad en los datos y en las variables es clave para que el modelo generalice mejor.

A diferencia de los métodos de *boosting*, como XGBoost, los árboles que componen un Random Forest no se entrenan de manera secuencial ni dependen unos de otros. Cada árbol aprende por su cuenta y aporta su propia "opinión", que posteriormente se integra con la del resto, normalmente mediante votación mayoritaria en clasificación. Esta independencia reduce la varianza del modelo y lo hace menos sensible al ruido o a pequeñas variaciones en los datos, algo especialmente útil en escenarios como la detección de intrusiones [rf_xgboost].

Tabla 2.1. Principales hiperparámetros de Random Forest y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles en el bosque
max_depth	Profundidad máxima de cada árbol
min_samples_split	Mínimo de muestras para dividir un nodo
min_samples_leaf	Mínimo de muestras en una hoja
max_features	Número de características consideradas en cada división
bootstrap	Si se usan muestras con reemplazo para entrenar cada árbol
criterion	Función para medir la calidad de una división (por ejemplo, <i>gini</i> o <i>entropy</i>)

2.1.2 Extreme Gradient Boosting (XGBoost)

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa de forma altamente optimizada el principio de *gradient boosting* (técnica de ensemble que combina modelos débiles secuencialmente, corrigiendo errores anteriores para mejorar la predicción). A diferencia de otros métodos basados en ensamblados de árboles, XGBoost construye el modelo de manera secuencial, añadiendo nuevos árboles que se centran en corregir los errores cometidos por los árboles previamente entrenados. Este proceso se basa en la optimización iterativa de una función objetivo que combina el error de clasificación con términos explícitos de regularización, lo que permite controlar la complejidad del modelo y reducir el sobreajuste.

En cada iteración, XGBoost utiliza información de primer (dirección) y segundo (qué tan pronunciado es el plano) orden del gradiente de la función de pérdida para determinar la estructura óptima de los árboles, lo que le permite capturar relaciones no lineales complejas e interacciones entre características de forma eficiente. Estas propiedades lo convierten en una opción especialmente adecuada para la detección de intrusiones en conjuntos de datos tabulares, donde los patrones maliciosos suelen manifestarse a través de combinaciones no triviales de múltiples variables.

En XGBoost, la predicción final del modelo no corresponde a la salida de un único árbol de decisión, sino que se obtiene como la combinación aditiva de las salidas de todos los árboles entrenados. Cada árbol aporta una contribución parcial a la predicción, y el resultado final se calcula sumando dichas contribuciones. El proceso de entrenamiento es secuencial: cada nuevo árbol se entrena para corregir los errores residuales cometidos por el conjunto de árboles previamente construidos. Como consecuencia, el último árbol no contiene una decisión completa por sí mismo, sino que únicamente modela aquellos patrones que aún no han sido correctamente capturados por los árboles anteriores, actuando como un ajuste fino del modelo global.

Tabla 2.2. Principales hiperparámetros de XGBoost y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles a construir
max_depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
subsample	Proporción de muestras usadas en cada iteración
colsample_bytree	Proporción de características usadas por árbol
gamma	Mínima reducción de pérdida para dividir un nodo
reg_alpha	Término de regularización L1
reg_lambda	Término de regularización L2
objective	Función objetivo a optimizar (por ejemplo, <i>binary:logistic</i>)

2.1.3 Light Gradient Boosting Machine (LightGBM)

Light Gradient Boosting Machine (LightGBM) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*. A diferencia de otros métodos de *boosting*, LightGBM utiliza un enfoque de crecimiento de árboles basado en hojas, lo que significa que en cada iteración se expande la hoja con el mayor error en lugar de expandir todos los nodos a un nivel determinado. Esta estrategia permite construir árboles más profundos y complejos sin aumentar significativamente el número de nodos, lo que mejora la capacidad de modelado y reduce el tiempo de entrenamiento [lightgbm].

Tabla 2.3. Principales hiperparámetros de LightGBM y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles a construir
num_leaves	Número máximo de hojas en cada árbol
max_depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
n_jobs	Número de threads para el entrenamiento paralelo
random_state	Semilla para reproducibilidad
verbosity	Nivel de detalle en los mensajes de salida

2.1.4 CatBoost

CatBoost es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*, desarrollada por Yandex.

Una de las principales características de CatBoost es su capacidad para manejar de manera eficiente variables categóricas sin necesidad de realizar una codificación previa, lo que simplifica el proceso de preparación de datos y mejora el rendimiento del modelo.

Otro punto clave es que a diferencia de otros modelos como XGBoost o LightGBM, CatBoost crea árboles simétricos, lo que nos da una latencia de inferencia muy baja, ya que cada árbol tiene la misma profundidad y se puede recorrer de manera más eficiente.

Tabla 2.4. Principales hiperparámetros de CatBoost y su significado.

Hiperparámetro	Significado
<code>iterations</code>	Número de árboles a construir
<code>depth</code>	Profundidad máxima de cada árbol
<code>learning_rate</code>	Tasa de aprendizaje (contribución de cada árbol)
<code>cat_features</code>	Índices de características categóricas
<code>loss_function</code>	Función de pérdida a optimizar
<code>random_seed</code>	Semilla para reproducibilidad

2.1.5 Support Vector Machine (SVM)

Support Vector Machines (SVM) es un algoritmo de aprendizaje supervisado cuyo objetivo principal es encontrar una frontera de decisión que separe de forma óptima las distintas clases en el espacio de características. En su formulación básica, SVM busca el hiperplano que maximiza el margen entre las muestras de distintas clases, utilizando únicamente un subconjunto de los datos denominado *vectores soporte*. Esta propiedad permite construir modelos robustos frente al ruido y con buena capacidad de generalización.

Una de las principales fortalezas de las SVM es el uso de *kernels*, que permiten proyectar los datos originales a espacios de mayor dimensionalidad sin realizar explícitamente dicha transformación. Gracias a este mecanismo, SVM puede modelar fronteras de decisión no lineales incluso cuando los datos no son separables linealmente en el espacio original. Entre los *kernels* más utilizados se encuentran:

- **Kernel lineal:** Define una frontera de decisión lineal en el espacio de características original. Es el *kernel* más simple y el más eficiente desde el punto de vista computacional, lo que lo hace especialmente adecuado para *datasets* de gran tamaño y para escenarios donde se prioriza la eficiencia.
- **Kernel polinómico:** Permite modelar relaciones no lineales mediante funciones polinómicas de distinto grado. Aunque puede capturar interacciones más complejas entre las características, su coste computacional aumenta rápidamente con el grado del polinomio y el tamaño del *dataset*.
- **Kernel radial (RBF):** Proyecta los datos a un espacio de dimensión infinita, permitiendo separar patrones altamente no lineales. Si bien suele ofrecer un alto rendimiento predictivo, su entrenamiento e inferencia son significativamente más costosos, especialmente en conjuntos de datos de gran tamaño.

En el contexto de este trabajo, el uso de SVM se plantea principalmente mediante un **kernel lineal**, ya que ofrece un compromiso adecuado entre rendimiento y eficiencia computacional [liblinear]. Los *kernels* no lineales, como el RBF, resultan poco escalables para los volúmenes de datos considerados y presentan un coste computacional elevado (el tiempo de entrenamiento escala al menos cuadráticamente con el número de muestras y puede ser impracticable con decenas de miles de muestras), lo que los hace menos compatibles con los principios de *Green AI* [nonlinear_more_complexity].

Tabla 2.5. Principales hiperparámetros de SVM y su significado.

Hiperparámetro	Significado
C	Parámetro de regularización que controla el equilibrio entre margen y error de clasificación
kernel	Tipo de función <i>kernel</i> utilizada (por ejemplo, <i>linear</i> , <i>poly</i> , <i>rbf</i>)
degree	Grado del polinomio (solo para <i>kernel</i> polinómico)
gamma	Coeficiente para <i>kernels rbf</i> , <i>poly</i> y <i>sigmoid</i>
coef0	Término independiente en <i>kernels poly</i> y <i>sigmoid</i>

2.2 Deep Learning

El *Deep Learning* (aprendizaje profundo) es un subárea del aprendizaje automático que se basa en redes neuronales artificiales con múltiples capas (redes neuronales profundas). Estas redes son capaces de aprender representaciones jerárquicas de los datos, lo que les permite capturar patrones complejos y realizar tareas como clasificación, reconocimiento de imágenes y procesamiento del lenguaje natural.

En el ámbito de los IDS, el *Deep Learning* se utiliza para mejorar la detección de intrusiones mediante la capacidad de aprender características complejas del tráfico de red. Sin embargo, es importante considerar que los modelos de aprendizaje profundo suelen requerir una gran cantidad de datos y recursos computacionales, lo que puede aumentar el consumo energético. Por lo tanto, es crucial buscar enfoques que optimicen el rendimiento sin comprometer la eficiencia energética.

2.2.1 Multilayer Perceptron (MLP)

El **Multi-Layer Perceptron** (MLP) es una red neuronal artificial de tipo *feed-forward* compuesta por una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa está formada por neuronas completamente conectadas, donde cada neurona realiza una combinación lineal de sus entradas seguida de una función de activación no lineal. Este tipo de arquitectura permite aproximar funciones no lineales complejas y es uno de los modelos más utilizados como punto de partida en problemas de aprendizaje profundo.

Desde el punto de vista computacional, su coste está directamente relacionado con el número de capas ocultas y el número de neuronas por capa. Arquitecturas profundas o excesivamente anchas

incrementan de forma notable el tiempo de entrenamiento y el consumo de memoria, sin garantizar necesariamente mejoras en el rendimiento para datos tabulares. Por este motivo, en este trabajo se utilizan arquitecturas MLP, con un número reducido de capas y neuronas, priorizando un equilibrio adecuado entre capacidad de modelado y eficiencia computacional [mlp_layers].

Tabla 2.6. Principales hiperparámetros de MLP y su significado.

Hiperparámetro	Significado
hidden_layer_sizes	Número y tamaño de las capas ocultas
activation	Función de activación utilizada en las neuronas (por ejemplo, <i>relu</i> , <i>tanh</i>)
solver	Algoritmo de optimización para el entrenamiento (por ejemplo, <i>adam</i> , <i>sgd</i>)
alpha	Parámetro de regularización L2
learning_rate_init	Tasa de aprendizaje inicial
batch_size	Tamaño del lote de muestras para cada actualización de los pesos
max_iter	Número máximo de iteraciones de entrenamiento
early_stopping	Si se detiene el entrenamiento anticipadamente al no mejorar el rendimiento

2.2.2 Convolutional Neural Networks (CNN)

Las **Convolutional Neural Networks** (CNN) son un tipo de red neuronal profunda diseñadas originalmente para el procesamiento de datos con estructura espacial, como imágenes o señales. Su principal característica es el uso de capas convolucionales, que aplican filtros locales sobre los datos de entrada para extraer patrones relevantes de forma jerárquica, reduciendo al mismo tiempo el número de parámetros respecto a redes completamente conectadas.

En el contexto de este trabajo, las CNN se emplean en su variante *1D*, donde las convoluciones se aplican sobre el vector de características de cada flujo de red, tratado como una señal unidimensional. Este enfoque permite capturar relaciones locales entre características adyacentes, modelando interacciones de corto alcance entre variables sin necesidad de introducir arquitecturas excesivamente complejas.

Desde el punto de vista computacional, el coste de una CNN depende principalmente del número de filtros, el tamaño del *kernel* y la profundidad de la red. Aunque las CNN suelen ser más eficientes que otras arquitecturas profundas al compartir pesos, un número elevado de filtros o capas puede incrementar significativamente el tiempo de entrenamiento y la latencia de inferencia.

Tabla 2.7. Principales hiperparámetros de CNN 1D y su significado.

Hiperparámetro	Significado
n_filters	Número de filtros en las capas convolucionales
kernel_size	Tamaño del <i>kernel</i> de convolución
dense_units	Número de neuronas en la capa densa final
time_steps	Número de pasos temporales considerados por la red
activation	Función de activación utilizada en las capas de la red
learning_rate	Tasa de aprendizaje del optimizador
batch_size	Tamaño del lote de muestras utilizado durante el entrenamiento
epochs	Número de épocas de entrenamiento

2.3 Justificación del enfoque en la fase de inferencia

En el contexto de los Sistemas de Detección de Intrusiones (IDS), resulta importante distinguir entre la **fase de entrenamiento** del modelo y la **fase de inferencia**.

Durante el entrenamiento, el modelo aprende patrones a partir de un conjunto de datos previamente etiquetado, ajustando sus parámetros internos con el objetivo de mejorar su capacidad de clasificación. Esta fase puede implicar un coste computacional elevado, especialmente en modelos complejos o con grandes volúmenes de datos, pero suele realizarse de forma puntual o con una frecuencia limitada. Por el contrario, la fase de inferencia corresponde al uso del modelo ya entrenado para clasificar nuevas muestras. En un IDS desplegado en un entorno real, esta fase se ejecuta de manera continua, analizando el tráfico de red entrante para identificar posibles comportamientos maliciosos. Por ello, aunque el entrenamiento pueda ser costoso, el coste asociado a la inferencia adquiere una relevancia especial, ya que se repite constantemente durante el funcionamiento del sistema.

Esta diferencia es especialmente importante en escenarios donde la detección debe realizarse en tiempo real o cerca del origen de los datos, como redes corporativas, dispositivos IoT, sistemas embebidos o infraestructuras críticas. En estos casos, un modelo con buen rendimiento predictivo puede no ser viable si presenta una latencia elevada, un consumo excesivo de memoria o una demanda computacional incompatible con el entorno de despliegue.

Por eso mismo, **el coste asociado a la inferencia es el que determina la viabilidad práctica del sistema**. Un modelo con excelente rendimiento predictivo pero con alta latencia o consumo excesivo de CPU y memoria puede resultar inviable para su despliegue en tiempo real. Así que, el presente trabajo **centra su análisis experimental en el consumo computacional durante la fase de inferencia**.

2.4 Métricas Utilizadas

No solo hemos comparado los modelos a través de su F1-Score y latencia, sino que también hemos realizado una evaluación global de su rendimiento en términos de latencia, *throughput*, uso de CPU y RAM. Para calcular las métricas de rendimiento, hemos usado los siguientes conceptos:

Métrica	Significado Técnico	Metodología de Código
F1-Score	Métrica de eficacia predictiva. Es la media armónica entre Precisión y Sensibilidad (Recall). Representa la capacidad del modelo para detectar ataques reales minimizando las falsas alarmas, siendo ideal para tráfico desbalanceado.	Calculada mediante <code>scikit-learn</code> (<code>f1_score</code>), cruzando las etiquetas reales del <i>dataset</i> de test frente a las predicciones generadas por el modelo.
Latencia (ms)	Tiempo de respuesta unitario. Representa los milisegundos que tarda el sistema en procesar y clasificar un único paquete desde entrada hasta veredicto.	Medida mediante <code>time.perf_counter()</code> al procesar el set de test por lotes, dividido entre el número total de paquetes procesados.
Throughput (paq/s)	Caudal de procesamiento. Representa el volumen de tráfico, expresado en paquetes por segundo, que el modelo puede soportar sin cuellos de botella.	Calculado dividiendo el número total de paquetes del conjunto de test entre el tiempo físico total de inferencia, también denominado <i>wall time</i> .
Incremento RAM (MB)	Huella máxima de memoria volátil. Representa la cantidad de memoria física extra que el sistema operativo cedió al proceso durante la predicción.	Monitorizado mediante <code>psutil.memory_info().rss</code> , restando la memoria base al pico máximo alcanzado durante la inferencia.
CPU Efectiva (%)	Porcentaje medio de uso efectivo de CPU durante la inferencia, normalizado por el número total de núcleos lógicos disponibles. Representa qué proporción de la capacidad total de CPU del sistema ha sido utilizada por el proceso durante la predicción.	Calculada a partir del tiempo de CPU consumido por el proceso, el tiempo real transcurrido y el número de núcleos lógicos disponibles: $\text{CPU efectiva} = \frac{T_{CPU}}{T_{real} \cdot N_{cores}} \cdot 100$ donde T_{CPU} es el tiempo acumulado de CPU consumido por el proceso, T_{real} el tiempo real medio transcurrido y N_{cores} el número de núcleos lógicos disponibles.

Tabla 2.8. Definición de métricas de evaluación de rendimiento

3

Tecnologías utilizadas

3.1 Lenguaje de Programación

Para el desarrollo, experimentación y análisis de este trabajo, hemos usado **Python** como lenguaje de programación principal. En la actualidad, Python se ha consolidado como el lenguaje estándar de la comunidad científica para proyectos de ciberseguridad o Inteligencia Artificial. En cuanto al desarrollo, el proyecto se ha estructurado y ejecutado utilizando **Jupyter Notebook** para mantener un flujo de trabajo ágil y ordenado [**vscode_jupyter_notebooks**]. Por ejemplo, esto nos permite renderizar tablas de resultados en la misma línea del código y probar diferentes parámetros sin tener que ejecutar el código completo.

3.2 Librerías Usadas

Las librerías de código usadas en este trabajo han sido las siguientes:

Tabla 3.1. Librerías utilizadas durante el desarrollo experimental

Librería	Objetivo principal
Optuna [akiba2019optuna]	Optimización de hiperparámetros para mejorar el rendimiento de los modelos.
NumPy [numpy]	Operaciones numéricas, manejo de arrays y cálculo eficiente sobre matrices.
Polars [polars]	Procesamiento eficiente de <i>datasets</i> tabulares con gran volumen de registros.
Pandas [pandas]	Manipulación de datos y creación de tablas sencillas de resultados.
Psutil [psutil]	Monitorización de recursos del sistema y apoyo en la medición del rendimiento.
Matplotlib [matplotlib]	Generación de gráficas y visualización de resultados experimentales.
Seaborn [seaborn]	Creación de visualizaciones estadísticas de forma más clara y legible.
TensorFlow/Keras [tensorflow]	Implementación, entrenamiento y evaluación de modelos de deep learning.
Scikit-learn [scikit_learn]	Preprocesamiento, métricas de evaluación y entrenamiento de modelos clásicos.
XGBoost [xgboost_library]	Entrenamiento y evaluación del modelo basado en <i>gradient boosting</i> .
LightGBM [lightgbm_library]	Entrenamiento eficiente de modelos de <i>boosting</i> sobre datos tabulares.
CatBoost [catboost_library]	Entrenamiento de modelos de <i>boosting</i> con buen rendimiento en datos tabulares.
Adversarial Robustness Toolbox [art_library]	Generación de ataques adversarios como FGSM y PGD.

3.3 Entorno de pruebas

Antes de analizar los resultados obtenidos, es importante describir el entorno de pruebas utilizado para la evaluación de los modelos. El entrenamiento de los modelos y la inferencia se ha llevado a cabo en un entorno controlado con las siguientes características:

Componente	Especificación Técnica
Sistema Operativo	Debian GNU/Linux 12 (bookworm)
Procesador (CPU)	2x Intel® Xeon® Gold 5418Y (Arquitectura Dual-Socket)
Núcleos de CPU	48 físicos / 96 lógicos (Hilos)
Memoria RAM (Sistema)	512 GB (503 GiB disponibles)
Aceleradores Gráficos (GPU)	4x NVIDIA L40S
Memoria VRAM (Total)	192 GB (48 GB dedicados por cada GPU)
Plataforma de Cómputo	CUDA Version 13.0 (Driver 580.95.05)

Tabla 3.2. Especificaciones del entorno experimental

Además, también adjuntamos las características del equipo personal, ya que también se ha usado en la inferencia de los modelos, para comparar tiempos y probar los modelos en un entorno más realista:

Componente	Especificación técnica
Sistema Operativo	Linux Mint 22 Wilma
Procesador (CPU)	11th Gen Intel® Core™ i7-1165G7 @ 2.80GHz
Núcleos de CPU	4 físicos / 8 lógicos
Memoria RAM	15 GiB
Aceleradores Gráficos (GPU)	No disponible para cómputo dedicado
Memoria VRAM	No disponible
Plataforma de Cómputo	CPU local / inferencia sin CUDA

Tabla 3.3. Características técnicas del equipo local utilizado para la inferencia

3.4 Optuna

Optuna es un framework de optimización de hiperparámetros de código abierto que permite a los desarrolladores y científicos de datos encontrar automáticamente los mejores hiperparámetros para sus modelos de aprendizaje automático [akiba2019optuna]. Optuna utiliza técnicas de optimización bayesiana para explorar el espacio de hiperparámetros de manera eficiente, lo que puede mejorar significativamente el rendimiento de los modelos [optuna].

A diferencia de *Grid Search*, que realiza una búsqueda exhaustiva y costosa, o *Random Search*, que no aprende de iteraciones pasadas, Optuna utiliza un enfoque bayesiano basado en el *Tree-structured Parzen Estimator* (TPE).

El algoritmo TPE construye un modelo probabilístico de la función objetivo. En lugar de probar combinaciones al azar, predice qué regiones del espacio de búsqueda tienen más probabilidades de mejorar los resultados actuales.

Otra de las ventajas de este framework es que nos permite establecer una **optimización multiobjetivo** [opt_multiobjective], lo que es especialmente útil cuando se desea optimizar varias métricas al mismo tiempo, como la precisión y eficiencia del modelo.

En un problema multiobjetivo no existe necesariamente una única solución óptima, sino un conjunto de soluciones no dominadas que representan distintos compromisos entre los objetivos. Este conjunto se conoce como **conjunto Pareto-óptimo**, y su representación en el espacio de objetivos constituye la **frontera de Pareto** [deb2001multiobjective].

Para estructurar la búsqueda de hiperparámetros y garantizar una evaluación justa del compromiso entre rendimiento y tiempo de inferencia, en este proyecto se ha diseñado el flujo de trabajo descrito en el Algoritmo 1. Aunque cada arquitectura tiene un espacio de hiperparámetros específico, la estrategia de evaluación se mantiene constante: cada configuración se evalúa mediante validación cruzada estratificada [sklearn_stratifiedkfold] y se optimiza simultáneamente el rendimiento predictivo y el coste de inferencia.

Algoritmo 1 Optimización multiobjetivo con validación cruzada estratificada

Entrada: Conjunto de entrenamiento completo $D_{train} = (X, y)$, modelo base M , espacio de hiperparámetros $\mathcal{H}$, número de ensayos N

Salida: Conjunto de configuraciones evaluadas y frontera de Pareto

- 1: Crear un estudio de Optuna con dos objetivos:
maximizar $F1$ y minimizar la latencia media de inferencia
 - 2: Definir una validación cruzada estratificada con $K = 3$ particiones
 - 3: **for** cada ensayo $t = 1, \dots, N$ **do**
 - 4: Seleccionar una configuración de hiperparámetros $h_t \in \mathcal{H}$
 - 5: Inicializar las listas $F1_scores$ y $Latencias$
 - 6: **for** cada partición $(D_{train}^{(k)}, D_{val}^{(k)})$ de la validación cruzada **do**
 - 7: Instanciar el modelo M_t con los hiperparámetros h_t
 - 8: Entrenar M_t sobre $D_{train}^{(k)}$
 - 9: Obtener las predicciones sobre $D_{val}^{(k)}$
 - 10: Calcular el $F1$ binario de la clase maliciosa
 - 11: Añadir el valor obtenido a $F1_scores$
 - 12: Seleccionar un subconjunto de validación para medir latencia
 - 13: Realizar una predicción inicial de calentamiento
 - 14: Medir varias repeticiones del tiempo de inferencia
 - 15: Calcular la latencia media por muestra en milisegundos
 - 16: Añadir la latencia obtenida a $Latencias$
 - 17: **end for**
 - 18: Calcular $F1_{medio}$ como la media de $F1_scores$
 - 19: Calcular $Latencia_{media}$ como la media de $Latencias$
 - 20: Devolver a Optuna el par $(F1_{medio}, Latencia_{media})$
 - 21: **end for**
 - 22: Obtener la frontera de Pareto a partir de todos los ensayos evaluados
-

4

Obtención y Análisis de Datos

Para cumplir con los objetivos de este Trabajo Fin de Grado, resulta imprescindible realizar una selección cuidadosa de los conjuntos de datos empleados en la fase experimental.

En particular, dado que este trabajo no se centra únicamente en la capacidad de detección, sino también en el rendimiento y el consumo de recursos computacionales, los conjuntos de datos seleccionados deben posibilitar el análisis de aspectos como la latencia de inferencia, el uso de CPU y memoria, y la escalabilidad de los modelos en distintos escenarios. El uso de *datasets* excesivamente simplificados o poco representativos podría conducir a conclusiones poco realistas, mientras que el empleo de conjuntos de datos desproporcionadamente grandes o complejos podría dificultar la reproducibilidad y el análisis experimental detallado.

Por este motivo, en este proyecto se han seleccionado *datasets* ampliamente utilizados en la literatura científica, que cubren diferentes entornos y tipologías de ataque, y que ofrecen un equilibrio adecuado entre realismo, diversidad y viabilidad experimental. Esta elección permite evaluar de forma comparativa el comportamiento de distintos modelos de aprendizaje automático y profundo.

4.1 UNSW-NB15 *DataSet*

4.1.1 Motivación para el uso del *dataset*

El conjunto de datos UNSW-NB15 se ha seleccionado como *dataset* principal en este trabajo con el objetivo de proporcionar un escenario más realista y actualizado para la evaluación de sistemas IDS [unsw-nb15]. Este *dataset* fue desarrollado por la *University of New South Wales* (UNSW).

4.1.2 Estructura del *dataset*

El *dataset* UNSW-NB15 está compuesto por registros de tráfico de red generados mediante la combinación de tráfico real y tráfico sintético, con el fin de simular de forma controlada distintos escenarios

de ataque en una red corporativa. El conjunto de datos se distribuye en dos archivos principales claramente diferenciados: uno destinado al entrenamiento de los modelos y otro reservado para su evaluación, lo que facilita la reproducibilidad experimental y minimiza el riesgo de *data leakage*.

Cada registro representa un flujo de red y se describe mediante un conjunto de características heterogéneas que incluyen métricas relacionadas con el volumen de tráfico, estadísticas temporales, información de protocolos y atributos derivados del comportamiento de la comunicación. En cuanto a las etiquetas, el *dataset* incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que especifica el tipo de ataque, lo que permite abordar tanto problemas de clasificación binaria como multiclase.

4.1.3 Distribución de ataques

Tabla 4.1. Distribución de clases y tipos de ataque en el dataset UNSW-NB15

Clase / Tipo de tráfico	Número de flujos
BENIGN	93 000
Generic	58 871
Exploits	44 525
Fuzzers	24 246
DoS	16 353
Reconnaissance	13 987
Analysis	2 677
Backdoor	2 329
Shellcode	1 511
Worms	174
Total	257 673

Como puede observarse, aunque el tráfico benigno constituye una parte relevante del *dataset*, una proporción significativa de los registros corresponde a tráfico malicioso, distribuido entre múltiples categorías de ataque. Destacan especialmente las clases *Generic*, *Exploits* y *Fuzzers*, mientras que otras, como *Worms* o *Shellcode*, presentan una frecuencia mucho menor. Esta distribución refleja un escenario realista y resulta adecuada para evaluar el comportamiento de los modelos tanto en términos de detección general de intrusiones como de clasificación específica de ataques.

4.2 ToN_IoT DataSet

4.2.1 Motivación para el uso del dataset

El conjunto de datos ToN_IoT (*Telemetry of Network IoT*) se ha incluido en este trabajo con el objetivo de evaluar el comportamiento de los modelos de detección de intrusiones en entornos IoT y *edge computing*, caracterizados por una elevada heterogeneidad, recursos computacionales limitados y requisitos estrictos de eficiencia. Este *dataset* fue desarrollado por la *University of New South*

Wales (UNSW) con el propósito de cubrir las limitaciones de los conjuntos de datos tradicionales, que no representan adecuadamente el tráfico ni los patrones de ataque presentes en infraestructuras IoT reales [ton-iot].

4.2.2 Estructura del dataset

El dataset ToN_IoT se distribuye en múltiples subconjuntos que incluyen tráfico de red. En concreto, se ha empleado el archivo `Train_Test_Network_dataset.csv`, que proporciona los datos en formato CSV e incluye una partición predefinida de entrenamiento y prueba. Cada registro representa un flujo de red y se describe mediante un conjunto de características extraídas automáticamente, que abarcan métricas relacionadas con el volumen de tráfico, estadísticas temporales, información de protocolos y atributos derivados del comportamiento de la comunicación.

En cuanto al etiquetado, el dataset incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que identifica el tipo de ataque. Esta estructura permite abordar tanto el problema de detección binaria de intrusiones como el análisis de los distintos tipos de tráfico malicioso presentes en entornos IoT.

4.2.3 Distribución de ataques

La distribución de clases del subconjunto de red de ToN_IoT presenta una proporción equilibrada entre tráfico benigno y varias categorías de ataques, lo que resulta especialmente interesante para evaluar el comportamiento de los modelos en un escenario IoT controlado. En la Tabla 4.2 se muestra el número total de flujos correspondiente a cada clase, considerando el conjunto completo de datos.

Como puede observarse, el dataset incluye múltiples tipos de ataques representativos de entornos IoT, tales como ataques de denegación de servicio, inyección, escaneo, ransomware y ataques de intermedio (Man-in-the-Middle).

Tabla 4.2. Distribución de clases y tipos de ataque en el dataset ToN_IoT

Clase / Tipo de tráfico	Número de flujos
BENIGN	50 000
backdoor	20 000
ddos	20 000
dos	20 000
injection	20 000
password	20 000
ransomware	20 000
scanning	20 000
xss	20 000
mitm	1 043
Total	191 043

4.3 IoT-23 DataSet

4.3.1 Motivación para el uso del dataset

El conjunto de datos **IoT-23** se ha incorporado en este trabajo como *dataset* complementario orientado a entornos IoT reales. IoT-23, desarrollado por el grupo *Stratosphere IPS* de la *Czech Technical University (CTU)*, es uno de los conjuntos de datos más representativos para el estudio de amenazas dirigidas específicamente a dispositivos IoT [iot-23].

4.3.2 Estructura del dataset

IoT-23 se distribuye en forma de escenarios independientes, donde cada escenario corresponde a una captura concreta asociada a un tipo específico de malware o comportamiento malicioso. Estos escenarios se almacenan en directorios separados y presentan tamaños muy variables, desde unos pocos megabytes hasta decenas de gigabytes, lo que dificulta su uso directo en entornos experimentales con recursos limitados.

Con el fin de garantizar la viabilidad experimental y mantener la reproducibilidad de los experimentos, en este trabajo se ha seleccionado un subconjunto reducido de escenarios representativos. En concreto, se han utilizado los archivos `capture-1-1.labeled` y `capture-3-1.labeled`, que presentan un tamaño manejable y contienen tráfico IoT real con actividad maliciosa claramente identificable.

4.3.3 Distribución de ataques

La distribución de clases en los escenarios seleccionados de IoT-23 presenta un fuerte desbalanceo entre tráfico benigno y tráfico malicioso, así como una predominancia de determinados comportamientos de ataque característicos de dispositivos IoT comprometidos. En la Tabla 4.3 se muestra el número total de flujos correspondiente a cada clase y tipo de tráfico, considerando conjuntamente los archivos `capture-1-1.labeled` y `capture-3-1.labeled`.

Tabla 4.3. Distribución de clases y tipos de tráfico en el dataset IoT-23

Clase / Tipo de tráfico	Número de flujos
Malicious – PartOfAHorizontalPortScan	685 062
Benign	473 811
Malicious Attack	5 962
Malicious – C&C	16
Total	1 164 851

Como puede observarse, el tráfico malicioso asociado a escaneos horizontales de puertos domina el conjunto de datos.

4.4 Relación entre los *datasets*

Los tres *datasets* proporcionan características extraídas a nivel de flujo de red (estadísticas de paquetes, bytes, duraciones, banderas TCP) a partir de capturas reales. Todos ellos reflejan la realidad de las redes actuales, donde el tráfico normal (benigno) suele ser la clase mayoritaria. Debido a este desbalanceo, usamos como métrica principal comparativa el F1-score. Todos los *datasets* son *datasets* reconocidos en la literatura de los análisis de IDS, ya que están diseñados específicamente para el entrenamiento y auditoría de algoritmos de Machine Learning y Deep Learning.

No obstante, también se han elegido porque presentan algunas diferencias entre ellos y así poder ver los resultados de *datasets* que tienen distinta distribución y topología. Por ejemplo, UNSW-NB15 está enfocado en una red corporativa tradicional y sirve como punto de referencia base para comprobar que nuestros modelos sepan cazar amenazas usuales.

Por otro lado, los *datasets* IoT-23 y ToN-IoT están enfocados en entornos puramente de IoT donde las capturas provienen de amenazas de infecciones reales de malware en dispositivos inteligentes.

A continuación se muestra una tabla simplificada que resume las características principales de UNSW-NB15:

Tabla 4.4. Correspondencia principal entre características de UNSW-NB15

Categoría	UNSW-NB15
Duración flujo	dur
Intervalo paquetes origen	sinpkt
Intervalo paquetes destino	dinpkt
Paquetes origen	spkts
Paquetes destino	dpkts
Bytes origen	sbytes
Bytes destino	dbytes
Tasa paquetes	rate
Tamaño medio paquetes origen	smean
Tamaño medio paquetes destino	dmean
SYN flag	synack
ACK flag	ackdat
Etiqueta	label / attack_cat

Ahora relacionamos los *datasets* ToN_IoT y IoT-23, destacando las características comunes.

Tabla 4.5. Correspondencia principal entre características de ToN_IoT e IoT-23

Categoría	ToN_IoT	IoT-23	Descripción
IP origen	src_ip	id.orig_h	Dirección IP origen
Puerto origen	src_port	id.orig_p	Puerto origen
IP destino	dst_ip	id.resp_h	Dirección IP destino
Puerto destino	dst_port	id.resp_p	Puerto destino
Protocolo	proto	proto	Protocolo de transporte
Duración	duration	duration	Duración de la conexión
Bytes origen	src_bytes	orig_bytes	Bytes enviados por el origen
Bytes destino	dst_bytes	resp_bytes	Bytes enviados por el destino
Paquetes origen	src_pkts	orig_pkts	Paquetes enviados por el origen
Paquetes destino	dst_pkts	resp_pkts	Paquetes enviados por el destino
Estado	conn_state	conn_state	Estado de la conexión
Etiqueta	label	label	Tráfico benigno o malicioso
Tipo ataque	type	detailed-label	Tipo de ataque

5

Análisis de Modelos y Búsqueda de Hiperparámetros

En este capítulo se presentan los resultados experimentales obtenidos tras el entrenamiento, optimización y evaluación de los modelos considerados en este trabajo. Primero se describen los procedimientos seguidos para la optimización de hiperparámetros estructurales y posteriormente se muestran los resultados obtenidos para cada modelo entrenado con los *datasets*. Se incluyen comparativas globales de rendimiento, curvas ROC y matrices de confusión para cada modelo para observar mejor la comparativa.

5.1 Metodología experimental

En esta sección se describe la metodología seguida para entrenar, optimizar y evaluar los modelos considerados en este trabajo. El objetivo, como hemos visto, no es únicamente comparar los modelos en términos de rendimiento predictivo, sino también analizar su eficiencia computacional durante la fase de inferencia, lo que define una **búsqueda multiobjetivo** entre precisión y eficiencia.

5.1.1 Flujo de trabajo experimental

En el trabajo se ha seguido un flujo de trabajo común para todos los modelos y *datasets*:

1. Para cada modelo, se ha definido un espacio de búsqueda de hiperparámetros específico, adaptado a las características de cada algoritmo.
2. Se ha utilizado validación cruzada para evaluar el rendimiento de cada configuración de hiperparámetros, asegurando una evaluación robusta y representativa del modelo.
3. Se han optimizado simultáneamente dos objetivos: el F1-Score (para maximizar la precisión) y la latencia de inferencia (para minimizar el tiempo de respuesta).

4. Se ha identificado la frontera de Pareto para cada modelo, lo que permite visualizar el compromiso entre precisión y eficiencia y seleccionar configuraciones que se ajusten a diferentes necesidades.
5. Finalmente, se han evaluado los candidatos seleccionados de la frontera de Pareto en el conjunto de test para obtener una comparativa final de su rendimiento y latencia.

Para la evaluación final de los modelos seleccionados se utilizó un procedimiento de *benchmarking* instrumentado. En primer lugar, el conjunto de test se dividió en bloques o paquetes de muestras, de forma que la inferencia no se realizó mediante una única llamada global al modelo, sino procesando secuencialmente dichos bloques. Esta decisión permitió monitorizar el proceso durante la predicción y obtener métricas adicionales de consumo, como el incremento de memoria RAM.

5.1.2 Optimización con Optuna

Antes de presentar los resultados finales, es importante señalar que el proceso completo de selección de hiperparámetros se recoge en el Apéndice A. En dicho apéndice se incluyen las fronteras de Pareto y las tablas completas de candidatos evaluados para cada combinación de modelo y *dataset*.

La optimización de hiperparámetros no se ha limitado a una selección manual de configuraciones, sino que se ha realizado de forma sistemática para cada combinación de modelo y *dataset*. En total, se ha aplicado el proceso de búsqueda sobre siete familias de modelos y tres conjuntos de datos distintos, generando para cada caso una frontera de Pareto basada en dos objetivos contrapuestos: maximizar el F1-Score y minimizar la latencia de inferencia.

Este enfoque permite poner en valor el compromiso central del trabajo: un modelo no se considera mejor únicamente por alcanzar una mayor puntuación predictiva, sino por ofrecer una relación adecuada entre calidad de detección y coste computacional. Por este motivo, tras la fase de búsqueda se evaluaron distintos candidatos de la frontera de Pareto en el conjunto de test, seleccionando finalmente aquellos que mantenían un F1-Score elevado sin introducir incrementos de latencia desproporcionados.

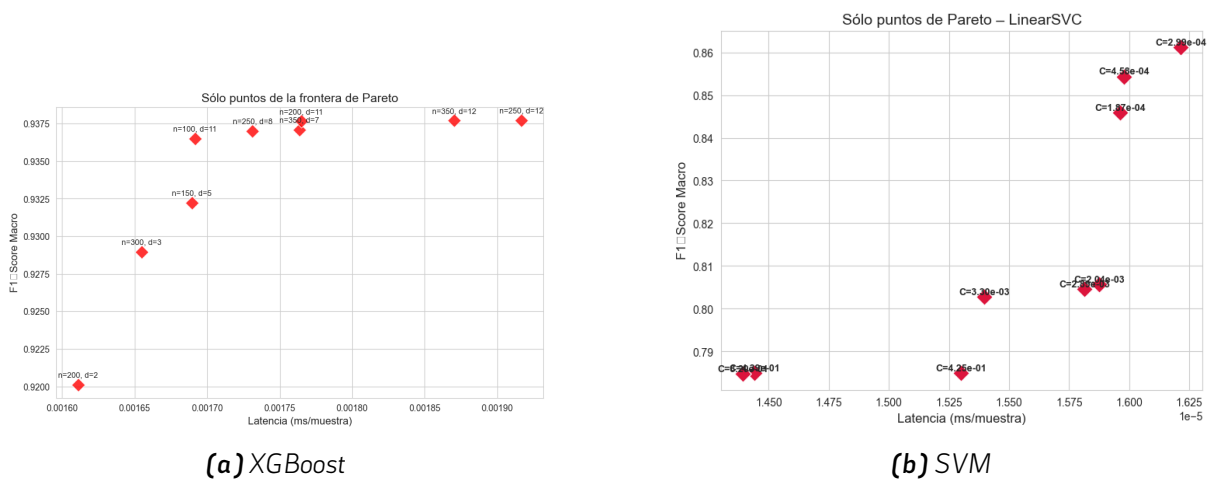


Figura 5.1. Fronteras de Pareto para XGBoost y SVM

La Figura 5.1, imágenes recogidas del Apéndice A, muestra dos comportamientos representativos observados durante la optimización. En el caso de XGBoost, se aprecia una tendencia de saturación (curva exponencial): las primeras mejoras en F1-Score se obtienen con un coste temporal reducido, pero a partir de cierto punto las mejoras predictivas son cada vez menores frente al aumento de latencia. Este tipo de comportamiento aparece en la mayoría de los modelos analizados.

Por el contrario, algunas gráficas, como la que vemos en la Figura 5.1b muestra una frontera de Pareto menos marcada. Los puntos aparecen más agrupados y las diferencias de latencia entre configuraciones son muy pequeñas, por lo que no se observa una curva de saturación tan clara. En este caso, la elección del candidato depende en mayor medida del F1-Score, ya que el coste temporal apenas varía entre las configuraciones evaluadas.

Esta comparación permite observar que el proceso de optimización no produce siempre el mismo tipo de frontera. En algunos modelos existe un compromiso evidente entre rendimiento y latencia, mientras que en otros la variación de coste computacional es tan reducida que la frontera resulta menos expresiva. Por ello, la selección final de candidatos se ha realizado mediante una observación conjunta de las métricas, y no aplicando una regla automática basada únicamente en el F1-Score máximo.

La elección del **candidato ganador** se ha realizado mediante una observación conjunta de la frontera de Pareto. No se ha seleccionado siempre el candidato con mayor F1-score, sino aquel que ofrecía el mejor equilibrio entre F1-score y latencia.

5.2 Optimización de hiperparámetros estructurales

El ajuste de hiperparámetros con Optuna se centra específicamente en aquellos que determinan la **estructura del modelo**. Los hiperparámetros estructurales influyen directamente en la complejidad computacional, el número de operaciones necesarias para generar una predicción y el tamaño final del modelo.

A diferencia de otros hiperparámetros relacionados con el proceso de entrenamiento (como la tasa de aprendizaje o el optimizador), los **hiperparámetros estructurales determinan la arquitectura final del modelo** y, por tanto, pueden influir de forma directa o relevante en su coste en tiempo de ejecución una vez desplegado.

A continuación, se describen los hiperparámetros estructurales más relevantes para cada uno de los modelos considerados en este trabajo.

5.2.1 Random Forest

En el caso de Random Forest, el coste de inferencia depende principalmente de:

- **n_estimators**: número de árboles que componen el bosque. Un mayor número de árboles

incrementa linealmente el número de recorridos necesarios para cada predicción.

- **max_depth**: profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones por predicción.

5.2.2 XGBoost

En XGBoost, la inferencia consiste en recorrer secuencialmente todos los árboles entrenados y sumar sus contribuciones. Los hiperparámetros estructurales más relevantes son:

- **n_estimators**: número total de árboles del modelo.
- **max_depth**: profundidad máxima de cada árbol.

5.2.3 LightGBM

En LightGBM, el coste de inferencia también está influenciado por:

- **n_estimators**: número de árboles en el modelo.
- **max_depth**: profundidad máxima de cada árbol.
- **num_leaves**: número máximo de hojas por árbol, que afecta la complejidad del modelo y el número de comparaciones necesarias para cada predicción.

5.2.4 CatBoost

En CatBoost, los hiperparámetros estructurales que afectan el coste de inferencia son:

- **iterations**: número de árboles a entrenar. Un mayor número de iteraciones incrementa linealmente el coste computacional en inferencia, ya que cada predicción requiere recorrer todos los árboles.
- **depth**: profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones y operaciones por predicción.

5.2.5 Support Vector Machine (SVM)

Para SVM, el coste de inferencia depende principalmente del tipo de *kernel* utilizado. En este trabajo se prioriza el uso de:

- **C**: parámetro de regularización que puede influir en la complejidad del modelo, aunque su impacto en la inferencia es menor que el tipo de *kernel*.

5.2.6 Multilayer Perceptron (MLP)

En redes MLP, el coste de inferencia está directamente relacionado con el número total de pesos del modelo. Los hiperparámetros estructurales clave son:

- **hidden_layer_sizes**: número de capas ocultas y número de neuronas por capa.

5.2.7 Convolutional Neural Networks (CNN 1D)

En las CNN 1D utilizadas en este trabajo, los hiperparámetros estructurales más relevantes son:

- **n_filters**: número de filtros en las capas convolucionales.
- **kernel_size**: tamaño del *kernel*.
- **dense_units**: número de unidades en la capa densa.

5.3 Modelos entrenados con UNSW-NB15

En esta sección se presetan los resultados obtenidos tras la optimización de hiperparámetros estructurales para cada modelo entrenado con el *dataset* UNSW-NB15. Se muestra la configuración ganadora de cada modelo, seguida de una comparativa global de *benchmarking*, además de la curva ROC y las matrices de confusión correspondientes a cada modelo. La combinación de hiperparámetros ganadores para cada modelo se muestra en la siguiente tabla:

Modelo	Hiperparámetros ganadores (UNSW-NB15)
Random Forest	n_estimators = 50, max_depth = 23
XGBoost	n_estimators = 200, max_depth = 11
LightGBM	n_estimators = 100, num_leaves = 145, max_depth = 12
CatBoost	n_estimators = 500, max_depth = 10
SVM	C = 0.000187
MLP	n_layers = 3, unidades = [96, 32, 64]
CNN-1D	n_filters = 64, kernel_size = 5, dense_units = 48

Tabla 5.1. Hiperparámetros ganadores de los modelos evaluados con UNSW-NB15

Tras hacer la fase de *benchmarking* con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00014	7.315991×10^6	0.09	0.9	0.7092
LightGBM	0.00064	1.551954×10^6	0.13	100	0.8592
XGBoost	0.00067	1.492419×10^6	0.0	100	0.8579
CatBoost	0.00156	641453.0	1.0	50.1	0.8570
Random Forest	0.00889	112423.0	9.04	1.6	0.8606
TensorFlow MLP	0.02443	40930.0	13.14	1.5	0.7862
CNN-1D	0.04931	20281.0	45.61	1.7	0.7665

Tabla 5.2. Comparativa global de rendimiento de modelos - UNSW-NB15

A partir de la comparativa global de rendimiento, se observa que los modelos basados en árboles presentan el mejor equilibrio entre capacidad predictiva y eficiencia computacional. Random Forest, LightGBM, XGBoost y CatBoost alcanzan los valores más altos de F1-score. Entre ellos, LightGBM y XGBoost destacan por ofrecer una latencia muy reducida y un *throughput* elevado, mientras que Random Forest obtiene un F1-score ligeramente superior, aunque con una latencia mayor.

Por otro lado, Linear SVM presenta la menor latencia de inferencia, pero también el F1-score más bajo entre los modelos evaluados. Esto indica que, aunque resulta muy eficiente desde el punto de vista temporal, su formulación lineal limita su capacidad para capturar relaciones no lineales presentes en el *dataset*. Los modelos neuronales, MLP y CNN-1D, obtienen resultados intermedios en términos de F1-score, pero con un coste de inferencia superior al de los modelos basados en árboles, especialmente en el caso de CNN-1D.

En cuanto al consumo de recursos, modelos como CNN-1D y MLP presentan un mayor incremento de memoria RAM, especialmente CNN-1D, que requiere más memoria durante la inferencia que los modelos clásicos. Por el contrario, SVM, XGBoost, LightGBM y CatBoost presentan una huella de memoria más reducida. Respecto al uso de CPU, LightGBM y XGBoost acaparan el 100% de la CPU durante la inferencia, puede explicarse por la propia implementación de estas librerías, pues son modelos diseñados para ser eficientes y paralelizables [xgboost_parallel] [lightgbm_parallel].

La curva ROC de cada modelo se muestra a continuación:

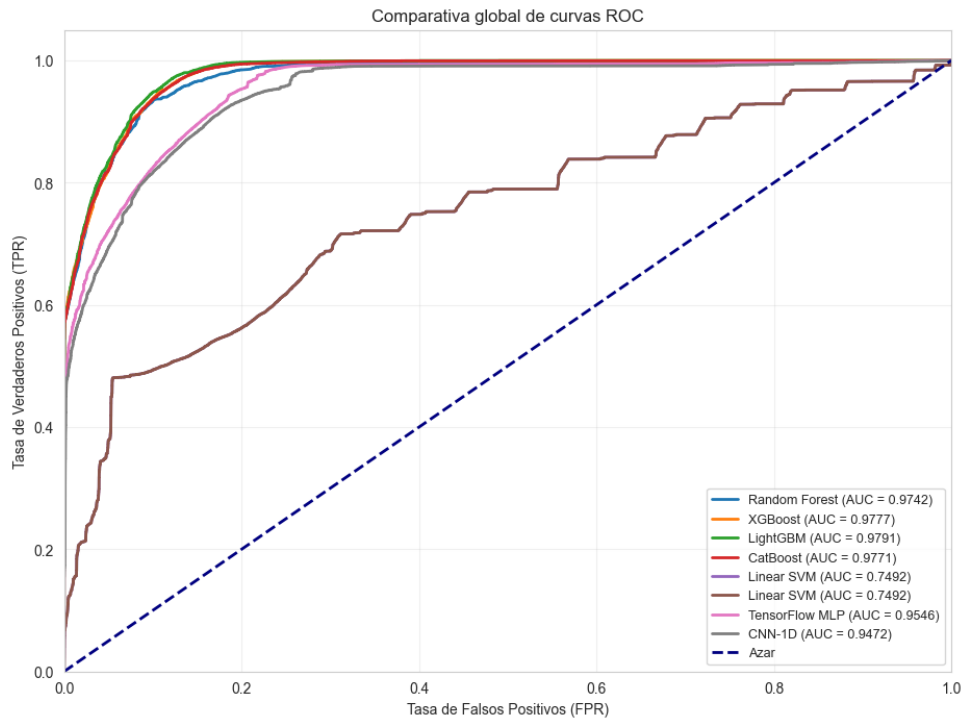


Figura 5.2. Curva ROC - Modelos

La curva ROC confirma la tendencia observada en la tabla anterior, mostrando que los modelos basados en árboles alcanzan las áreas bajo la curva más elevadas, lo que indica una mejor capacidad de discriminación entre clases. SVM presenta un rendimiento inferior en términos de AUC, mientras que los modelos neuronales muestran un rendimiento intermedio.

Las matrices de confusión nos permiten observar con más detalle los errores cometidos por cada modelo. Se presentan a continuación:

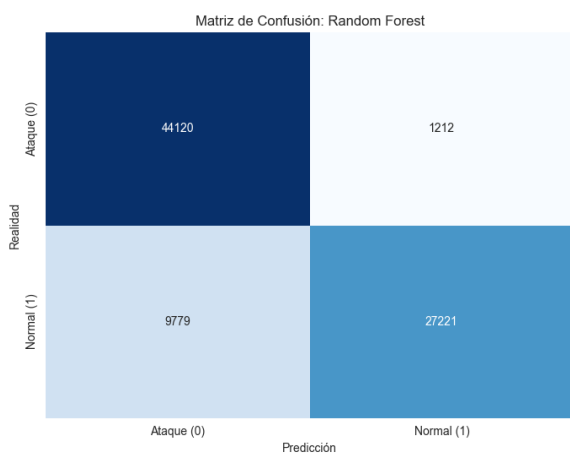


Figura 5.3. MC - Random Forest

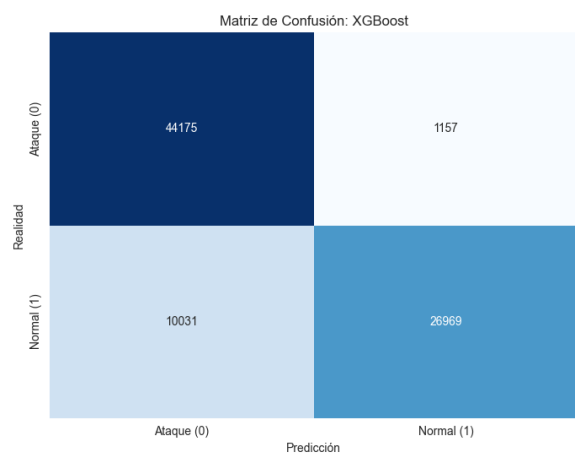


Figura 5.4. MC - XGBoost

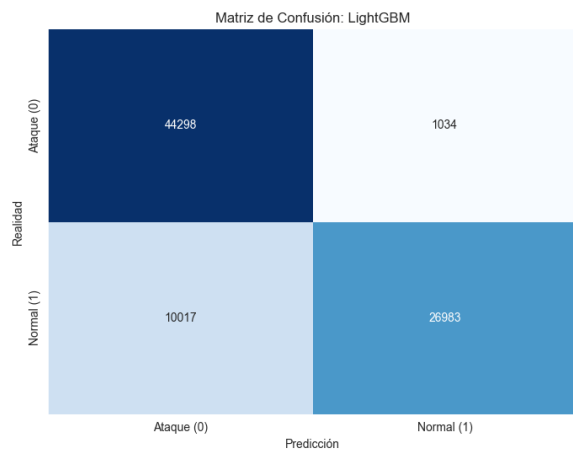


Figura 5.5. MC - LightGBM

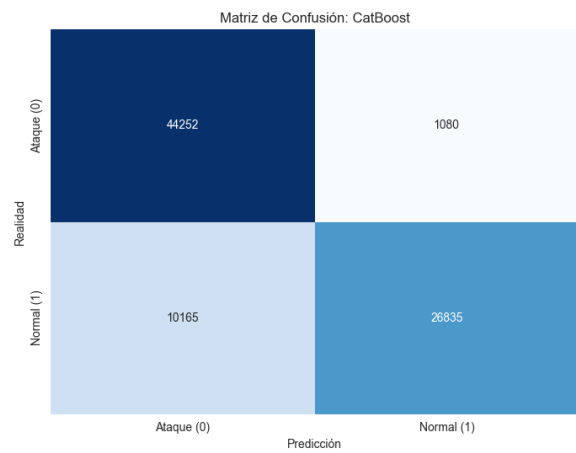


Figura 5.6. MC - CatBoost

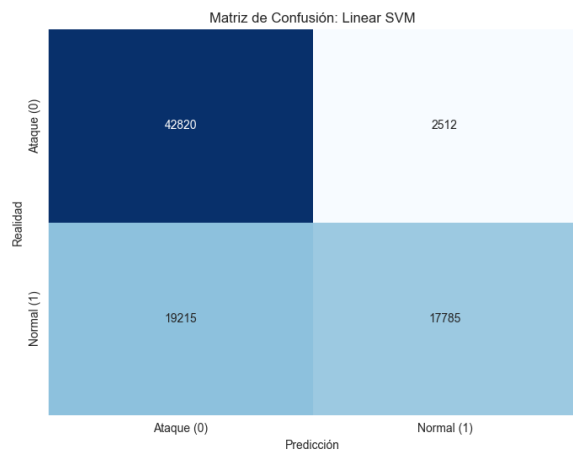


Figura 5.7. MC - SVM

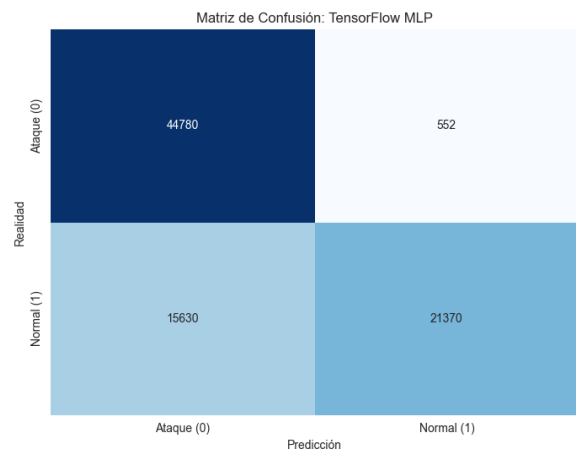


Figura 5.8. MC - MLP

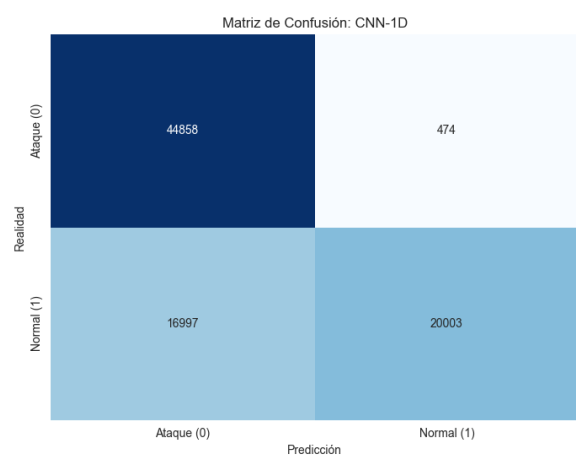


Figura 5.9. MC - CNN

5.4 Modelos entrenados con IOT-23

Para completar el análisis y las comparaciones, es interesante ver cómo se comportan los modelos con un *dataset* adaptado a un entorno de Internet de las Cosas (IoT), que suele presentar características muy diferentes a los *datasets* tradicionales de tráfico de red. En este caso, se han entrenado los mismos modelos que el apartado anterior, pero usando este *dataset*.

La tabla de los mejores hiperparámetros obtenidos para cada modelo con el *dataset* IOT-23 es la siguiente:

Modelo	Hiperparámetros ganadores (IOT-23)
Random Forest	n_estimators = 50, max_depth = 21
XGBoost	n_estimators = 50, max_depth = 8
LightGBM	n_estimators = 50, num_leaves = 15, max_depth = 6
CatBoost	n_estimators = 100, max_depth = 8
SVM	C = 0.801714
MLP	n_layers = 3, unidades = [32, 48, 48]
CNN-1D	n_filters = 64, kernel_size = 4, dense_units = 80

Tabla 5.3. Hiperparámetros ganadores de los modelos evaluados con IOT-23

Tras hacer una fase de *benchmarking* con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00018	5.684672×10^6	2.01	1.1	0.955
LightGBM	0.00047	2.144772×10^6	0.11	100	0.9998
XGBoost	0.00584	171225	0.31	96.2	0.9991
CatBoost	0.00102	982856	0.02	69.2	0.9999
Random Forest	0.00892	112103	13.39	1.4	0.9997
TensorFlow MLP	0.02377	42079	0.72	1.5	0.9956
CNN-1D	0.04959	20153	8.25	1.8	0.9952

Tabla 5.4. Comparativa global de rendimiento de modelos - IOT-23

En el caso de IoT-23, la tabla de rendimiento muestra valores de F1-score muy elevados en la mayoría de modelos, especialmente en los clasificadores basados en árboles. Random Forest, XGBoost, LightGBM y CatBoost alcanzan resultados cercanos a 1, lo que indica una gran capacidad para distinguir entre tráfico benigno y malicioso en este *dataset*. Además, estos modelos mantienen latencias reducidas y *throughputs* elevados, por lo que ofrecen un compromiso muy favorable entre rendimiento predictivo y eficiencia.

Los modelos neuronales, MLP y CNN-1D, también presentan un rendimiento muy competitivo, aunque con un coste de inferencia superior, especialmente en el caso de CNN-1D. Por su parte, SVM vuelve a destacar por su rapidez, pero su F1-score queda por debajo del resto de modelos.

Respecto al consumo de recursos, se observa que Random Forest es el modelo con mayor incremento de RAM, seguido de CNN-1D. En términos de uso de CPU, al igual que con UNSW-NB15, LightGBM y XGBoost son los modelos que mayor porcentaje de uso de CPU acaparan.

La curva ROC de los modelos evaluados con el *dataset* IOT-23 es la siguiente:

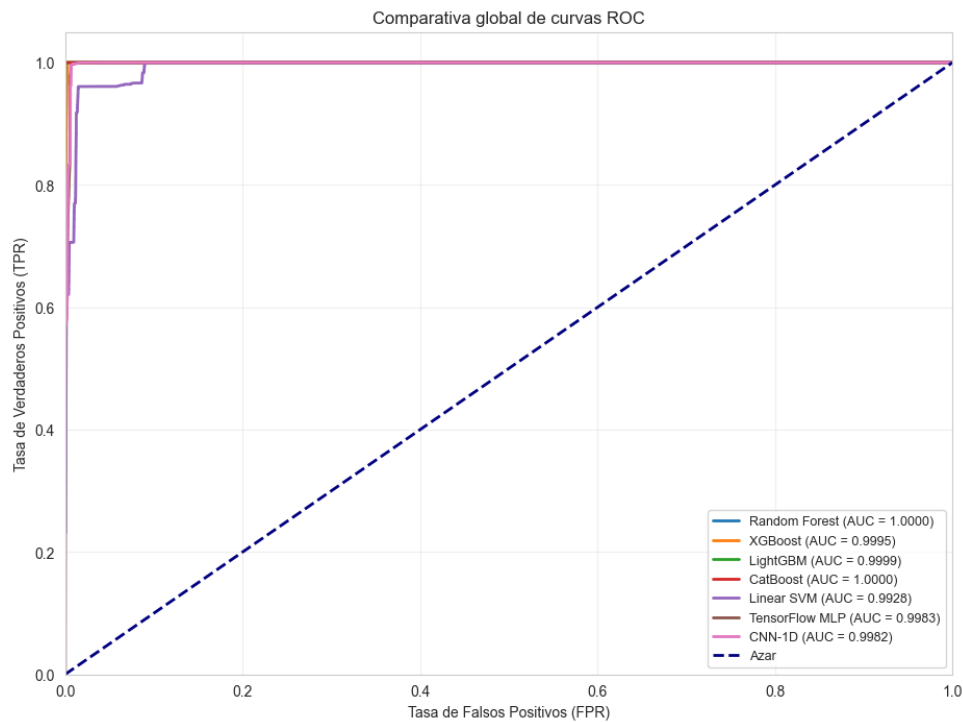


Figura 5.10. Curva ROC - Modelos (IOT-23)

Las curvas ROC obtenidas confirman una excelente separación entre clases en la mayoría de modelos, coherentes con los valores de F1-score observados en la tabla anterior.

Las matrices de confusión de cada modelo son las siguientes:

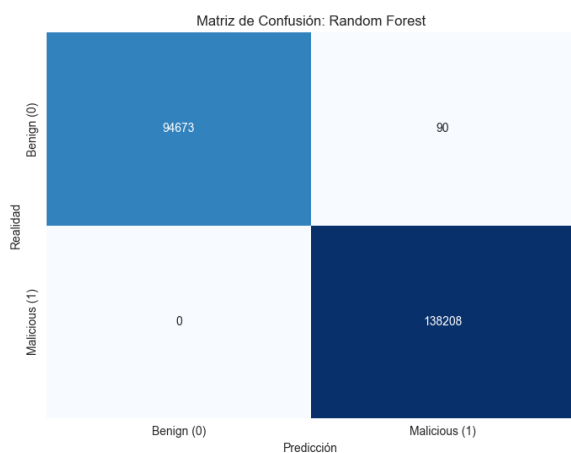


Figura 5.11. MC - Random Forest (IOT-23)

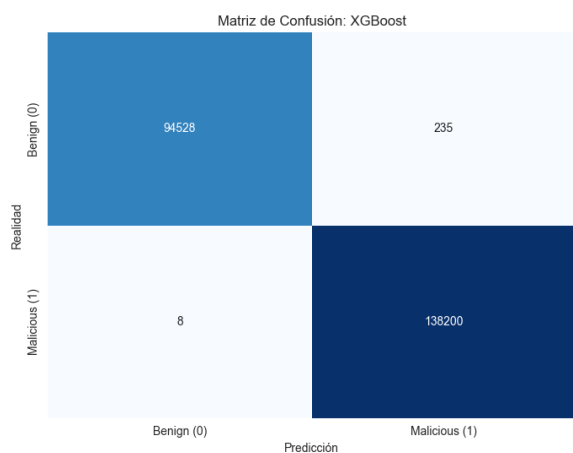


Figura 5.12. MC - XGBoost (IOT-23)

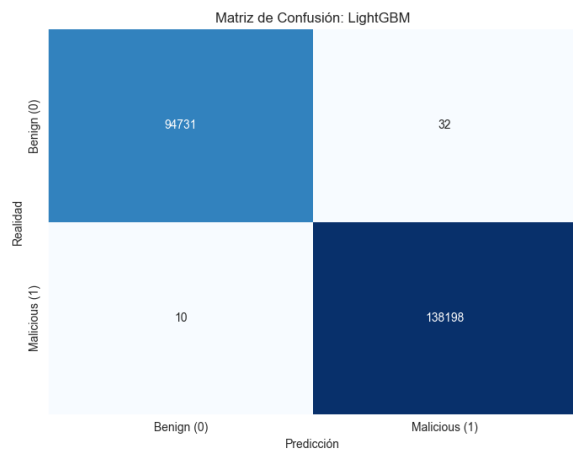


Figura 5.13. MC - LightGBM (IOT-23)

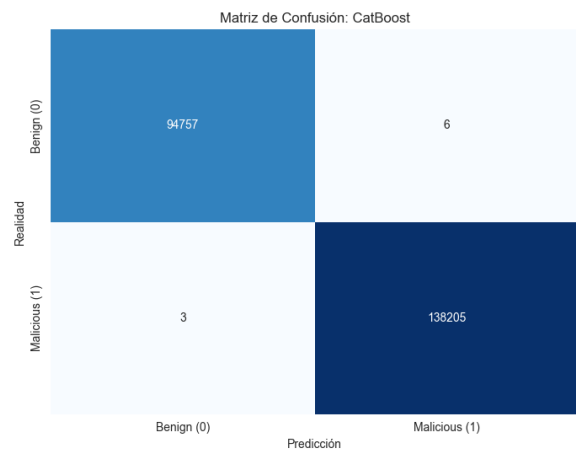


Figura 5.14. MC - CatBoost (IOT-23)

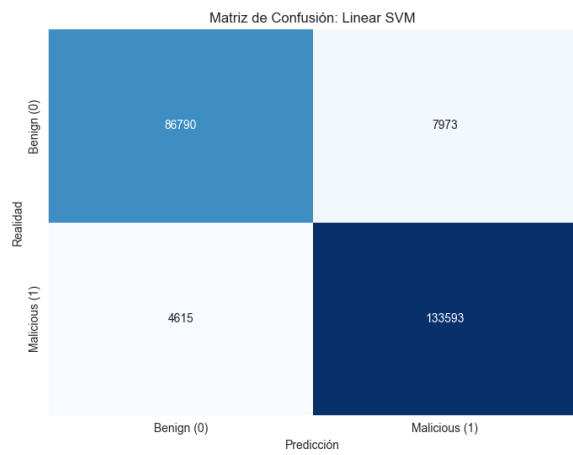


Figura 5.15. MC - SVM (IOT-23)

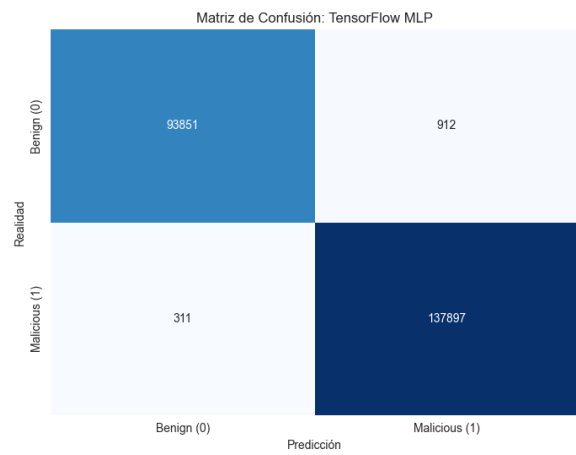


Figura 5.16. MC - MLP (IOT-23)

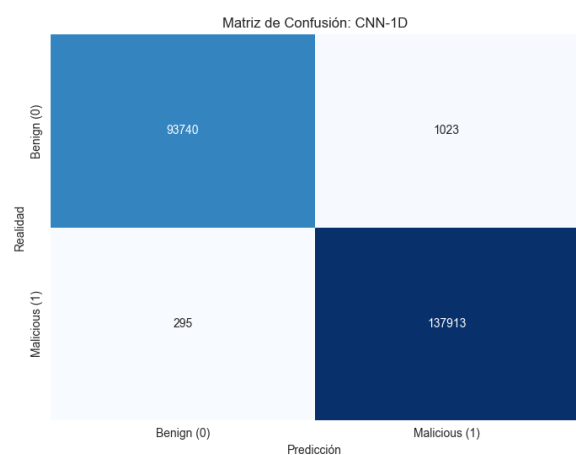


Figura 5.17. MC - CNN (IOT-23)

5.5 Modelos entrenados con ToN-IoT

Una de las sospechas de los resultados obtenidos con el *dataset* IOT-23 es que en el *dataset* la mayoría de muestras de ataque son de escaneos horizontales de puertos, lo que hace que el modelo pueda aprender a detectar ataques simplemente con la información de los puertos, sin necesidad de aprender patrones más complejos. Para comprobar esta hipótesis, se ha entrenado el mismo proceso de entrenamiento y *benchmarking* con el *dataset* ToN-IoT, que es un *dataset* de IoT con una mayor variedad de ataques y características.

La tabla de los mejores hiperparámetros obtenidos para cada modelo con el *dataset* ToN-IoT es la siguiente:

Modelo	Hiperparámetros ganadores (ToN-IoT)
Random Forest	n_estimators = 50, max_depth = 18
XGBoost	n_estimators = 50, max_depth = 7
LightGBM	n_estimators = 100, num_leaves = 233, max_depth = 11
CatBoost	n_estimators = 250, max_depth = 9
SVM	C = 2.870875
MLP	n_layers = 3, unidades = [64, 16, 64]
CNN-1D	n_filters = 96, kernel_size = 5, dense_units = 96

Tabla 5.5. Hiperparámetros ganadores de los modelos evaluados con ToN-IoT

Tras hacer una fase de *benchmarking* con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00019	5.185815×10^6	0.02	1.3	0.9522
LightGBM	0.0003	3.333129×10^6	0.0	100	0.9977
XGBoost	0.00065	1.536528×10^6	0.2	100	0.9986
CatBoost	0.00129	773537.0	0.03	57.3	0.9985
Random Forest	0.00886	112889.0	8.04	1.5	0.9983
TensorFlow MLP	0.02633	37985.0	1.92	1.5	0.9829
CNN-1D	0.0531	18831.0	89.74	2.4	0.9826

Tabla 5.6. Comparativa global de rendimiento de modelos - IOT-23

En TON-IoT, los modelos basados en árboles vuelven a situarse entre las mejores alternativas, combinando valores elevados de F1-score con tiempos de inferencia reducidos. XGBoost, CatBoost, Random Forest y LightGBM muestran un rendimiento muy alto, lo que confirma la eficacia de este tipo de modelos en *datasets* tabulares de detección de intrusiones.

SVM, de nuevo, mantiene una latencia muy baja, pero obtiene un F1-score inferior al de los modelos basados en árboles. Los modelos neuronales, MLP y CNN-1D, presentan un comportamiento competitivo, aunque con mayor coste temporal.

Si miramos el consumo de recursos, CNN-1D destaca como el modelo con mayor incremento de RAM. En cuanto al uso de CPU, LightGBM y XGBoost vuelven a ser los modelos que muestran mayores porcentajes durante la fase de inferencia.

La curva ROC de los modelos evaluados con el *dataset* ToN-IoT es la siguiente:

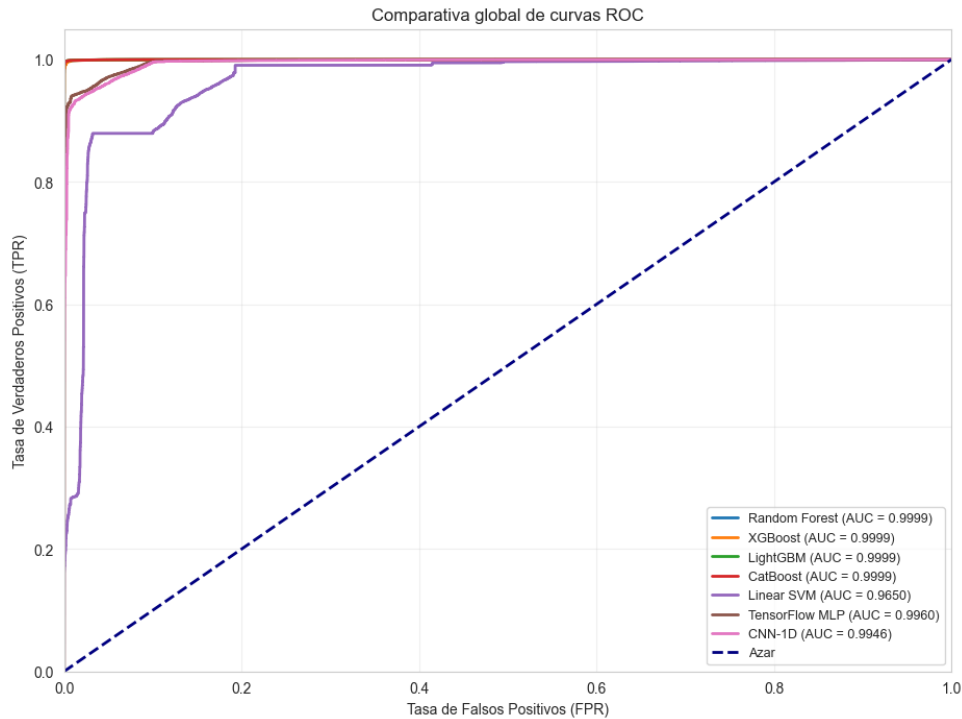


Figura 5.18. Curva ROC - Modelos (ToN-IoT)

Al igual que el *dataset* anterior, las curvas ROC obtenidas muestran una excelente capacidad de discriminación entre clases para la mayoría de modelos, especialmente los basados en árboles, que alcanzan áreas bajo la curva cercanas a 1, lo que es coherente con los valores de F1-score observados en la tabla de rendimiento.

Las matrices de confusión de cada modelo se muestran a continuación:

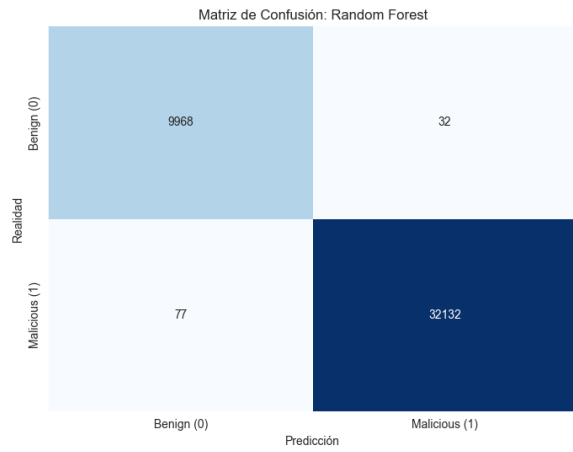


Figura 5.19. MC - Random Forest (ToN-IoT)

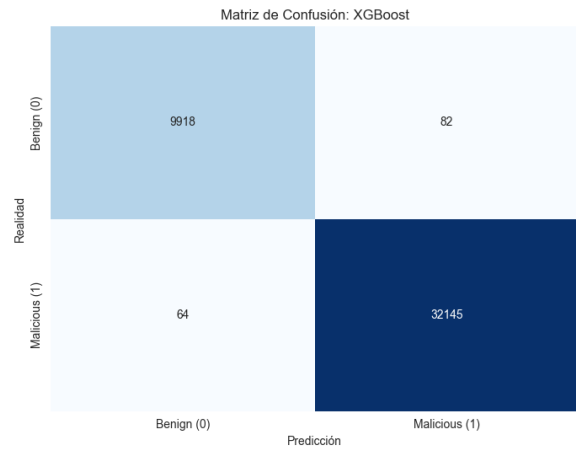


Figura 5.20. MC - XGBoost (ToN-IoT)

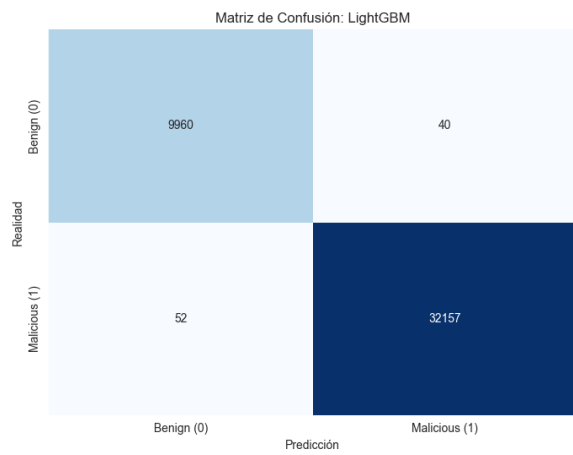


Figura 5.21. MC - LightGBM (ToN-IoT)

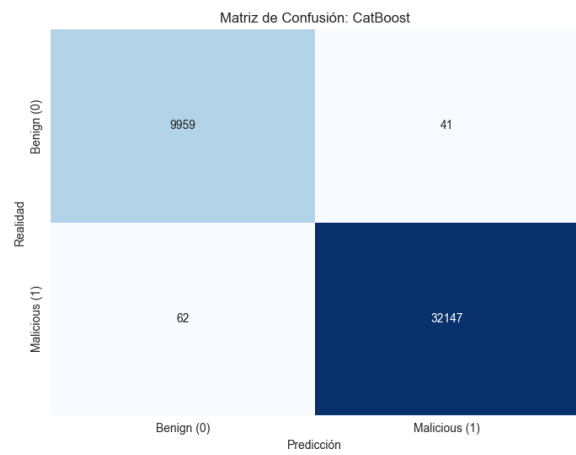


Figura 5.22. MC - CatBoost (ToN-IoT)

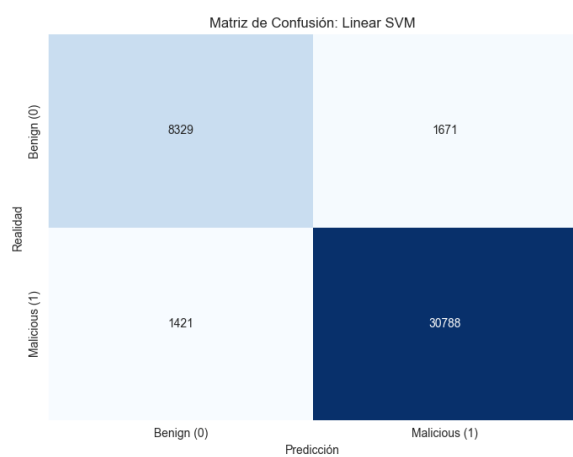


Figura 5.23. MC - SVM (ToN-IoT)

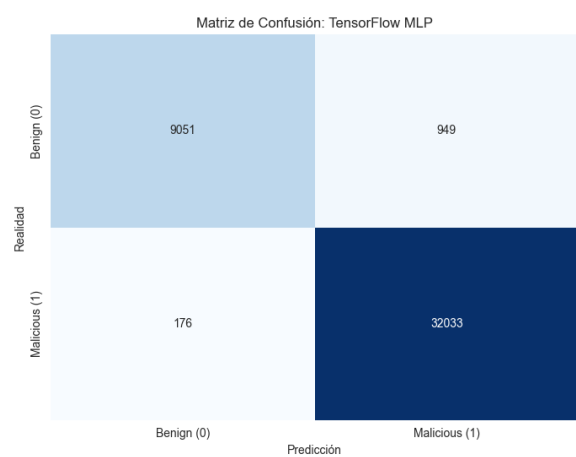


Figura 5.24. MC - MLP (ToN-IoT)

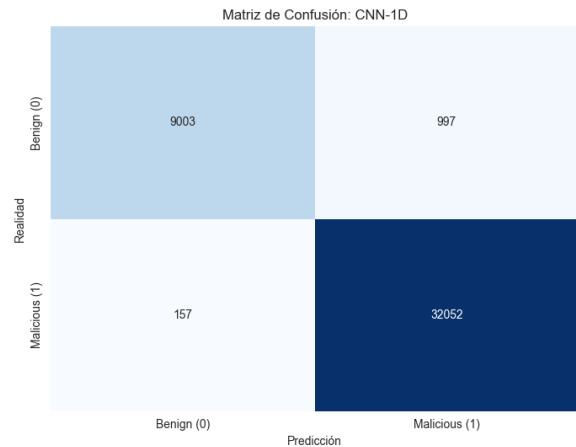


Figura 5.25. MC - CNN (ToN-IoT)

5.6 Potencial de Análisis en Tiempo Real

Como se ha indicado anteriormente, los *benchmarks* realizados en el servidor tenían como objetivo medir no solo el tiempo de inferencia, sino también el uso de CPU y memoria RAM durante la predicción. Por este motivo, las métricas temporales obtenidas en dichos *benchmarks* incluyen la sobrecarga asociada al procesamiento por bloques y a la monitorización del proceso.

Para complementar ese análisis, en esta sección se realiza una medición adicional centrada exclusivamente en el tiempo puro de inferencia. El objetivo es estimar la capacidad máxima de procesamiento de los modelos una vez cargados en memoria, por lo que la comparación se centra principalmente en el *throughput*, expresado como el número de muestras procesadas por segundo.

Además, esta medición se ha repetido en un equipo local, sin utilizar el servidor de altas prestaciones empleado durante las fases anteriores. De esta forma, se pretende analizar y comparar el comportamiento de los modelos en un entorno más limitado y cercano a un posible escenario real de despliegue, donde los recursos disponibles pueden ser inferiores a los del entorno de entrenamiento.

5.6.1 Resultados UNSW-NB15

Tabla 5.7. Comparativa de F1-score y throughput de inferencia pura para UNSW-NB15

Modelo	F1-Score	Throughput Portátil	Throughput Servidor
RF	0.8606	1209890	3721179
XGBoost	0.8579	1113803	9916876
CatBoost	0.8570	1639079	3058951
LightGBM	0.8592	1260820	5452214
SVM	0.7092	16600905	31271516
MLP	0.7862	1150412	1199516
CNN	0.7665	225353	646466

5.6.2 Resultados IOT-23

Tabla 5.8. Comparativa de F1-score y throughput de inferencia pura para IoT-23

Modelo	F1-Score	Throughput Portátil	Throughput Servidor
RF	0.9997	1149778	5684020
XGBoost	0.9991	5391713	48100521
CatBoost	0.9999	3859140	4200133
LightGBM	0.9998	3932579	8290807
SVM	0.9550	5323551	9656967
MLP	0.9956	1961042	2278408
CNN	0.9952	187062	629180

5.6.3 Resultados TON-IOT

Tabla 5.9. Comparativa de F1-score y throughput de inferencia pura para TON-IoT

Modelo	F1-Score	Throughput Portátil	Throughput Servidor
RF	0.9983	1401017	3924738
XGBoost	0.9986	2605794	41432520
CatBoost	0.9985	2313078	3063462
LightGBM	0.9977	1259440	5034803
SVM	0.9522	5113359	10344547
MLP	0.9829	837747	1133827
CNN	0.9826	98625	334864

5.6.4 Comparación con cargas de red de referencia

Una vez que hemos obtenido los resultados de *throughput* para cada modelo en los tres *datasets*, es interesante compararlos con las cargas de red típicas que se pueden encontrar en diferentes escenarios, para así estimar qué modelo nos convendría más en función del entorno de despliegue y del volumen de tráfico que se espera analizar.

Para ello, se han tomado como referencia varios escenarios: una red IoT local básica de 21 dispositivos [*cargas_iot*], una red universitaria [*cargas_universitarias*] y distintos casos de pequeñas y medianas empresas [*cargas_pymes*].

Tabla 5.10. Cargas de red de referencia expresadas en paquetes por segundo

Escenario	Tráfico considerado	Tamaño medio paquete	Carga estimada (paq/s)
Red IoT local básica – caso mínimo	400 Kbps	225 bytes	222
Red IoT local básica – caso intermedio	400 Kbps	75 bytes	667
Red IoT local básica – caso alto	17 Mbps	225 bytes	9444
Red IoT local básica – peor caso	17 Mbps	75 bytes	28333
Red universitaria	17.8M paquetes / 10 min	–	29667
PYME pequeña – 8 Mbps	8 Mbps	225 bytes	4444
PYME pequeña – 32 Mbps	32 Mbps	225 bytes	17778
PYME 50 empleados – 189 Mbps	189 Mbps	225 bytes	105000
PYME 50 empleados – peor caso	189 Mbps	75 bytes	315000

Como podemos ver en la tabla 5.10, las cargas de red de referencia varían desde unos pocos cientos de paquetes por segundo en el caso de una red IoT local básica, hasta varios cientos de miles de paquetes por segundo en el caso de una PYME con un tráfico elevado.

Al comparar estas cargas con los *throughputs* obtenidos por los modelos en los tres *datasets*, podemos observar que la mayoría de modelos presentan una capacidad de procesamiento superior a las cargas de referencia, incluso cuando se ejecutan en nuestro equipo personal.

Modelos como SVM, XGBoost, LightGBM, Random Forest y CatBoost alcanzan valores superiores al millón de muestras por segundo, lo que los posiciona como opciones viables para escenarios con cargas de red elevadas, al igual que MLP.

Por otro lado, modelos como CNN-1D ejecutados en equipo local pueden ser viables para escenarios con cargas más moderadas, pero insuficientes para casos más exigentes como 315000 paquetes por segundo, requiriendo hardware más potente.

6

Ataques de Evasión

6.1 Evaluación de robustez frente a ataques adversarios

Hasta este punto del trabajo, los modelos de detección de intrusiones desarrollados han mostrado un rendimiento muy elevado sobre los conjuntos de prueba originales. En particular, las arquitecturas profundas, como MLP y CNN-1D, han alcanzado valores de F1-score superiores al 96%, mientras que los modelos basados en ensambles de árboles de decisión, como Random Forest, XGBoost, LightGBM y CatBoost, han obtenido resultados cercanos al 99%. Estos resultados indican que los modelos son capaces de aprender patrones relevantes en los *datasets* empleados y ofrecen una alta capacidad de clasificación en condiciones normales de evaluación.

Sin embargo, en el ámbito de la ciberseguridad, un buen rendimiento sobre datos estáticos no garantiza por sí solo que un sistema sea robusto en escenarios reales. Los atacantes pueden conocer o inferir el funcionamiento de los sistemas de detección y adaptar su comportamiento para evadirlos. En este contexto surge el **aprendizaje automático adversario**, cuyo objetivo es estudiar cómo pequeñas modificaciones en las entradas pueden provocar errores de clasificación en modelos de inteligencia artificial.

En el caso de los sistemas de detección de intrusiones en red, los ataques de evasión se producen durante la fase de inferencia. En este tipo de ataques, una muestra maliciosa se modifica de forma controlada para intentar que el modelo la clasifique como tráfico benigno.

Para analizar esta problemática, en esta fase del proyecto se evaluará la **robustez adversaria de los modelos finales seleccionados en las etapas anteriores**. Es importante destacar que no se realizará una nueva optimización de hiperparámetros, sino que se partirá de las **configuraciones ya elegidas para cada modelo**. De este modo, la evaluación adversaria se plantea como una prueba adicional de resistencia sobre los modelos previamente considerados como mejores bajo condiciones limpias.

6.1.1 Uso de Adversarial Robustness Toolbox

La herramienta principal empleada para esta evaluación será **Adversarial Robustness Toolbox** (ART) [art], una librería orientada al análisis de seguridad y robustez de modelos de aprendizaje automático.

ART proporciona implementaciones estandarizadas de ataques adversarios utilizados habitualmente en la literatura, como FGSM, BIM o PGD, y permite integrarlos con modelos desarrollados en distintos entornos de aprendizaje automático y aprendizaje profundo.

La evaluación de robustez de los distintos clasificadores NIDS propuestos en este trabajo exige adaptar la estrategia de ataque a la naturaleza matemática subyacente de cada algoritmo. Por este motivo, se ha diseñado una metodología en dos fases, **basada en ataques directos y en la transferencia de vulnerabilidades**.

6.1.1.1 Ataques Empleados

En esta evaluación se han empleado dos ataques de evasión ampliamente reconocidos en la literatura de aprendizaje automático adversario:

- **Fast Gradient Sign Method (FGSM):** Este método genera perturbaciones adversarias a partir del gradiente de la función de pérdida con respecto a las características de entrada. La perturbación se calcula como el signo del gradiente multiplicado por un factor de escala ϵ , que controla la magnitud de la alteración. FGSM es un ataque rápido y eficiente, aunque su efectividad puede ser limitada frente a modelos robustos o con mecanismos de defensa [FGSM].
- **Projected Gradient Descent (PGD):** PGD es una extensión iterativa de FGSM que aplica múltiples pasos de perturbación, proyectando cada iteración dentro de un espacio permitido definido por ϵ . Este método es más potente que FGSM, ya que permite explorar un espacio más amplio de perturbaciones y puede superar defensas basadas en la limitación de la magnitud de las alteraciones [PGD].

6.1.1.2 Ataques de Caja Blanca basados en Gradiente (MLP y CNN-1D)

Los algoritmos de evasión empleados en este estudio, como *Projected Gradient Descent* (PGD) o *Fast Gradient Sign Method* (FGSM), requieren de un requisito matemático indispensable: **el modelo objetivo debe ser continuo y diferenciable**.

En arquitecturas de aprendizaje profundo como el Perceptrón Multicapa (MLP) y las Redes Neuronales Convolucionales (CNN-1D), el atacante emplea la retropropagación (*backpropagation*) para calcular el gradiente de la función de pérdida con respecto a las características de entrada. Esto permite identificar la dirección matemática exacta para alterar el tráfico de red original e inducir a una clasificación errónea (falsos negativos).

6.1.1.3 Transferencia Adversaria para Modelos Discretos (Ensamblados y SVM)

A diferencia de las redes neuronales, los modelos que fundamentan sus decisiones en particiones del espacio paramétrico, como los ensambles de árboles de decisión (Random Forest, XGBoost, LightGBM,

CatBoost) o las Máquinas de Vectores de Soporte (SVM), presentan fronteras lógicas abruptas que inutilizan la aplicación directa del cálculo de gradientes.

Para solventar esta limitación e incorporar estos algoritmos a la evaluación de seguridad, se explota la técnica de **Transferencia Adversaria** (*Adversarial Transferability*) [transferencia_adversaria]. Esta estrategia asume que las perturbaciones diseñadas para engañar a un modelo específico suelen evadir con éxito clasificadores con arquitecturas completamente distintas. Es decir, el veneno que fabricamos para matar a un modelo específico, suele ser letal también para otros modelos que no tienen nada que ver con el primero. Por ejemplo, un ataque diseñado para engañar a un MLP podría transferirse con éxito a un Random Forest o a una SVM, incluso sin acceso directo a sus parámetros internos.

El procedimiento aplicado es el siguiente:

1. **Generación (White-box):** Los modelos diferenciables (MLP y CNN-1D) actúan como modelos generadores del nuevo conjunto de tráfico de red evadido.
2. **Evaluación Cruzada (Black-box):** El *Dataset Adversario* se introduce directamente en la fase de inferencia de los clasificadores basados en árboles y SVM.
3. **Cuantificación del Impacto:** Se mide la degradación de las métricas de rendimiento frente al estado basal.

6.1.2 Análisis de Resultados en UNSW-NB15

En esta sección se presentarán los resultados obtenidos tras la aplicación de los ataques de evasión descritos en la sección anterior sobre los modelos ganadores de la sección anterior y con su *dataset* correspondiente.

El procedimiento que se siguió fue entrenar cada modelo nuevamente con la configuración de hiperparámetros seleccionada en la fase anterior con su *dataset* correspondiente. Posteriormente, se aplicaron los ataques de evasión para generar un nuevo conjunto de datos adversarios sobre los modelos MLP y CNN-1D (transferencia adversaria).

Es fundamental destacar que estos ataques se han configurado con un límite de perturbación extremadamente restrictivo ($\epsilon = 0.05$). Esto implica que el algoritmo de evasión solo ha modificado, como máximo, un 5% el valor original de las características numéricas continuas, garantizando que el tráfico malicioso generado sea prácticamente indistinguible del tráfico original a nivel de red.

Finalmente, los datos adversarios generados se introdujeron en la fase de inferencia de los distintos clasificadores para extraer sus respectivas métricas de rendimiento. El análisis se fundamentó en métricas estándar como el *Accuracy*, el F1-Score y el *Recall*, priorizando en todo momento la **evaluación sobre la clase maliciosa**. Esta decisión metodológica se basa en que el objetivo crítico del estudio es medir la degradación en la capacidad de detección de ataques del sistema.

6.1.2.1 Resultados PGD

Presentamos los resultados de la aplicación del ataque PGD sobre los modelos entrenados con el *dataset* UNSW-NB15. En los gráficos de barras podemos ver la métrica que corresponde a cada modelo, tanto en su estado original (barra azul), tras la aplicación del ataque con el *dataset* alterado con MLP (barra naranja) y tras la aplicación del ataque con el *dataset* alterado con CNN-1D (barra verde).

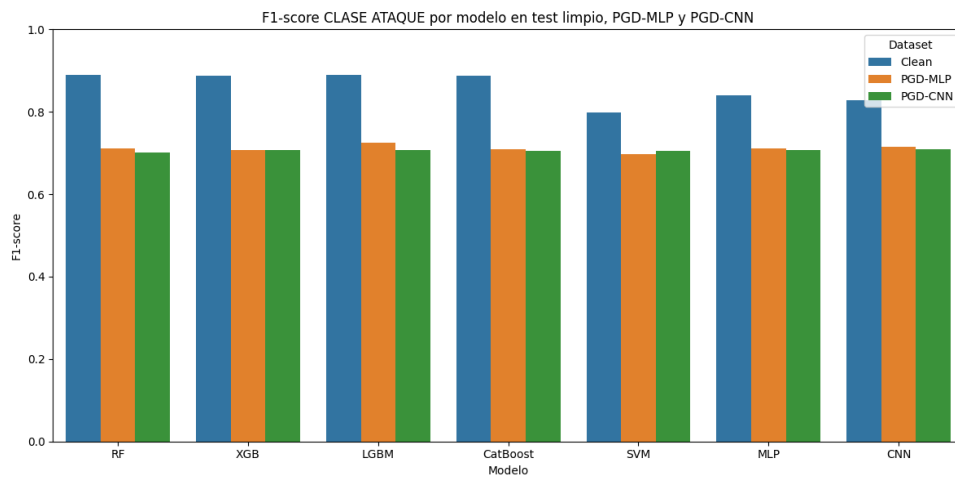


Figura 6.1. F1-PGD-UNSW-NB15

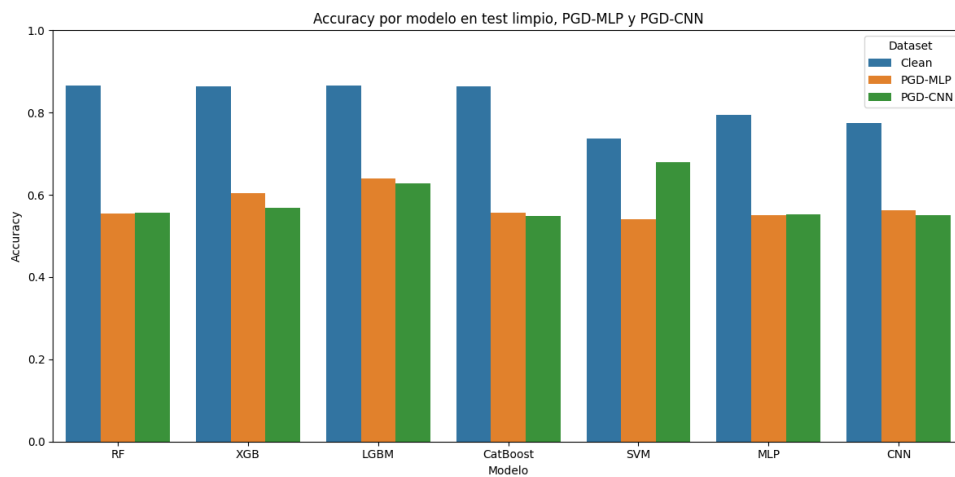


Figura 6.2. Accuracy-PGD-UNSW-NB15

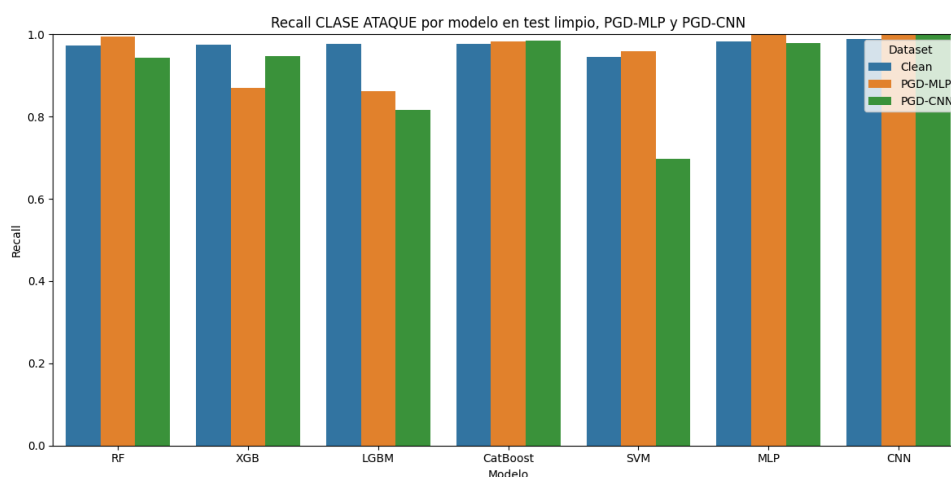


Figura 6.3. Recall-PGD-UNSW-NB15

Podemos concluir que, incluso con un límite de perturbación moderado de $\epsilon = 0.05$, el ataque PGD consigue degradar el rendimiento de los modelos respecto al test limpio. Esta caída se observa especialmente en la **accuracy** y en el **F1-score de la clase ataque**, mientras que el *recall* se mantiene elevado en varios modelos.

Este comportamiento indica que los modelos siguen detectando una proporción alta de ataques, pero pierden capacidad para clasificar correctamente el conjunto completo de muestras. En otras palabras, la perturbación afecta al equilibrio general del clasificador, aunque no siempre reduzca de forma drástica la detección de ataques.

Como el *recall* de la clase ataque sigue siendo alto en muchos casos, pero el F1-score disminuye, es probable que la precisión de dicha clase se vea reducida. Esto sugiere que algunos modelos tienden a clasificar un mayor número de muestras como ataque, aumentando así los falsos positivos.

Por último, las perturbaciones generadas sobre los modelos MLP y CNN presentan un impacto similar en términos de F1-score, lo que evidencia nuestra hipótesis sobre la transferencia adversaria de los ataques hacia modelos clásicos como RF, XGB, LGBM, CatBoost y SVM.

6.1.2.2 Resultados FGSM

Estos son los resultados de la aplicación del ataque FGSM:

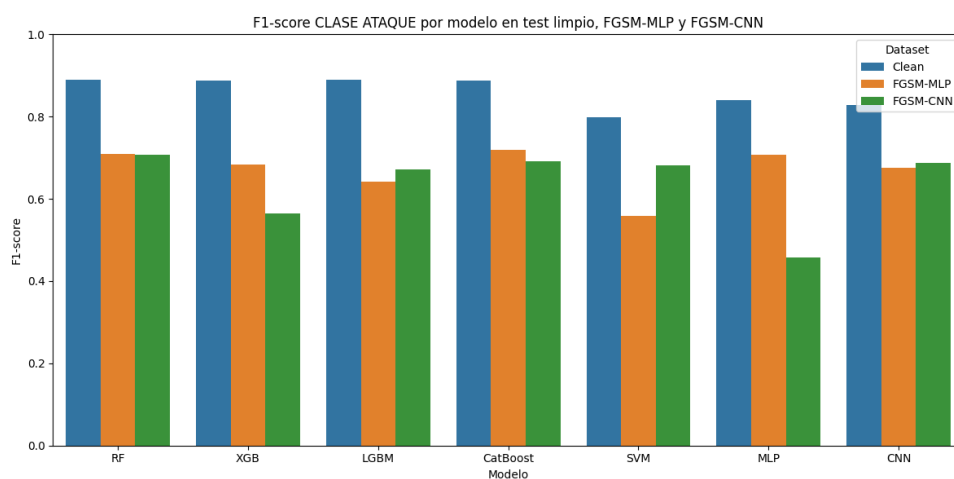


Figura 6.4. F1-FGSM-UNSW-NB15

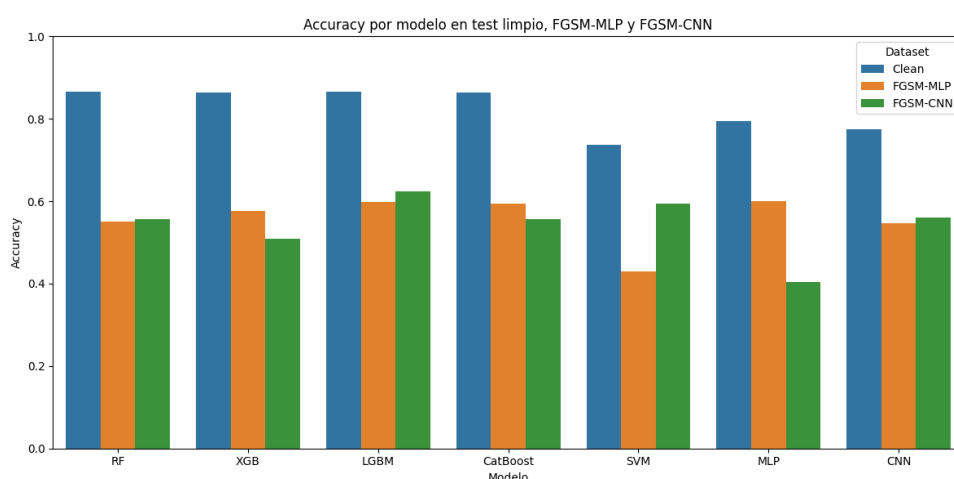


Figura 6.5. Accuracy-FGSM-UNSW-NB15

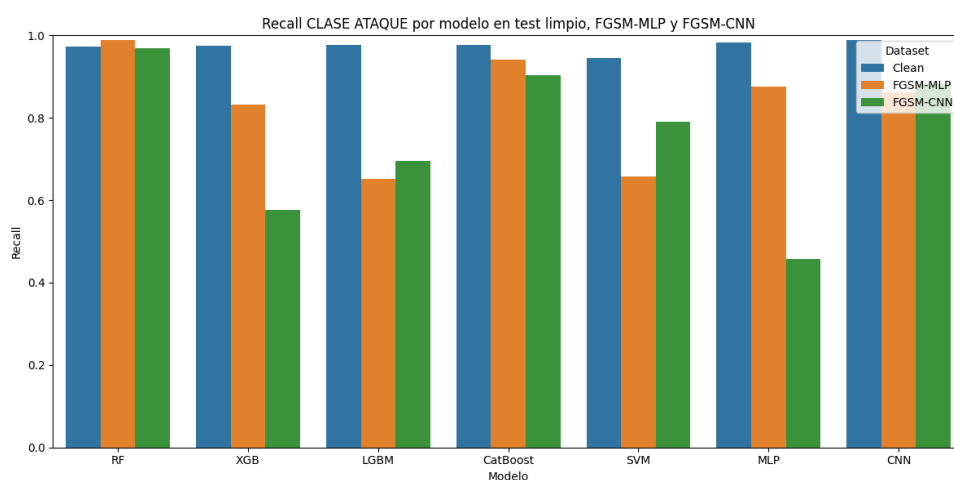


Figura 6.6. Recall-FGSM-UNSW-NB15

En el caso del ataque FGSM también se observa una degradación clara del rendimiento respecto al

test limpio. Esta caída aparece tanto en la **accuracy** como en el **F1-score de la clase ataque**, aunque el impacto varía según el modelo evaluado y según si la perturbación se genera a partir del MLP o de la CNN.

En términos de *accuracy*, todos los modelos reducen su rendimiento respecto al escenario limpio. Esta caída indica que FGSM afecta a la capacidad global de clasificación, provocando un mayor número de errores sobre el conjunto de test. En algunos modelos, como MLP frente a FGSM-CNN o SVM frente a FGSM-MLP, la reducción es especialmente notable.

Respecto al F1-score de la clase ataque, se aprecia que los modelos mantienen valores moderados en varios casos, pero siempre por debajo del rendimiento limpio. Esto indica que el ataque afecta al equilibrio entre precisión y *recall* de la clase ataque. En particular, algunos modelos como XGB, SVM o MLP muestran una mayor sensibilidad dependiendo del modelo utilizado para generar la perturbación.

El *recall* de la clase ataque muestra un comportamiento más desigual. Algunos modelos, como RF, CatBoost y CNN, mantienen una alta capacidad de detección de ataques incluso bajo perturbación FGSM. Sin embargo, otros modelos presentan caídas importantes, especialmente XGB, LGBM, SVM y MLP en determinados escenarios.

6.1.3 Análisis de Resultados en IOT-23

6.1.3.1 Resultados PGD

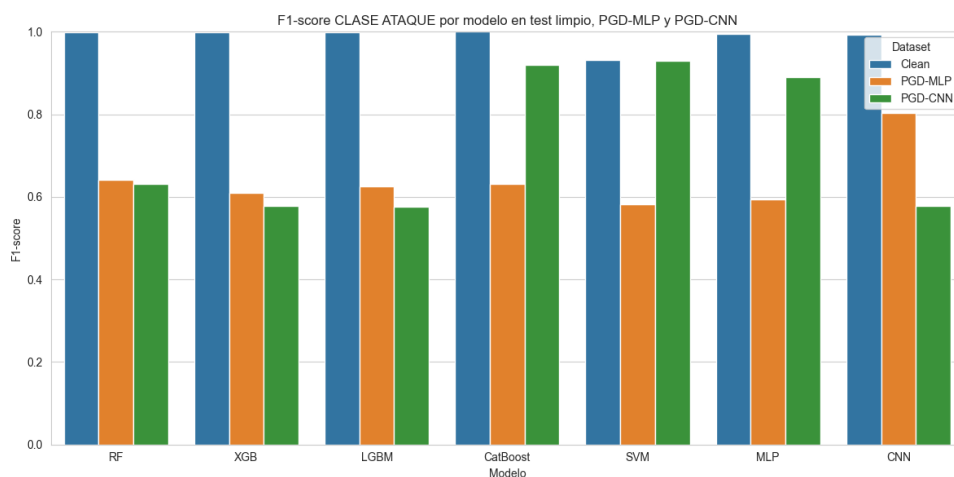


Figura 6.7. F1-PGD-IOT-23

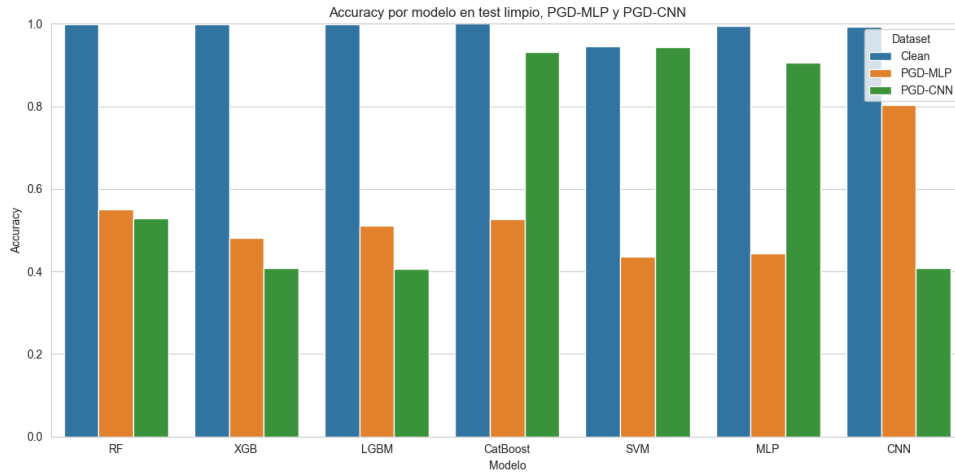


Figura 6.8. Accuracy-PGD-IOT-23

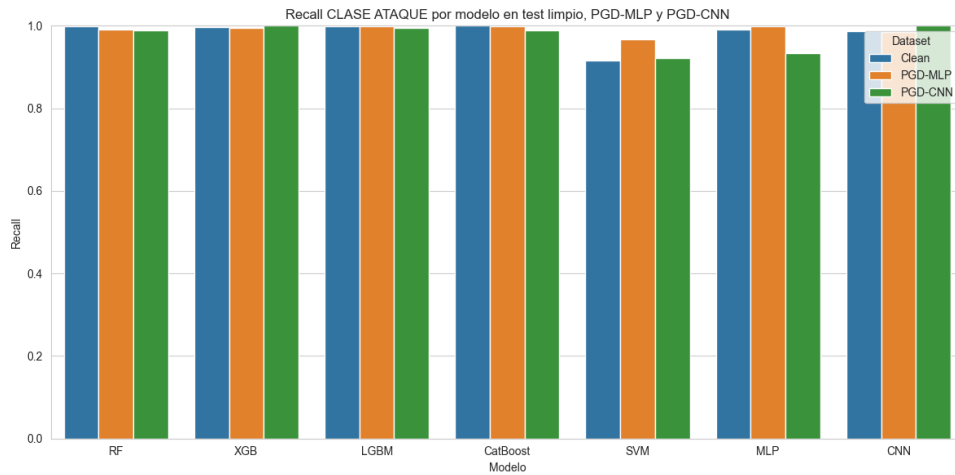


Figura 6.9. Recall-PGD-IOT-23

En el *dataset* IoT-23, para el ataque PGD con límite de perturbación $\epsilon = 0.05$, se observa una degradación notable del rendimiento respecto al test limpio, especialmente en términos de **accuracy** y **F1-score de la clase ataque**. Aunque los modelos presentan un rendimiento casi perfecto en el escenario limpio, las perturbaciones adversarias reducen de forma clara la capacidad global de clasificación.

Sin embargo, el *recall* de la clase ataque se mantiene muy elevado en prácticamente todos los modelos, incluso tras aplicar las perturbaciones PGD generadas sobre MLP y CNN. Esto indica que los modelos siguen detectando la mayoría de las muestras maliciosas, por lo que el ataque no provoca principalmente una pérdida de detección de ataques.

La diferencia entre el alto *recall* y la caída observada en *accuracy* y F1-score sugiere que el principal efecto del ataque es un aumento de falsos positivos. Es decir, los modelos tienden a clasificar más muestras normales como ataques, manteniendo una alta detección de tráfico malicioso, pero reduciendo su capacidad para distinguir correctamente entre tráfico benigno y malicioso.

Además, se observa que las perturbaciones generadas sobre MLP y CNN presentan capacidad de transferencia hacia modelos clásicos, aunque con diferencias entre clasificadores. En particular, PGD-MLP provoca una caída importante en modelos como RF, XGB, LGBM, CatBoost, SVM y MLP, mientras que PGD-CNN afecta de forma muy acusada a XGB, LGBM y CNN, pero tiene menor impacto sobre CatBoost, SVM y MLP.

6.1.3.2 Resultados FGSM

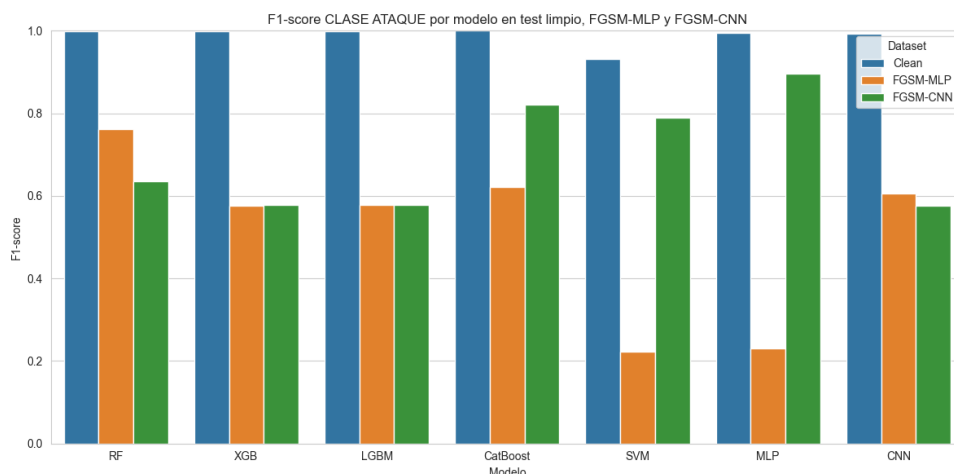


Figura 6.10. F1-FGSM-IOT-23

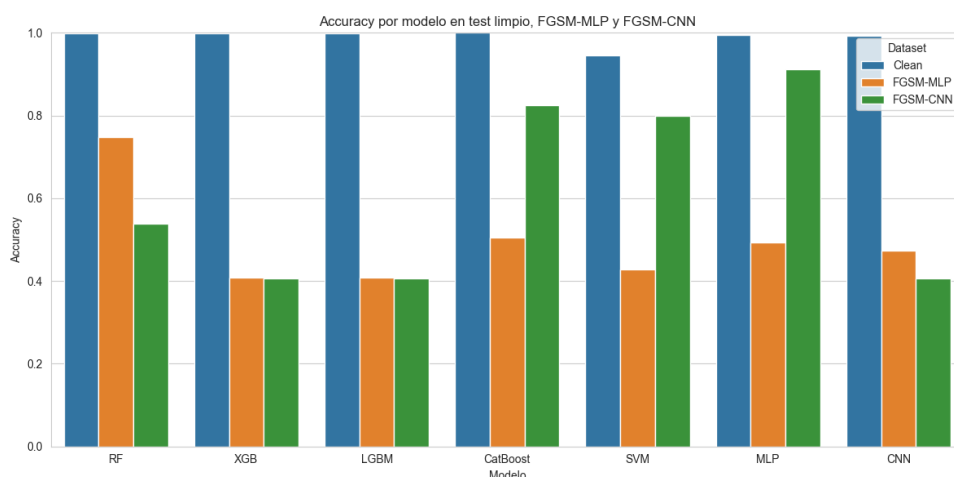


Figura 6.11. Accuracy-FGSM-IOT-23

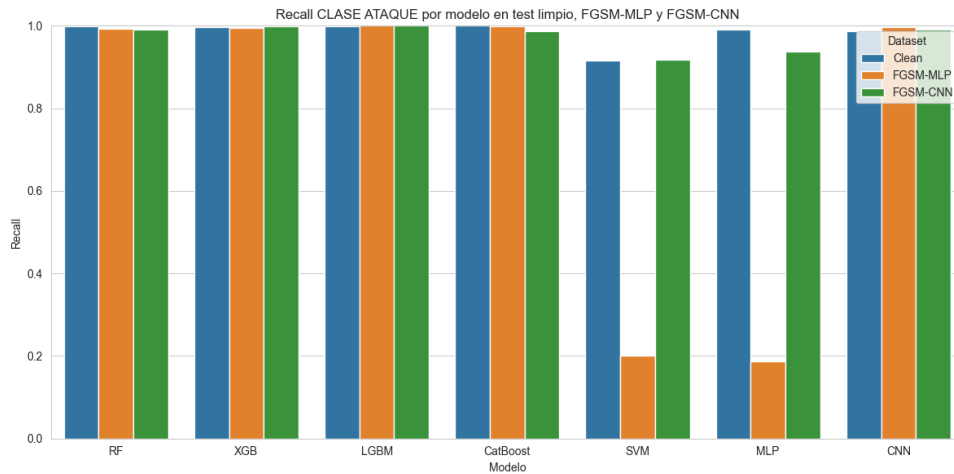


Figura 6.12. Recall-FGSM-IOT-23

Aquí observamos una degradación notable del rendimiento respecto al test limpio. Esta caída se aprecia principalmente en la **accuracy** y en el **F1-score de la clase ataque**.

Respecto al F1-score de la clase ataque, se observa una reducción clara frente al escenario limpio. El caso más llamativo se produce en SVM y MLP bajo FGSM-MLP, donde el F1-score cae de forma muy pronunciada. En cambio, bajo FGSM-CNN, modelos como CatBoost, SVM y MLP mantienen valores considerablemente más altos, lo que evidencia que el efecto del ataque depende tanto del modelo atacado como del modelo usado para generar la perturbación.

El *recall* de la clase ataque se mantiene prácticamente perfecto en la mayoría de modelos, especialmente en RF, XGB, LGBM, CatBoost y CNN. Sin embargo, aparecen caídas muy fuertes en SVM y MLP cuando las perturbaciones se generan sobre MLP. Esto indica que, en estos casos concretos, FGSM no solo incrementa los falsos positivos, sino que también provoca que una parte importante de los ataques deje de ser detectada correctamente.

6.1.4 Análisis de Resultados en TON-IOT

6.1.4.1 Resultados PGD

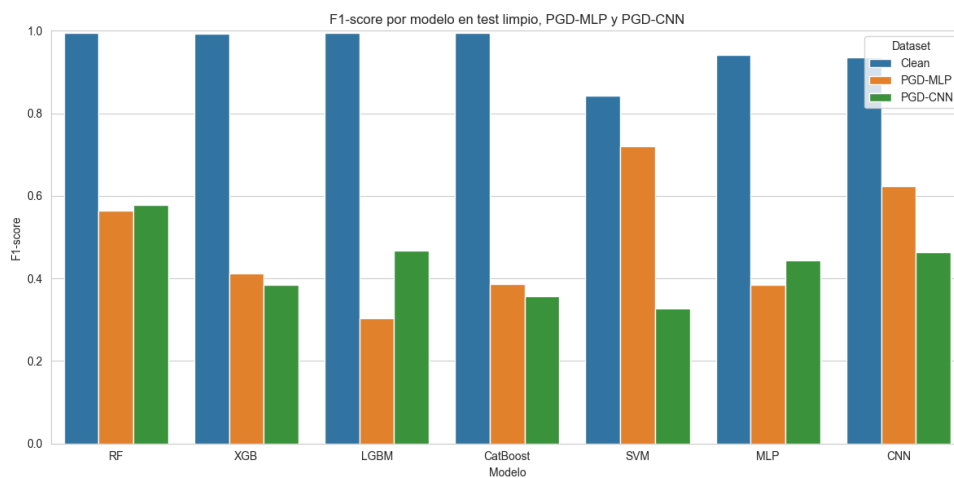


Figura 6.13. F1-PGD-TON-IOT

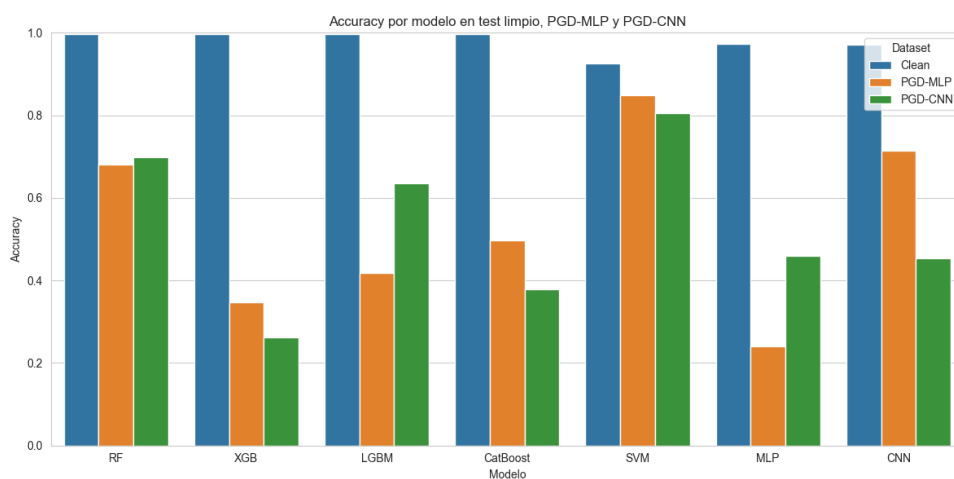


Figura 6.14. Accuracy-PGD-TON-IOT

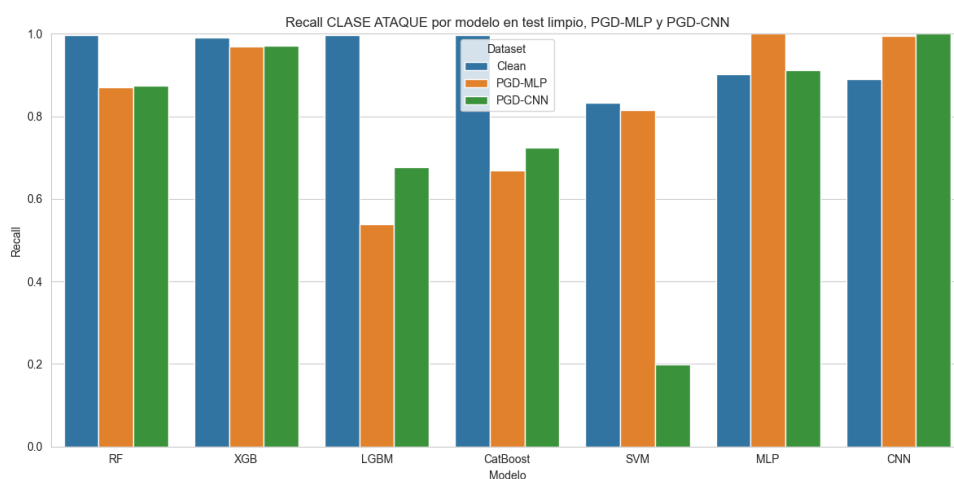


Figura 6.15. Recall-PGD-TON-IOT

En el *dataset* TON-IoT, para el ataque PGD, se observa una degradación significativa del rendimiento respecto al test limpio, donde la mayoría de modelos presentan valores muy elevados de *accuracy* y F1-score, cercanos a 1. Sin embargo, tras aplicar las perturbaciones PGD, ambos indicadores descienden de forma clara en la mayoría de clasificadores.

En términos de **accuracy**, los modelos más afectados son XGB, LGBM, CatBoost y MLP.

Respecto al **F1-score**, la degradación es todavía más evidente. Modelos como LGBM, CatBoost y MLP sufren caídas pronunciadas, lo que indica que el ataque afecta de forma importante al equilibrio entre precisión y *recall* de la clase ataque.

El *recall* de la clase ataque presenta un comportamiento desigual. En algunos modelos, como XGB, MLP y CNN, se mantiene elevado bajo determinadas perturbaciones, lo que indica que siguen detectando gran parte de los ataques. Sin embargo, en otros casos se producen caídas importantes, especialmente en LGBM con PGD-MLP, CatBoost con PGD-MLP y SVM con PGD-CNN.

En conjunto, los resultados muestran que PGD con $\epsilon = 0.05$ tiene un impacto relevante sobre TON-IoT. A diferencia de otros *datasets* donde el *recall* se mantenía alto en casi todos los modelos, aquí el ataque también reduce la capacidad de detección de ataques en determinados clasificadores. Esto sugiere que TON-IoT presenta una mayor sensibilidad frente a perturbaciones adversarias, especialmente en modelos como LGBM, CatBoost, MLP y SVM dependiendo del modelo generador de la perturbación.

6.1.4.2 Resultados FGSM

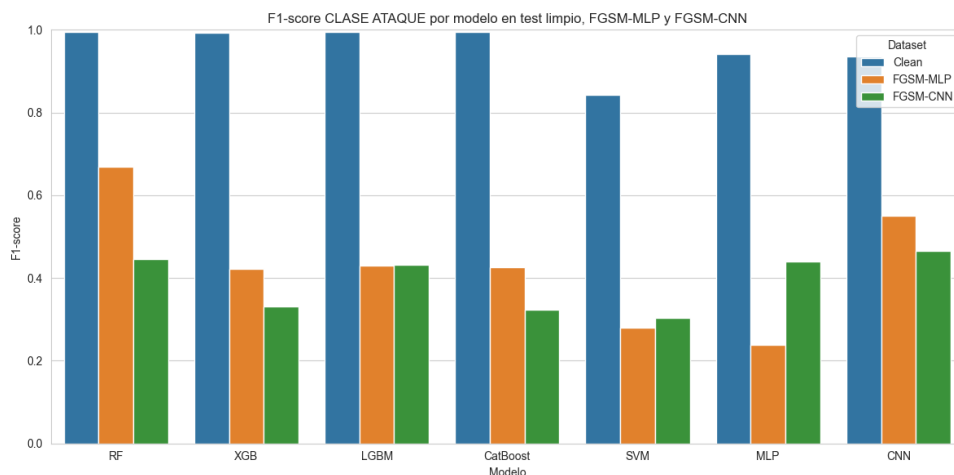


Figura 6.16. F1-FGSM-TON-IOT

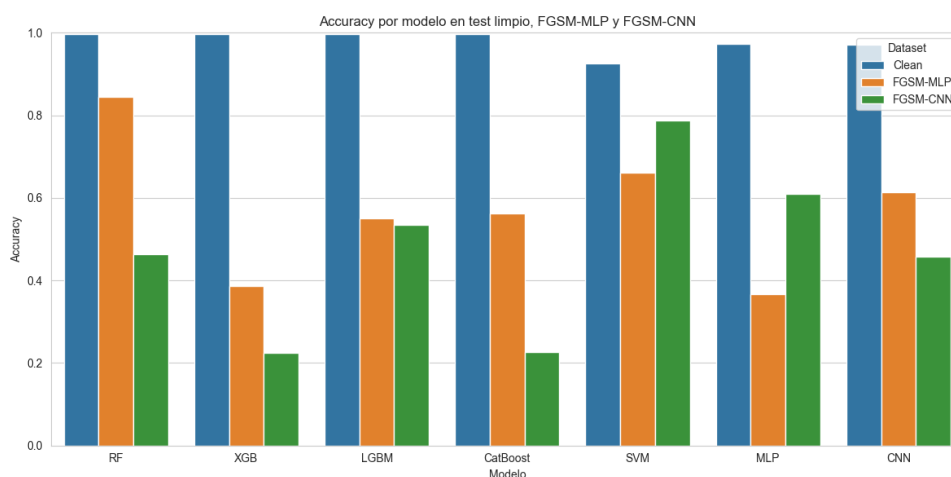


Figura 6.17. Accuracy-FGSM-TON-IOT

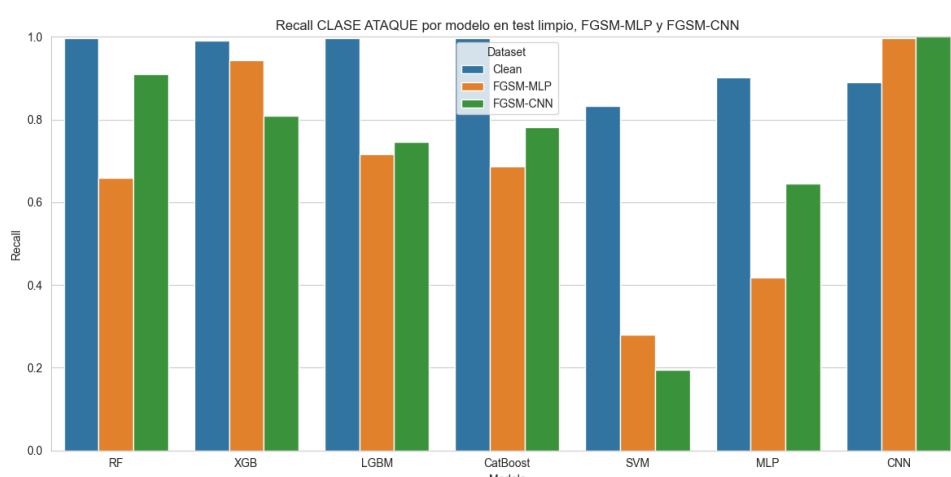


Figura 6.18. Recall-FGSM-TON-IOT

Finalmente, para el ataque FGSM, en el escenario limpio, la mayoría de modelos presentan valores muy elevados de *accuracy* y F1-score, cercanos a 1. Sin embargo, tras aplicar las perturbaciones FGSM, ambos indicadores descienden de forma clara en prácticamente todos los clasificadores.

En términos de **accuracy**, la caída es especialmente acusada en XGB, CatBoost y CNN cuando las perturbaciones se generan sobre CNN. También se observa una degradación importante con FGSM-MLP, especialmente en XGB, LGBM, CatBoost y MLP.

Respecto al **F1-score de la clase ataque**, la degradación es muy notable en todos los modelos.

El *recall* de la clase ataque presenta un comportamiento desigual. Algunos modelos, como CNN, mantienen un *recall* muy elevado e incluso cercano a 1 bajo perturbación, lo que indica que siguen detectando la mayoría de ataques. Sin embargo, otros modelos sufren caídas importantes, como SVM y MLP.

En resumen, hemos visto como un límite de perturbación de ataque pequeño, como 0.05, puede generar

caídas bastante grandes en las métricas analizadas, siempre dependiendo del modelo y del *dataset* perturbado que se aplique.

6.2 Análisis Límite de Perturbación

Los *datasets* perturbados con PGD y FGSM se generaron inicialmente con un límite de perturbación de $\epsilon = 0.05$. Sin embargo, resulta relevante analizar cómo varía la robustez de los modelos a medida que se incrementa dicho límite. Para ello, se ha realizado un análisis adicional sobre los *datasets*, evaluando **diferentes valores de ϵ** : 0.01, 0.03, 0.05, 0.075, 0.1, 0.2 y 0.5. En cada caso, se ha calculado el F1-score asociado a la clase ataque, con el objetivo de estudiar la relación entre la magnitud de la perturbación y la degradación del rendimiento de los modelos.

Con este análisis se pretende identificar el valor de ϵ a partir del cual cada modelo presenta una degradación significativa. Para ello, se ha tomado como referencia el umbral $F1 = 0.5$, ya que un valor inferior indica una pérdida importante en la capacidad del modelo para detectar correctamente la clase maliciosa. En las tablas siguientes, los valores resaltados en **negrita** indican el primer punto en el que cada modelo cae por debajo de dicho umbral.

6.2.1 Resultados UNSW-NB15

6.2.1.1 FGSM

Tabla 6.1. F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo MLP en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7290	0.7208	0.6652	0.7136	0.6677	0.7111	0.7294
0.030	0.7071	0.7202	0.6420	0.7091	0.6219	0.6851	0.6848
0.050	0.7082	0.6841	0.6411	0.7191	0.5590	0.7071	0.6765
0.075	0.7094	0.6774	0.6250	0.7020	0.4503	0.7080	0.6710
0.100	0.7125	0.6840	0.6015	0.6889	0.4097	0.7090	0.6680
0.200	0.7062	0.6608	0.5387	0.7181	0.3791	0.7097	0.6759
0.500	0.7112	0.6626	0.5239	0.7383	0.3811	0.7085	0.6995

Tabla 6.2. F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo CNN en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7255	0.7287	0.6826	0.6985	0.7431	0.7321	0.7156
0.030	0.7013	0.6789	0.6688	0.7076	0.7201	0.4889	0.7034
0.050	0.7067	0.5639	0.6706	0.6913	0.6822	0.4581	0.6882
0.075	0.7178	0.6979	0.6710	0.6299	0.6418	0.4983	0.6846
0.100	0.7194	0.6727	0.6745	0.6496	0.6396	0.5328	0.6847
0.200	0.7159	0.6564	0.6409	0.7293	0.6441	0.5298	0.6841
0.500	0.6806	0.6410	0.6032	0.7404	0.6424	0.5378	0.6847

6.2.1.2 PGD

Tabla 6.3. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7210	0.6873	0.7124	0.6579	0.7046	0.7165	0.7772
0.030	0.7196	0.7019	0.6780	0.6922	0.7054	0.7110	0.7162
0.050	0.7113	0.7075	0.7251	0.7095	0.6965	0.7103	0.7157
0.075	0.7093	0.7073	0.6508	0.6874	0.7105	0.7102	0.7103
0.100	0.7084	0.7000	0.6101	0.7091	0.7145	0.7102	0.7100
0.200	0.7089	0.6983	0.7188	0.7037	0.7102	0.7102	0.7083
0.500	0.7098	0.7087	0.4985	0.7102	0.7102	0.7102	0.7137

Tabla 6.4. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7247	0.7199	0.7669	0.6825	0.7444	0.7384	0.7156
0.030	0.6778	0.7015	0.7192	0.6880	0.6099	0.7157	0.7150
0.050	0.7013	0.7071	0.7073	0.7060	0.7053	0.7068	0.7102
0.075	0.7133	0.7124	0.7191	0.7003	0.6629	0.7126	0.7102
0.100	0.7160	0.6833	0.7201	0.7110	0.7142	0.6717	0.7102
0.200	0.7215	0.7111	0.7302	0.7036	0.6758	0.6134	0.7107
0.500	0.6781	0.7310	0.7148	0.6915	0.6761	0.6159	0.6918

En el caso de UNSW-NB15, los resultados muestran una degradación moderada del F1-score a medida que se introducen perturbaciones adversarias. Para FGSM, se observa que la mayoría de modelos

mantienen valores superiores al umbral de 0.5 incluso con perturbaciones elevadas, aunque aparecen casos concretos de vulnerabilidad, como SVM cuando la perturbación se genera sobre MLP y MLP cuando la perturbación se genera sobre CNN. En cambio, PGD presenta un comportamiento más estable para la mayoría de clasificadores, con caídas menos acusadas y únicamente un descenso por debajo del umbral en LGBM para perturbaciones generadas sobre MLP con $\epsilon = 0.5$.

6.2.2 Resultados IOT-23

6.2.2.1 FGSM

Tabla 6.5. *F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP*

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6736	0.6092	0.5789	0.5784	0.9437	0.8052	0.9484
0.030	0.7356	0.5764	0.5790	0.5834	0.7617	0.5243	0.7623
0.050	0.7620	0.5771	0.5786	0.6219	0.2217	0.2314	0.6059
0.075	0.7619	0.5766	0.5786	0.6182	0.2203	0.1808	0.5887
0.100	0.7793	0.5768	0.5789	0.6119	0.2197	0.1786	0.6225
0.200	0.5790	0.6088	0.6097	0.5916	0.2155	0.1782	0.5888
0.500	0.5781	0.6085	0.6096	0.5916	0.2155	0.1782	0.5829

Tabla 6.6. *F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN*

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6318	0.6104	0.5769	0.5786	0.8853	0.8944	0.9346
0.030	0.6167	0.5778	0.5769	0.6350	0.7888	0.8942	0.7642
0.050	0.6355	0.5777	0.5782	0.8217	0.7888	0.8958	0.5756
0.075	0.6541	0.5775	0.5782	0.8072	0.7634	0.8832	0.6223
0.100	0.6234	0.5769	0.5776	0.7083	0.7605	0.8711	0.6581
0.200	0.5826	0.6267	0.6268	0.5784	0.7602	0.8703	0.6012
0.500	0.5717	0.6194	0.6200	0.5783	0.7611	0.8588	0.5685

6.2.2.2 PGD

Tabla 6.7. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6736	0.6092	0.5789	0.5784	0.9437	0.8052	0.9484
0.030	0.7356	0.5764	0.5790	0.5834	0.7617	0.5243	0.7623
0.050	0.7620	0.5771	0.5786	0.6219	0.2217	0.2314	0.6059
0.075	0.7619	0.5766	0.5786	0.6182	0.2203	0.1808	0.5887
0.100	0.7793	0.5768	0.5789	0.6119	0.2197	0.1786	0.6225
0.200	0.5790	0.6088	0.6097	0.5916	0.2155	0.1782	0.5888
0.500	0.5781	0.6085	0.6096	0.5916	0.2155	0.1782	0.5829

Tabla 6.8. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6318	0.6104	0.5769	0.5786	0.8853	0.8944	0.9346
0.030	0.6167	0.5778	0.5769	0.6350	0.7888	0.8942	0.7642
0.050	0.6355	0.5777	0.5782	0.8217	0.7888	0.8958	0.5756
0.075	0.6541	0.5775	0.5782	0.8072	0.7634	0.8832	0.6223
0.100	0.6234	0.5769	0.5776	0.7083	0.7605	0.8711	0.6581
0.200	0.5826	0.6267	0.6268	0.5784	0.7602	0.8703	0.6012
0.500	0.5717	0.6194	0.6200	0.5783	0.7611	0.8588	0.5685

En IoT-23, la caída respecto al escenario limpio es mucho más acusada que en UNSW-NB15, especialmente porque los modelos parten de valores de F1-score muy elevados, cercanos a 1. Tanto FGSM como PGD provocan una reducción clara del rendimiento en varios clasificadores, aunque el impacto no es homogéneo. En particular, SVM y MLP presentan caídas muy pronunciadas cuando las perturbaciones se generan sobre MLP, llegando a valores claramente inferiores al umbral de 0.5. Por el contrario, cuando las perturbaciones se generan sobre CNN, modelos como SVM y MLP mantienen valores más altos, lo que evidencia una transferencia adversaria desigual.

6.2.3 Resultados TON-IOT

6.2.3.1 FGSM

Tabla 6.9. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.6799	0.5647	0.6577	0.3210	0.4029	0.3369	0.8454
0.030	0.6425	0.5620	0.6391	0.4003	0.3612	0.2857	0.6518
0.050	0.6690	0.5517	0.5496	0.4257	0.2852	0.2861	0.6237
0.075	0.6807	0.5510	0.6003	0.4358	0.2329	0.2867	0.6239
0.100	0.6630	0.5453	0.6004	0.4570	0.2034	0.3372	0.5768
0.200	0.5255	0.5568	0.5666	0.3917	0.2041	0.3531	0.7194
0.500	0.5082	0.5327	0.6206	0.3025	0.2044	0.2107	0.6785

Tabla 6.10. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.5548	0.6436	0.8253	0.3025	0.3217	0.4965	0.7411
0.030	0.4435	0.6053	0.7713	0.3184	0.3209	0.4768	0.6186
0.050	0.4459	0.6052	0.7218	0.3238	0.3066	0.4412	0.5818
0.075	0.4469	0.6118	0.7506	0.3308	0.2954	0.4406	0.5817
0.100	0.4510	0.6099	0.7504	0.3363	0.2888	0.4637	0.5781
0.200	0.4571	0.6002	0.6939	0.3458	0.2204	0.4591	0.5447
0.500	0.4643	0.6905	0.6761	0.3386	0.2178	0.3854	0.6591

6.2.3.2 PGD

Tabla 6.11. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.7322	0.4618	0.6258	0.4997	0.7548	0.3946	0.9192
0.030	0.5675	0.4086	0.5527	0.2887	0.7922	0.4104	0.6605
0.050	0.5725	0.3994	0.5433	0.3501	0.7608	0.3850	0.6488
0.075	0.5221	0.3537	0.6107	0.3290	0.6546	0.3831	0.6317
0.100	0.4931	0.3531	0.5877	0.3367	0.5737	0.3830	0.6114
0.200	0.4590	0.3066	0.5526	0.2307	0.4415	0.3826	0.4588
0.500	0.3671	0.3278	0.4360	0.1261	0.3610	0.3826	0.0817

Tabla 6.12. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.5529	0.5838	0.7368	0.3758	0.3273	0.5400	0.7410
0.030	0.5078	0.6789	0.7228	0.3322	0.3208	0.4126	0.6080
0.050	0.5977	0.4666	0.6414	0.4006	0.4115	0.4366	0.5804
0.075	0.4611	0.4123	0.5110	0.3748	0.4057	0.4201	0.5501
0.100	0.5492	0.4199	0.4765	0.4390	0.4901	0.4025	0.5360
0.200	0.4576	0.4317	0.4569	0.3485	0.4224	0.3734	0.5026
0.500	0.4910	0.3701	0.4295	0.4782	0.6064	0.3919	0.4413

En TON-IoT se observa la degradación más severa entre los tres *datasets* analizados. En este caso, tanto FGSM como PGD consiguen reducir el F1-score por debajo del umbral de 0.5 en numerosos modelos, incluso para valores bajos de ϵ . Con FGSM, la caída es especialmente notable en modelos como CatBoost, SVM y MLP, que muestran valores inferiores a 0.5 desde los primeros niveles de perturbación. Además, cuando la perturbación se genera sobre CNN, también se observa una afectación temprana en RF y también en CatBoost, SVM y MLP.

PGD presenta un comportamiento todavía más agresivo en TON-IoT. Varios modelos caen por debajo de 0.5 desde $\epsilon = 0.01$, especialmente XGBoost, CatBoost y MLP cuando la perturbación se genera sobre MLP, así como CatBoost y SVM cuando se genera sobre CNN. A diferencia de UNSW-NB15, donde muchos clasificadores mantenían cierta estabilidad, en TON-IoT la mayoría de mecanismos de detección muestran una sensibilidad elevada frente a perturbaciones adversarias. Esto sugiere que las características de este *dataset* generan fronteras de decisión más vulnerables o más fácilmente alterables por pequeñas modificaciones en las variables de entrada.

6.2.4 Conclusiones del análisis del límite de perturbación

A partir de los resultados obtenidos, es curioso ver cómo el incremento del límite de perturbación no produce siempre una degradación estrictamente monótona del F1-score. Aunque cabría esperar que valores mayores de ϵ provocasen una caída progresiva del rendimiento, en varios modelos aparecen comportamientos irregulares, con recuperaciones parciales del F1-score para perturbaciones superiores. Esto se debe a que aplicar tanta perturbación puede destruir el camuflaje del paquete, por lo que el modelo detecta una deformación tan extrema que lo clasifica inmediatamente como una anomalía o ataque.

El análisis también muestra que la robustez adversaria depende de varios factores simultáneamente: el tipo de ataque utilizado, el modelo sobre el que se genera la perturbación, el clasificador evaluado y el *dataset* empleado. En este sentido, no puede afirmarse que FGSM o PGD sean siempre más perjudiciales en todos los casos, ya que el impacto varía según la combinación concreta de ataque y modelo. Del mismo modo, las perturbaciones generadas sobre MLP y CNN no se transfieren con la

misma intensidad a todos los mecanismos de detección.

Conclusiones y Líneas Futuras

7.1 Conclusiones

El presente Trabajo de Fin de Grado tenía el objetivo de abordar el análisis de modelos de aprendizaje automático y aprendizaje profundo aplicados a la detección de intrusiones, considerando no solo su rendimiento predictivo, sino también su eficiencia computacional, su comportamiento frente a ataques adversarios y su posible viabilidad en escenarios de despliegue real. Para ello, se ha realizado una evaluación experimental sobre tres *datasets* diferentes, UNSW-NB15, IoT-23 y TON-IoT, y se han comparado siete familias de modelos: Random Forest, XGBoost, LightGBM, CatBoost, SVM, MLP y CNN-1D.

En primer lugar, *la optimización multiobjetivo mediante Optuna ha permitido seleccionar configuraciones de hiperparámetros atendiendo simultáneamente al F1-score y a la latencia de inferencia*. Esta parte ha sido especialmente relevante, ya que no se ha realizado una selección manual de modelos, sino un proceso sistemático de búsqueda para cada combinación de modelo y *dataset*. Para ello, se han definido espacios de búsqueda específicos para cada algoritmo, centrados principalmente en hiperparámetros estructurales que afectan directamente a la complejidad del modelo y a su coste de inferencia. El uso de la frontera de Pareto ha resultado útil para identificar configuraciones equilibradas, evitando seleccionar modelos excesivamente complejos que apenas aportan mejoras predictivas pero incrementan el coste computacional. De esta forma, la elección de los candidatos finales no se ha basado únicamente en maximizar el F1-score, sino en encontrar un equilibrio razonable entre capacidad de detección y eficiencia temporal.

En la comparativa entre modelos, *los métodos basados en árboles han mostrado el comportamiento más equilibrado*. Random Forest, XGBoost, LightGBM y CatBoost han obtenido valores elevados de F1-score en los tres *datasets*, manteniendo además una eficiencia temporal muy competitiva. Este resultado es especialmente importante porque confirma que, para datos tabulares de tráfico de red, los modelos de tipo *ensemble* pueden ofrecer una combinación muy sólida entre precisión, latencia

y *throughput*. SVM ha destacado por su baja latencia y reducido coste de inferencia, aunque su rendimiento predictivo ha sido inferior al de los modelos no lineales, especialmente en UNSW-NB15. Por su parte, MLP y CNN-1D han alcanzado resultados competitivos, pero con un mayor coste de inferencia, especialmente en el caso de CNN-1D.

Los resultados también muestran diferencias importantes entre *datasets*. En IoT-23 y TON-IoT los modelos han alcanzado valores de F1-score muy elevados en condiciones limpias, mientras que UNSW-NB15 ha presentado un escenario más exigente, con resultados más moderados. Esto confirma que *el comportamiento de un IDS no depende únicamente del algoritmo utilizado, sino también del tipo de tráfico, la distribución de clases y las características propias de cada dataset*. Esta comparación entre conjuntos de datos ha permitido comprobar que un modelo puede comportarse de forma muy diferente según el escenario evaluado. Por tanto, evaluar únicamente sobre un dataset podría llevar a conclusiones incompletas o demasiado optimistas sobre la capacidad real del modelo.

El análisis frente a ataques de evasión mediante FGSM y PGD ha puesto de manifiesto que *un buen rendimiento en test limpio no garantiza necesariamente robustez adversaria*. Algunos modelos que obtienen valores elevados de F1-score en condiciones normales sufren caídas importantes al evaluar muestras perturbadas. Además, el impacto de los ataques no es uniforme: depende del *dataset*, del modelo atacado, del modelo utilizado para generar las perturbaciones y del límite de perturbación considerado. También se ha observado que el incremento de ϵ no siempre produce una degradación estrictamente monótona, especialmente en datos tabulares, donde perturbaciones elevadas pueden desplazar las muestras fuera de la distribución original o generar efectos de saturación. Esta parte del trabajo permite reforzar la idea de que la evaluación de un IDS no debe limitarse a datos limpios, ya que en un escenario real un atacante puede intentar modificar el tráfico para evadir la detección.

Desde el punto de vista computacional, los resultados de inferencia muestran que muchos de los modelos evaluados presentan un *throughput* elevado, tanto en servidor como en un equipo local con menos recursos. Esto sugiere que, una vez entrenados y cargados en memoria, *varios modelos clásicos podrían ser viables para escenarios de análisis en tiempo real*. Además, la comparación entre las mediciones realizadas en el servidor y en el equipo local ha permitido diferenciar entre el coste de un *benchmark* instrumentado, donde se monitorizan recursos como CPU y RAM, y una medición centrada exclusivamente en inferencia pura.

En conjunto, los resultados obtenidos permiten concluir que *la selección de un modelo IDS no debe basarse únicamente en maximizar una métrica predictiva*. Es necesario considerar de forma conjunta la distribución del *dataset*, la capacidad de detección, la latencia, el *throughput*, el consumo de recursos y la robustez frente a ataques adversarios.

7.2 Líneas Futuras

Esta línea de trabajo ha permitido obtener una visión integral del rendimiento y la eficiencia de distintos modelos de aprendizaje automático y profundo aplicados a la detección de intrusiones. Sin embargo,

existen múltiples caminos de interés para continuar explorando y profundizando en esta área.

Incorporar y analizar modelos basados en secuencias temporales (como LSTM) podría ser especialmente relevante para capturar patrones de comportamiento a lo largo del tiempo, lo que es crucial en la detección de intrusiones que pueden manifestarse como anomalías temporales.

Además, estudiar estrategias de reentrenamiento incremental permitiría evaluar cómo los modelos pueden adaptarse a nuevos datos sin necesidad de un reentrenamiento completo. Por ejemplo, se podría analizar con qué porcentaje de nuevos datos conviene reentrenar los modelos para mantener un buen rendimiento sin incurrir en un coste computacional excesivo.

Otro camino interesante sería aprovechar la similitud de características de los *datasets* para evaluar la transferencia entre ellos, es decir, entrenar el modelo con un *dataset* y probar la inferencia con otro. Esto permitiría analizar la capacidad de generalización de los modelos y su robustez.

Por último, una línea futura de gran interés sería analizar el entrenamiento para los ataques de evasión, reentrenando los modelos con muestras adversarias para mejorar su robustez frente a este tipo de amenazas, donde como se ha visto en el trabajo, los modelos pueden sufrir una degradación significativa de su eficacia.

Apéndice A. Resultados completos de la optimización de hiperparámetros

En este apéndice se presenta el análisis detallado de los resultados obtenidos durante la fase de optimización de hiperparámetros para cada modelo y *dataset*.

A.1 Modelos entrenados con UNSW-NB15

A.1.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

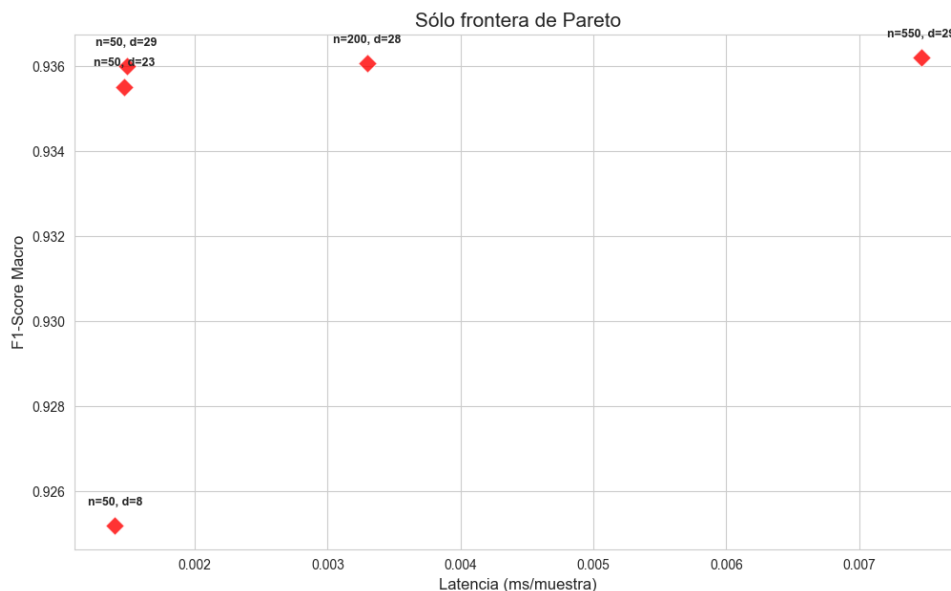


Figura 7.1. Frontera de Pareto – Random Forest

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 23)

2. (100, 29)
3. (200, 28)
4. (550, 29)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	23	0.885392	0.888622	0.001516
Candidato 2	50	29	0.873085	0.877411	0.001486
Candidato 3	200	28	0.876276	0.880338	0.003443
Candidato 4	550	29	0.87463	0.878881	0.007078

Tabla 7.1. Comparativa final de modelos Random Forest en el set de test

El **Candidato 1** no solo es el más preciso, sino también uno de los más rápidos, con una latencia de 0,0015 ms por muestra.

A.1.2 XGBoost

A continuación se muestran los resultados obtenidos por XGBoost:

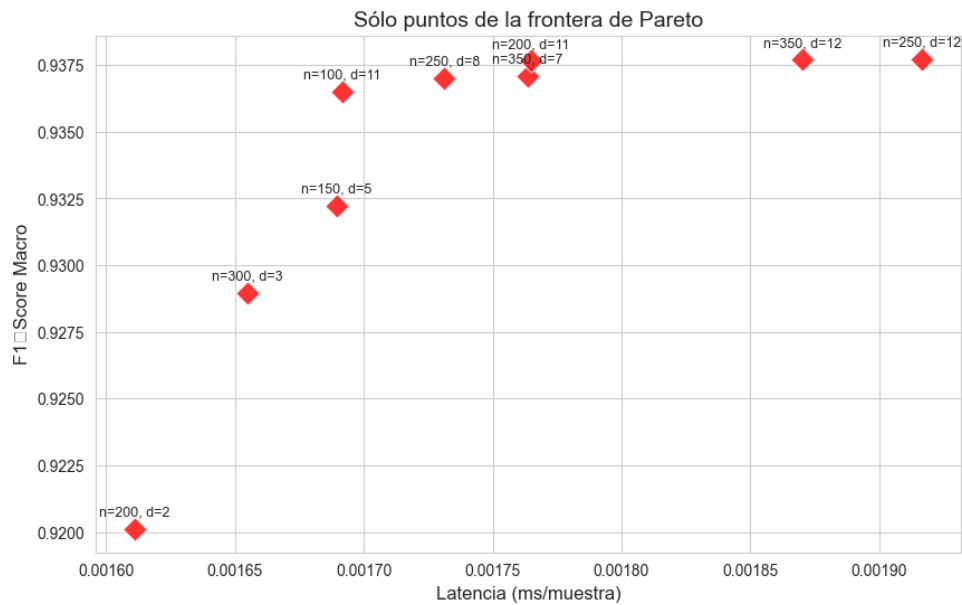


Figura 7.2. Frontera de Pareto - XGBoost

Hemos seleccionando las siguientes combinaciones de la frontera de Pareto para testarlas:

1. (350, 12)
2. (200, 11)
3. (350, 7)
4. (250, 8)

5. (100, 11)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	12	0.858299	0.864378	0.000501
Candidato 2	200	11	0.857905	0.864111	0.000418
Candidato 3	350	7	0.856623	0.863152	0.000561
Candidato 4	250	8	0.856923	0.863322	0.000543
Candidato 5	100	11	0.857487	0.86388	0.000443

Tabla 7.2. Comparativa final de modelos XGBoost en el set de test

Nos quedamos con el **Candidato 2**, que ofrece un excelente equilibrio entre calidad y coste.

A.1.3 LightGBM

LightGBM ha sido entrenado con GPU, obteniendo la siguiente frontera de Pareto:

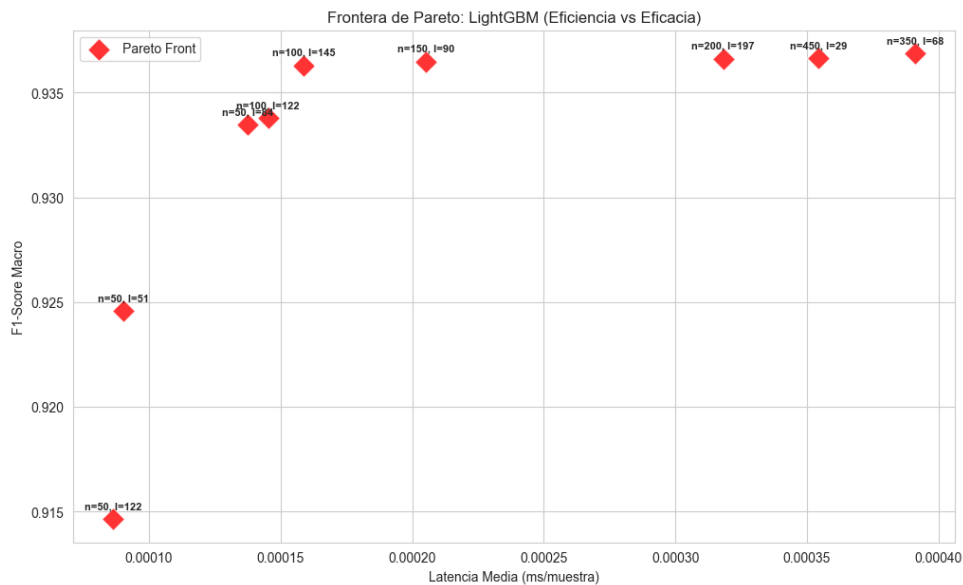


Figura 7.3. Frontera de Pareto - LightGBM

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evaluación en el set de test:

1. (350, 68, 9)
2. (450, 29, 12)
3. (150, 90, 12)
4. (100, 145, 12)
5. (100, 122, 7)

Los resultados obtenidos en el *dataset* de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	68	9	0.861467	0.867354	0.000371
Candidato 2	450	29	12	0.860196	0.86631	0.00036
Candidato 3	150	90	12	0.860457	0.866552	0.0002
Candidato 4	100	145	12	0.859564	0.865775	0.000159
Candidato 5	100	122	7	0.852491	0.859654	0.000137

Tabla 7.3. Comparativa final de modelos *LightGBM* en el set de test

Si tuviéramos que quedarnos con un único modelo, nos quedamos con el **Candidato 4**.

A.1.4 CatBoost

La frontera de Pareto obtenida para CatBoost con el *dataset* UNSW-NB15 es la siguiente:

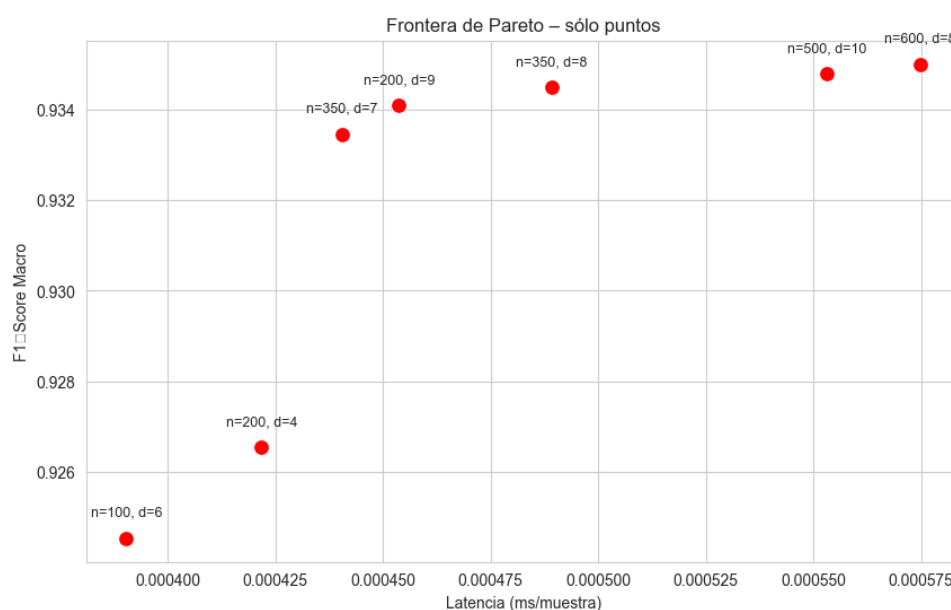


Figura 7.4. Frontera de Pareto – CatBoost

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evaluación en el set de test:

1. (200, 4)
2. (350, 7)
3. (200, 9)
4. (350, 8)
5. (500, 10)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	200	4	0.828623	0.838957	0.000298
Candidato 2	350	7	0.854301	0.861135	0.00044
Candidato 3	200	9	0.85276	0.859787	0.000477
Candidato 4	350	8	0.855648	0.862277	0.000489
Candidato 5	500	10	0.85702	0.863419	0.000375

Tabla 7.4. Comparativa final de modelos CatBoost en el set de test

El dato más revelador de la tabla es la latencia del **Candidato 5** ($n=500$, $d=10$). A pesar de ser el modelo más masivo y profundo de los evaluados, presenta una latencia de 0.000375 ms, siendo más rápido que modelos teóricamente más sencillos como el Candidato 4 ($n=350$, $d=8$, con 0.000489 ms).

A.1.5 SVM

La gráfica de la frontera de Pareto obtenida para SVM:

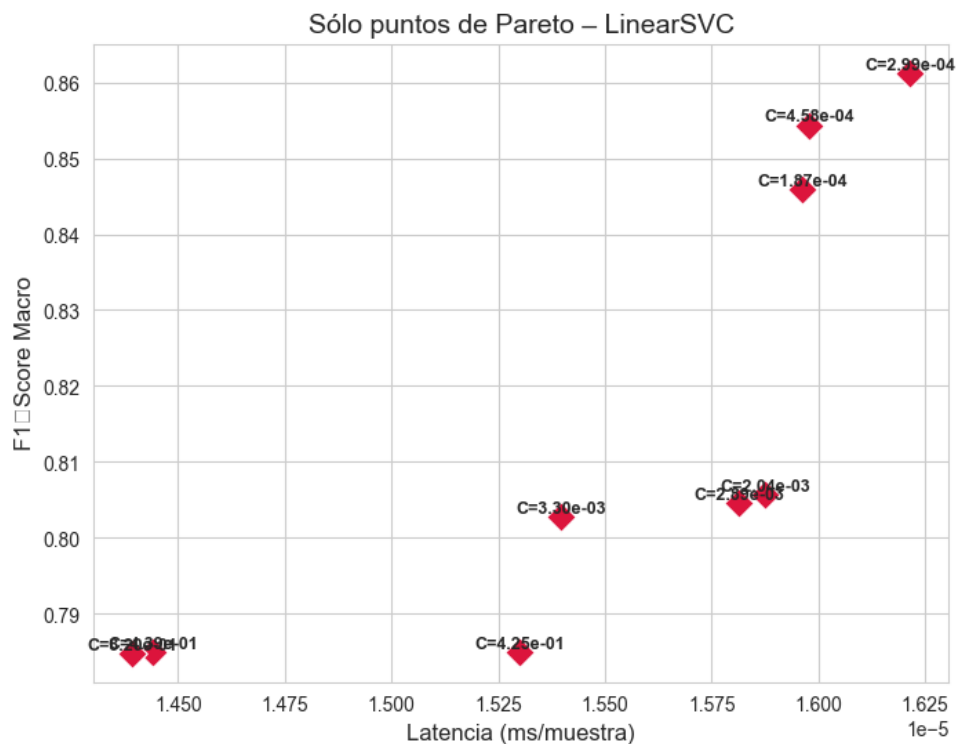


Figura 7.5. Frontera de Pareto – SVM

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	0.000299	0.708654	0.735461	0.000081
Candidato 2	0.000458	0.692147	0.722987	0.000093
Candidato 3	0.000187	0.709219	0.736105	0.000018
Candidato 4	0.002037	0.649204	0.692003	0.000019
Candidato 5	0.002889	0.648003	0.691104	0.000018

Tabla 7.5. Comparativa final de modelos SVM en el set de test

A diferencia de los modelos basados en árboles (RF, XGBoost, LightGBM) que superaban holgadamente el 85% de F1-Score, LinearSVC se estanca en un máximo de 0.7092 (Candidato 3). Esta drástica caída de rendimiento sugiere que las intrusiones de red no son linealmente separables en este espacio de características.

Dentro de las limitaciones del enfoque lineal, el **Candidato 3** (C=0.000187) es el ganador.

A.1.6 MLP

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

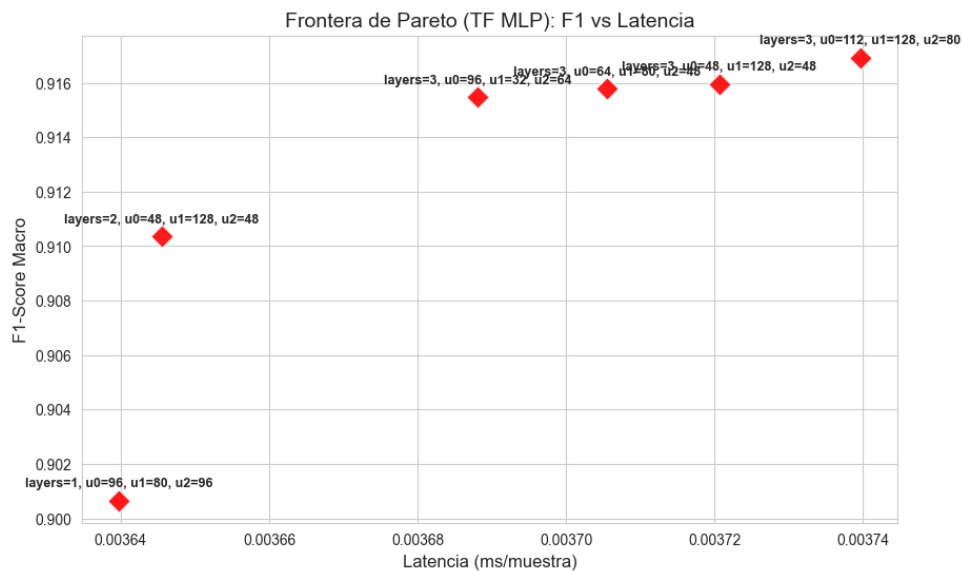


Figura 7.6. Frontera de Pareto - MLP

Ahora mostramos los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	96	80	96	0.777448	0.797017	0.00159
Candidato 2	3	48	128	48	0.758004	0.781397	0.001505
Candidato 3	3	64	80	48	0.782289	0.800515	0.001505
Candidato 4	3	96	32	64	0.786982	0.804147	0.001421

Tabla 7.6. Comparativa final de modelos MLP en el set de test

Seleccionamos el **Candidato 4**, que ofrece el mejor rendimiento (F1-Score de 0.7869) y una latencia de inferencia de 0.001421 ms, lo que lo posiciona como un modelo competitivo dentro de los modelos no lineales, aunque aún por debajo de los basados en árboles.

A.1.7 CNN-1D

Aunque CNN se suele asociar principalmente a datos con estructura espacial (como imágenes), también se ha explorado su aplicación en la detección de intrusiones. Para su implementación, se descartó el uso de secuencias temporales (time steps) asociadas a arquitecturas como LSTM. Los conjuntos de datos empleados proporcionan información ya agregada en flujos de red independientes (formato tabular). Agrupar estas filas de forma secuencial introduciría dependencias temporales ficticias y violaría la asunción de independencia (i.i.d.) de los datos, invalidando la robustez de la evaluación mediante validación cruzada.

En su lugar, se implementó una arquitectura CNN-1D adaptando el vector de características de cada flujo mediante un redimensionamiento (reshape) a un tensor de la forma (muestras, características, 1). Bajo este enfoque, la convolución actúa exclusivamente como un extractor automático de características latentes, descubriendo correlaciones locales no lineales entre las variables tabulares sin asumir una jerarquía temporal estricta. Esta es la gráfica de la frontera de Pareto obtenida para CNN:

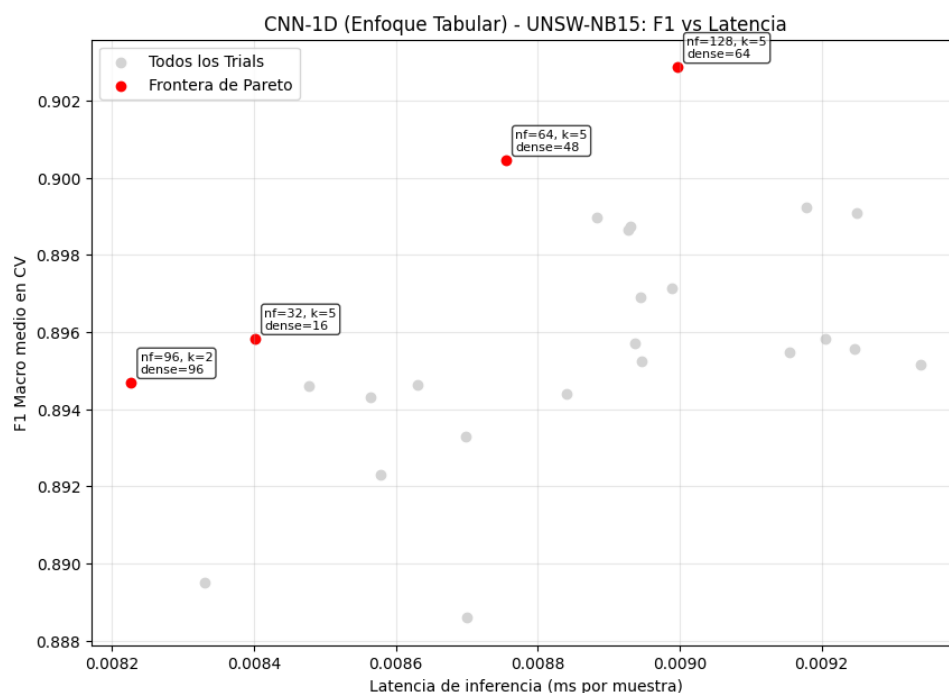


Figura 7.7. Frontera de Pareto - CNN

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	64	5	48	0.773052	0.793045	0.00888
Candidato 2	128	5	64	0.761881	0.784118	0.009398
Candidato 3	96	2	96	0.749724	0.77445	0.008625
Candidato 4	32	5	16	0.738305	0.76551	0.008725

Tabla 7.7. Comparativa final de modelos CNN en el set de test

El **Candidato 1** es el modelo ganador, con un F1-Score de 0.773052 y una latencia de inferencia de 0.00888 ms. Aunque su rendimiento es competitivo, su latencia es significativamente mayor que la de los modelos basados en árboles, lo que podría limitar su aplicabilidad en entornos de producción donde la velocidad de detección es crítica.

A.2 Modelos entrenados con IoT-23

A.2.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

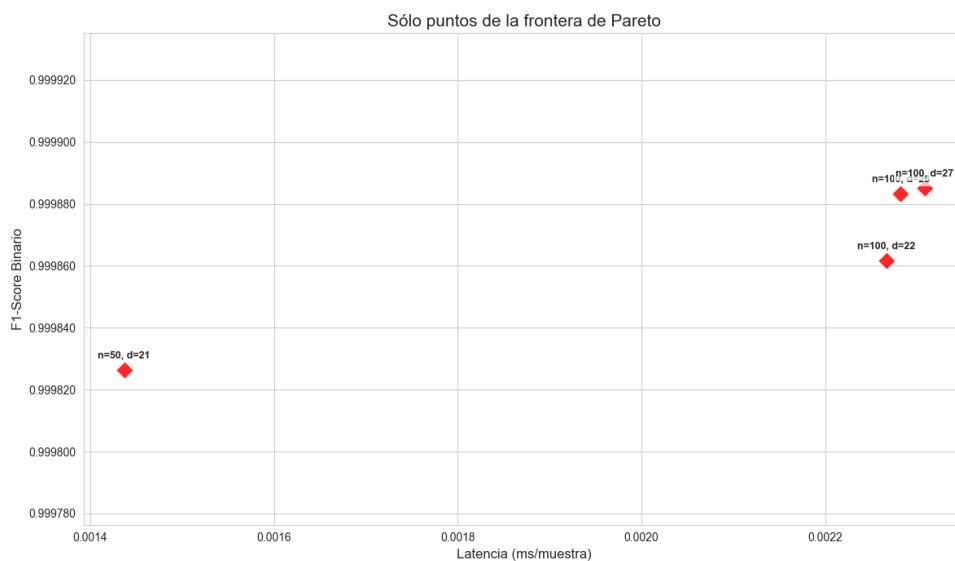


Figura 7.8. Frontera de Pareto - Random Forest (IoT-23)

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 21)
2. (100, 22)
3. (100, 28)
4. (100, 27)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	21	0.999675	0.999614	0.001496
Candidato 2	100	22	0.999855	0.999828	0.002252
Candidato 3	100	28	0.999906	0.999888	0.002363
Candidato 4	100	27	0.999913	0.999897	0.00223

Tabla 7.8. Comparativa final de modelos Random Forest en el set de test (IOT-23)

En este caso, el **Candidato 1** es el modelo más eficiente, con un F1-Score de 0.999675 y una latencia de inferencia de 0.001496 ms, lo que lo posiciona como un modelo preciso y rápido para este dataset específico de IoT.

A.2.2 XGBoost

Para XGBoost, la frontera de Pareto obtenida es la siguiente:

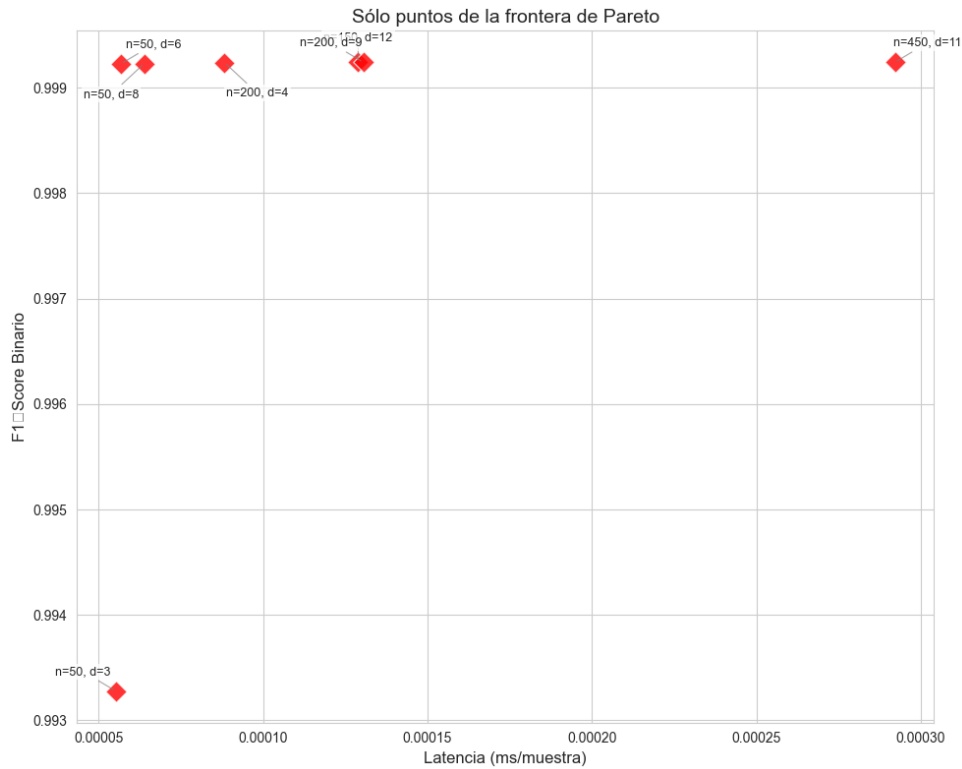


Figura 7.9. Frontera de Pareto - XGBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 8)
2. (50, 6)
3. (200, 4)
4. (200, 9)

5. (150, 12)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	8	0.999114	0.998948	0.000028
Candidato 2	50	6	0.999122	0.998957	0.000027
Candidato 3	200	4	0.999122	0.998957	0.000047
Candidato 4	200	9	0.999132	0.998970	0.000080
Candidato 5	150	12	0.999132	0.998970	0.000080

Tabla 7.9. Comparativa final de modelos XGBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999122 y una latencia de inferencia de 0.000027 ms, lo que lo posiciona como un modelo preciso y rápido en este *dataset* de IoT, superando incluso a los modelos de Random Forest en términos de latencia.

A.2.3 LightGBM

La frontera de Pareto obtenida para LightGBM es la siguiente:

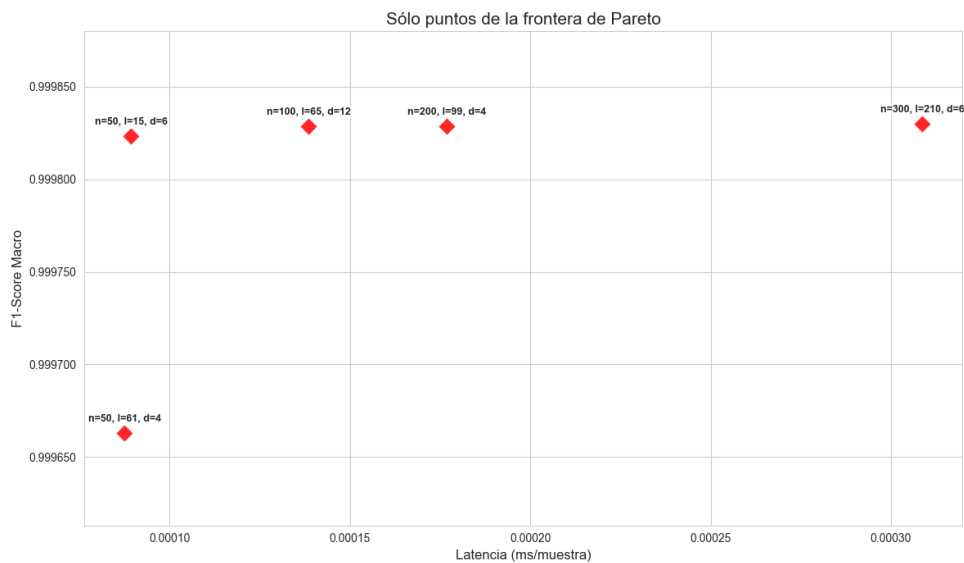


Figura 7.10. Frontera de Pareto - LightGBM (IOT-23)

Para este modelo, se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (50, 15, 6)
2. (100, 65, 12)
3. (200, 99, 4)
4. (50, 61, 4)
5. (300, 210, 6)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	15	6	0.999813	0.99982	0.000057
Candidato 2	100	65	12	0.999831	0.999837	0.000099
Candidato 3	200	99	4	0.999795	0.999803	0.000170
Candidato 4	50	61	4	0.999631	0.999644	0.000058
Candidato 5	300	210	6	0.881924	0.882891	0.000261

Tabla 7.10. Comparativa final de modelos LightGBM en el set de test (IOT-23)

En este caso, el **Candidato 1** es el mejor modelo, con un F1-Score de 0.999813 y una latencia de inferencia de 0.000057 ms.

A.2.4 CatBoost

Este es el gráfico de la frontera de Pareto obtenida para CatBoost:

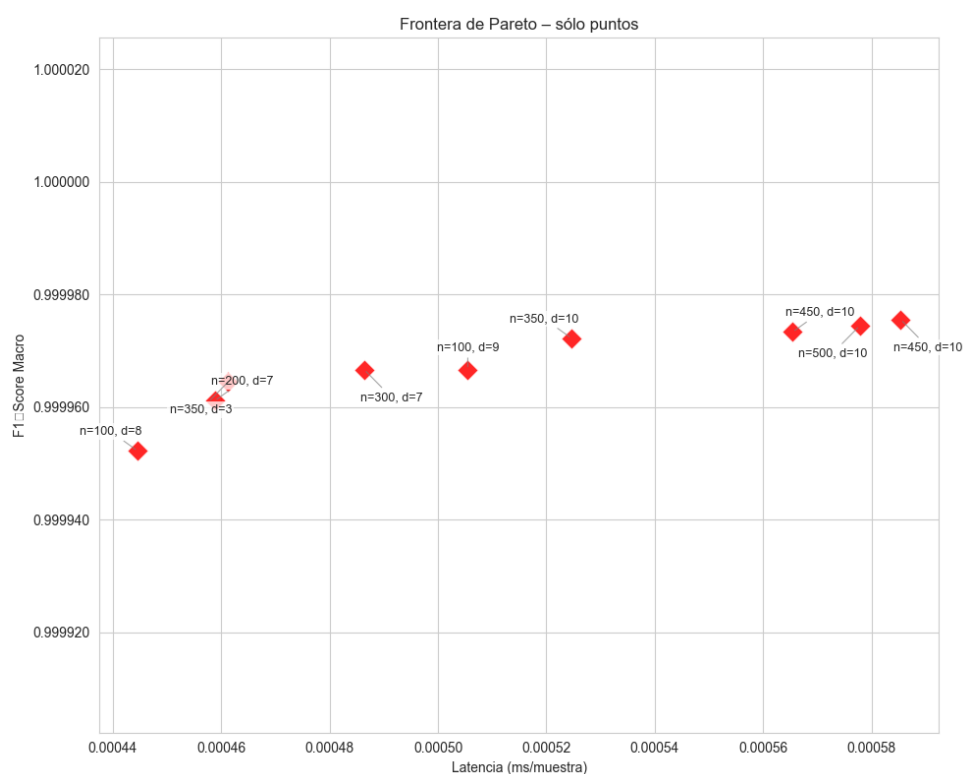


Figura 7.11. Frontera de Pareto - CatBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (100, 8)
2. (350, 3)
3. (200, 7)

4. (300, 7)
5. (100, 9)
6. (350, 10)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	100	8	0.999947	0.999948	0.000242
Candidato 2	350	3	0.999960	0.999961	0.000256
Candidato 3	200	7	0.999951	0.999953	0.000263
Candidato 4	300	7	0.999951	0.999953	0.000250
Candidato 5	100	9	0.999951	0.999953	0.000259
Candidato 6	350	10	0.999951	0.999953	0.000269

Tabla 7.11. Comparativa final de modelos CatBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999960 y una latencia de inferencia de 0.000256 ms, lo que lo posiciona como un modelo bastante bueno.

A.2.5 SVM

La gráfica de la frontera de Pareto obtenida para SVM es la siguiente:

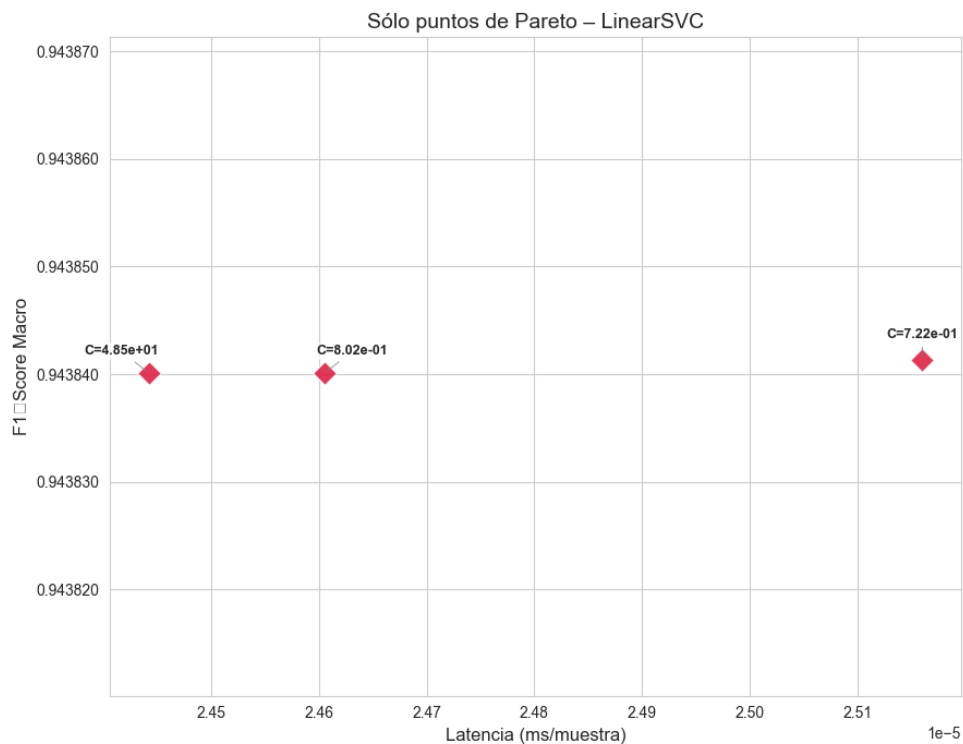


Figura 7.12. Frontera de Pareto – SVM (IOT-23)

Se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (48.498258)

2. (0.801714)
3. (0.721883)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	48.498258	0.943691	0.945963	0.000027
Candidato 2	0.801714	0.943695	0.945968	0.000025
Candidato 3	0.721883	0.943691	0.945963	0.000023

Tabla 7.12. Comparativa final de modelos SVM en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.943695 y una latencia de inferencia de 0.000025 ms, lo que lo posiciona como un modelo muy bueno para este *dataset* de IoT, aunque su rendimiento es significativamente inferior al de los modelos basados en árboles.

A.2.6 MLP

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

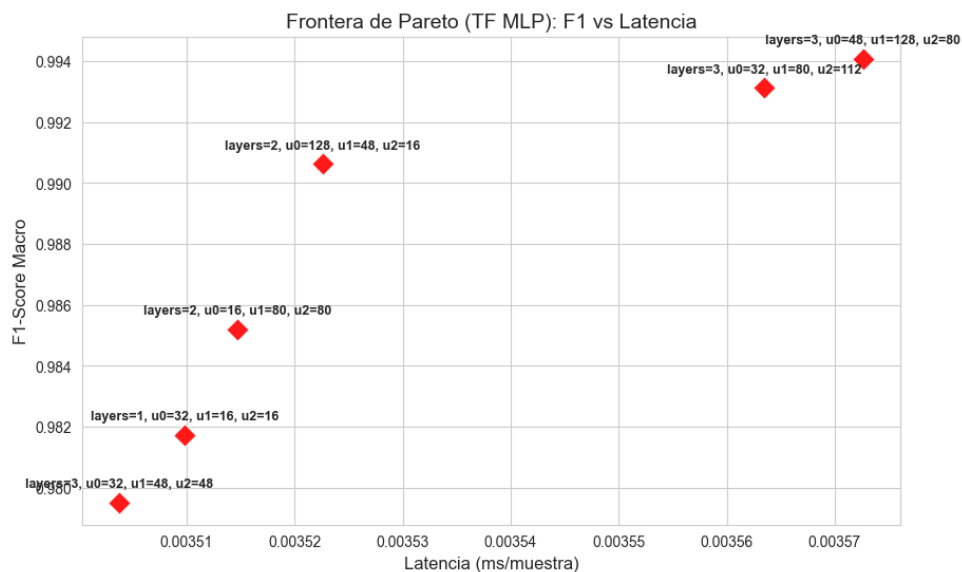


Figura 7.13. Frontera de Pareto - MLP (IOT-23)

La combinación de hiperparámetros seleccionada para su evaluación en el set de test es la siguiente:

1. (3, 32, 48, 48)
2. (1, 32, 0, 0)
3. (2, 16, 80, 0)
4. (2, 128, 48, 0)

5. (3, 32, 80, 112)

6. (3, 48, 128, 80)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	32	48	48	0.994819	0.995004	0.000717
Candidato 2	1	32	0	0	0.985158	0.985741	0.000621
Candidato 3	2	16	80	0	0.990610	0.990969	0.000719
Candidato 4	2	128	48	0	0.993502	0.993737	0.000772
Candidato 5	3	32	80	112	0.995042	0.995223	0.000919
Candidato 6	3	48	128	80	0.996544	0.996665	0.000906

Tabla 7.13. Comparativa final de modelos MLP en el set de test (IOT-23)

En este caso, nos quedamos con el **Candidato 1**, que ofrece un excelente rendimiento (F1-Score de 0.994819) y una latencia de inferencia de 0.000717 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

A.2.7 CNN-1D

La gráfica de la frontera de Pareto obtenida para CNN-1D es la siguiente:

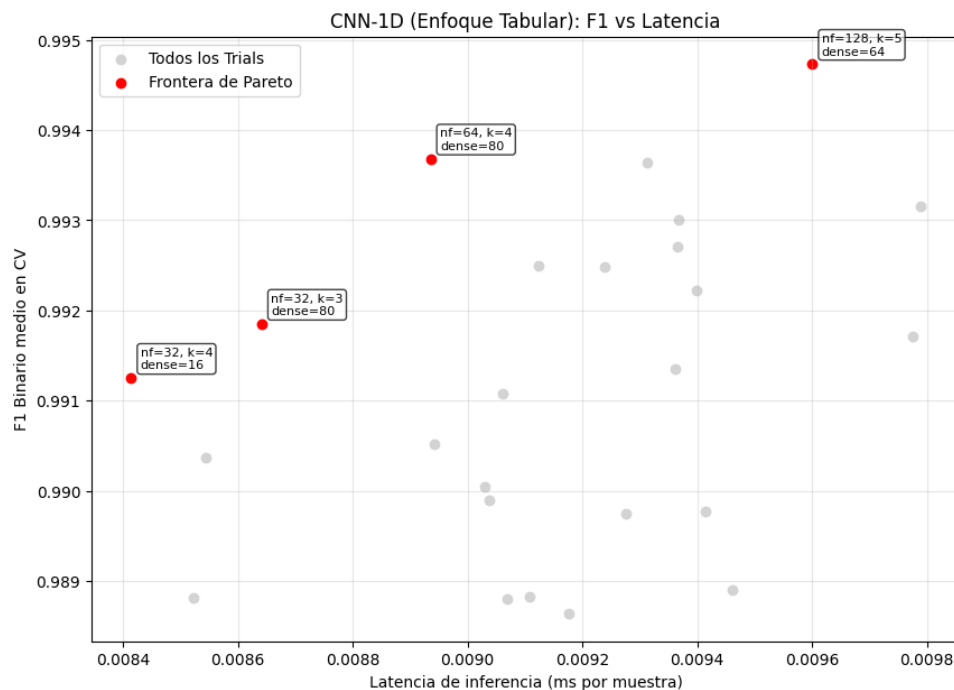


Figura 7.14. Frontera de Pareto - CNN-1D (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (32, 4, 16)
2. (32, 3, 80)
3. (64, 4, 80)
4. (128, 5, 64)

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	32	4	16	0.992844	0.991467	0.008293
Candidato 2	32	3	80	0.988798	0.986561	0.008442
Candidato 3	64	4	80	0.995359	0.99448	0.00892
Candidato 4	128	5	64	0.99583	0.995042	0.009345

Tabla 7.14. Comparativa final de modelos CNN-1D en el set de test (IOT-23)

En este caso, el **Candidato 3** es el modelo más eficiente, con un F1-Score de 0.995359 y una latencia de inferencia de 0.00892 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

A.3 Modelos entrenados con TON-IoT

A.3.1 Random Forest

La frontera de Pareto obtenida para Random Forest con el dataset ToN-IoT es la siguiente:

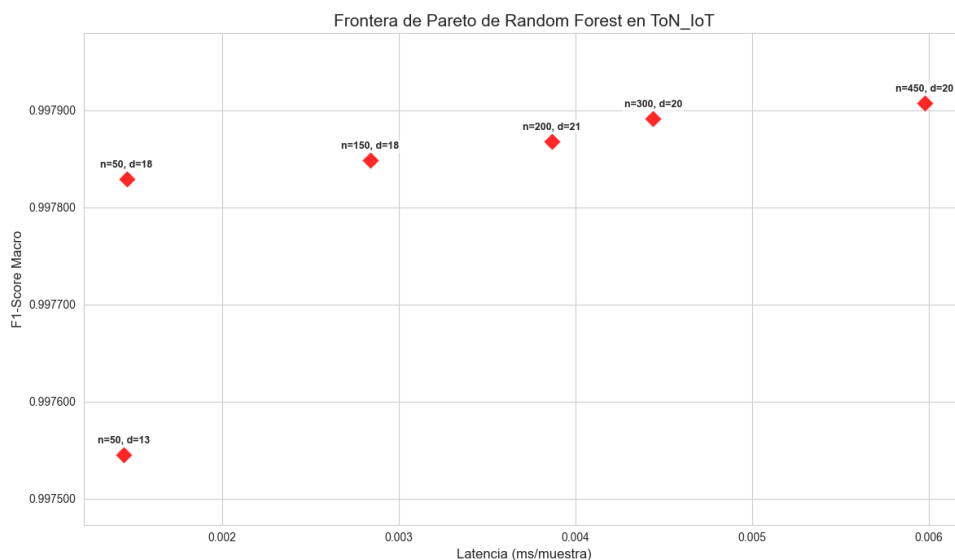


Figura 7.15. Frontera de Pareto – Random Forest (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 13)
2. (50, 18)

3. (150, 18)
4. (200, 21)
5. (300, 20)
6. (450, 20)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	13	0.995912	0.997039	0.001468
Candidato 2	50	18	0.996434	0.997418	0.001459
Candidato 3	150	18	0.996369	0.99737	0.002886
Candidato 4	200	21	0.996598	0.997536	0.004955
Candidato 5	300	20	0.996271	0.997299	0.004452
Candidato 6	450	20	0.996369	0.99737	0.00759

Tabla 7.15. Comparativa final de modelos Random Forest en el set de test (ToN-IoT)

El **Candidato 2** representa el punto de equilibrio perfecto entre capacidad de detección y coste computacional.

A.3.2 XGBoost

La frontera de Pareto obtenida para XGBoost con el dataset ToN-IoT es la siguiente:

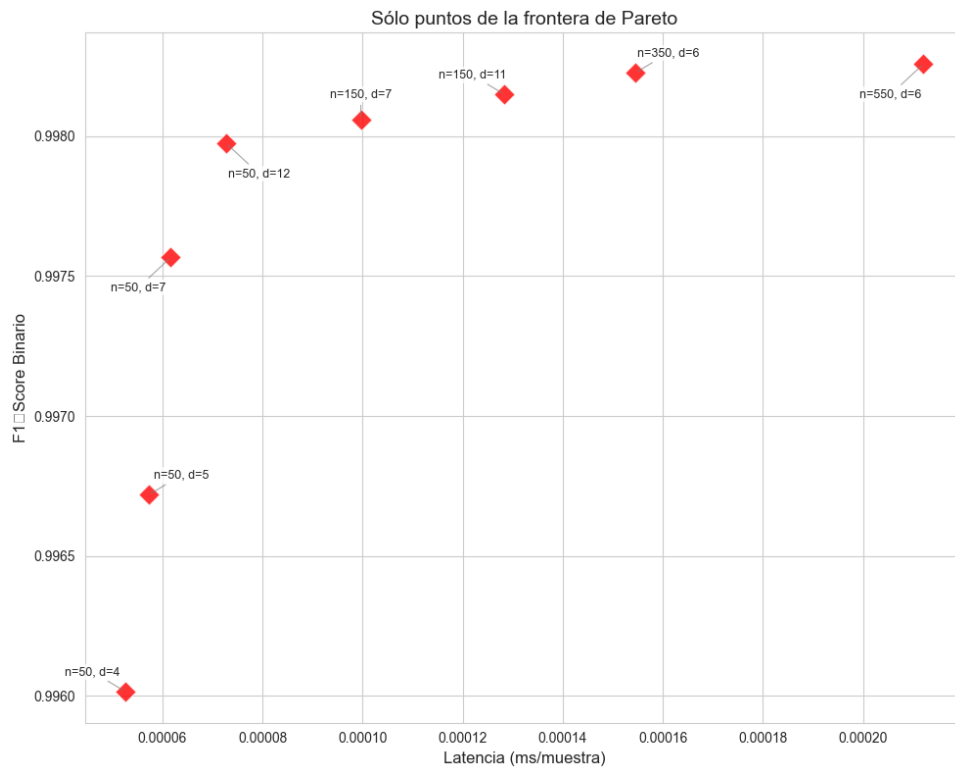


Figura 7.16. Frontera de Pareto - XGBoost (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 4)
2. (50, 5)
3. (50, 7)
4. (50, 12)
5. (150, 7)
6. (150, 11)
7. (350, 6)
8. (550, 6)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	4	0.995979	0.993864	0.00005
Candidato 2	50	5	0.99713	0.995617	0.000039
Candidato 3	50	7	0.997734	0.996541	0.000042
Candidato 4	50	12	0.998168	0.997204	0.000071
Candidato 5	150	7	0.998231	0.997299	0.000071
Candidato 6	150	11	0.998246	0.997323	0.000096
Candidato 7	350	6	0.998261	0.997347	0.000116
Candidato 8	550	6	0.99823	0.997299	0.000167

Tabla 7.16. Comparativa final de modelos XGBoost en el set de test (ToN-IoT)

Vamos a seleccionar al **Candidato 3** ya que representa un compromiso perfecto entre capacidad de detección y coste computacional, con un F1-Score de 0.997734 y una latencia de inferencia de 0.000042 ms.

A.3.3 LightGBM

La frontera de Pareto obtenida para LightGBM con el *dataset* ToN-IoT es la siguiente:

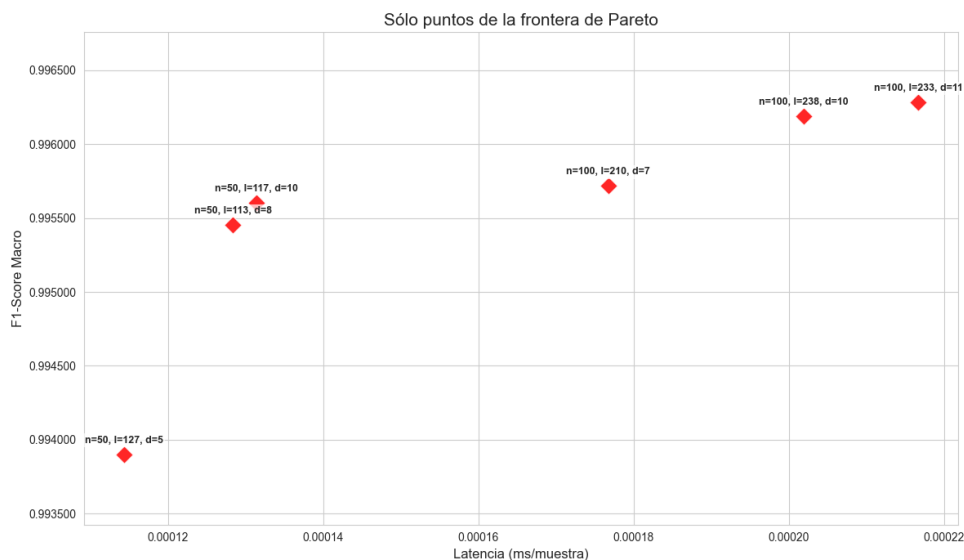


Figura 7.17. Frontera de Pareto - LightGBM (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 127, 5)
2. (50, 113, 8)
3. (50, 117, 10)
4. (100, 210, 7)
5. (100, 238, 10)
6. (100, 233, 11)

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	127	5	0.99456	0.996067	0.00011
Candidato 2	50	113	8	0.996134	0.997204	0.000113
Candidato 3	50	117	10	0.996299	0.997323	0.000298
Candidato 4	100	210	7	0.996594	0.997536	0.000181
Candidato 5	100	238	10	0.996955	0.997797	0.000191
Candidato 6	100	233	11	0.997085	0.997891	0.000202

Tabla 7.17. Comparativa final de modelos LightGBM en el set de test (ToN-IoT)

Nos quedamos con el **Candidato 6** que ofrece un excelente rendimiento (F1-Score de 0.997085) y una latencia de inferencia de 0.000202 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento es inferior comparado con los otros ensembles de árboles.

A.3.4 CatBoost

Para CatBoost, la frontera de Pareto obtenida con el dataset ToN-IoT es la siguiente:

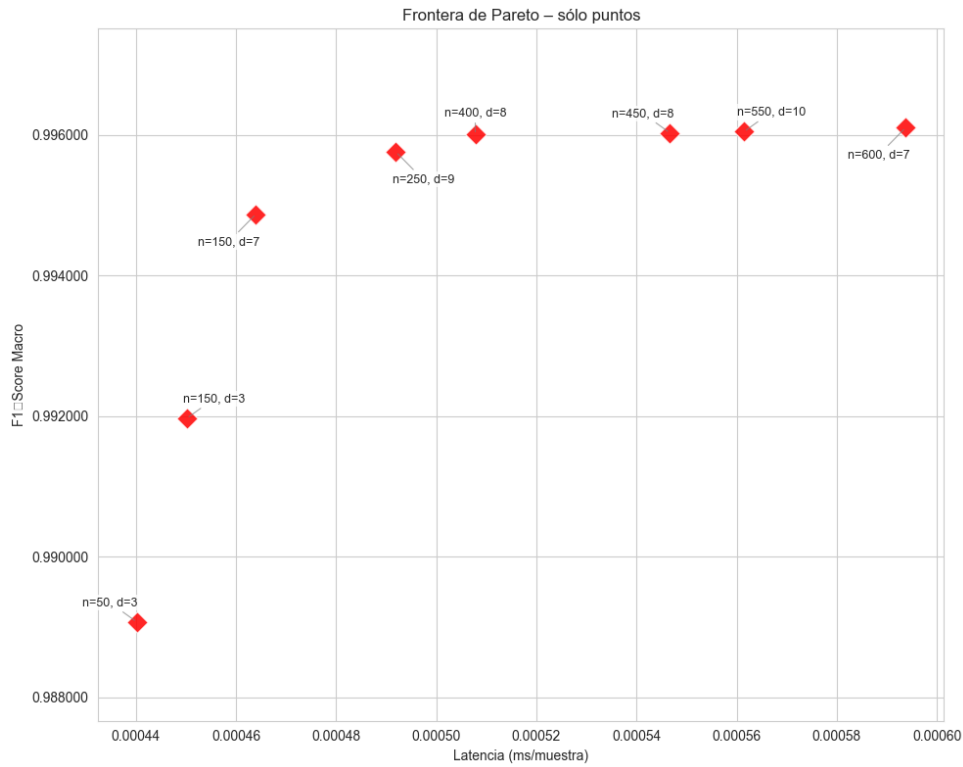


Figura 7.18. Frontera de Pareto – CatBoost (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 3)
2. (150, 3)
3. (150, 7)
4. (250, 9)
5. (400, 8)
6. (450, 8)
7. (550, 10)
8. (600, 7)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	3	0.98919	0.992205	0.000274
Candidato 2	150	3	0.992427	0.994527	0.00031
Candidato 3	150	7	0.995842	0.996991	0.00033
Candidato 4	250	9	0.996628	0.99756	0.000354
Candidato 5	400	8	0.996726	0.997631	0.000456
Candidato 6	450	8	0.996758	0.997655	0.00057
Candidato 7	550	10	0.996759	0.997655	0.000461
Candidato 8	600	7	0.996792	0.997678	0.000419

Tabla 7.18. Comparativa final de modelos CatBoost en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto entre capacidad de detección y coste computacional, con un F1-Score de 0.996628 y una latencia de inferencia de 0.000354 ms.

A.3.5 SVM

La frontera de Pareto obtenida para SVM con el dataset ToN-IoT es la siguiente:

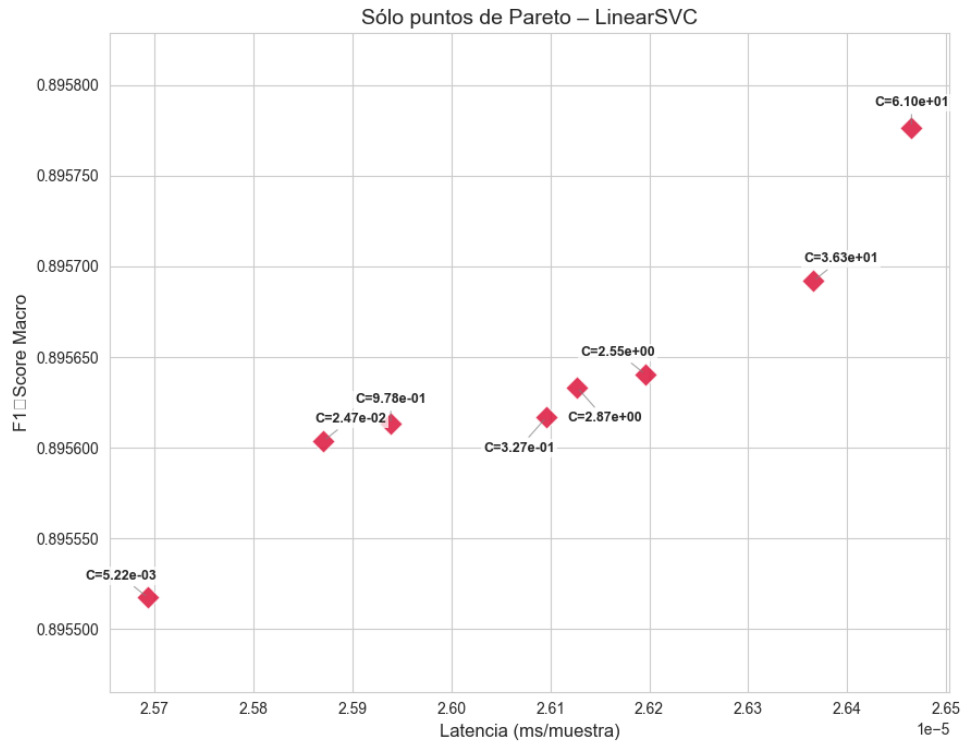


Figura 7.19. Frontera de Pareto – SVM (ToN-IoT)

Los candidatos seleccionados de la frontera de Pareto son:

1. (0.024697)
2. (0.977892)
3. (0.326653)
4. (2.870875)
5. (2.552291)
6. (36.274336)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	0.024697	0.897624	0.926603	0.000027
Candidato 2	0.977892	0.897712	0.926674	0.000051
Candidato 3	0.326653	0.897712	0.926674	0.000029
Candidato 4	2.870875	0.897815	0.926745	0.000031
Candidato 5	2.552291	0.897712	0.926674	0.000047
Candidato 6	36.274336	0.897785	0.926722	0.000029

Tabla 7.19. Comparativa final de modelos SVM en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto entre capacidad de detección y coste computacional, con un F1-Score de 0.897815 y una latencia de inferencia de 0.000031 ms, lo que lo posiciona como un modelo bastante bueno para este *dataset* de IoT, aunque su rendimiento es significativamente inferior al de los modelos basados en árboles.

A.3.6 MLP

La frontera de Pareto obtenida para MLP con el *dataset* ToN-IoT es la siguiente:

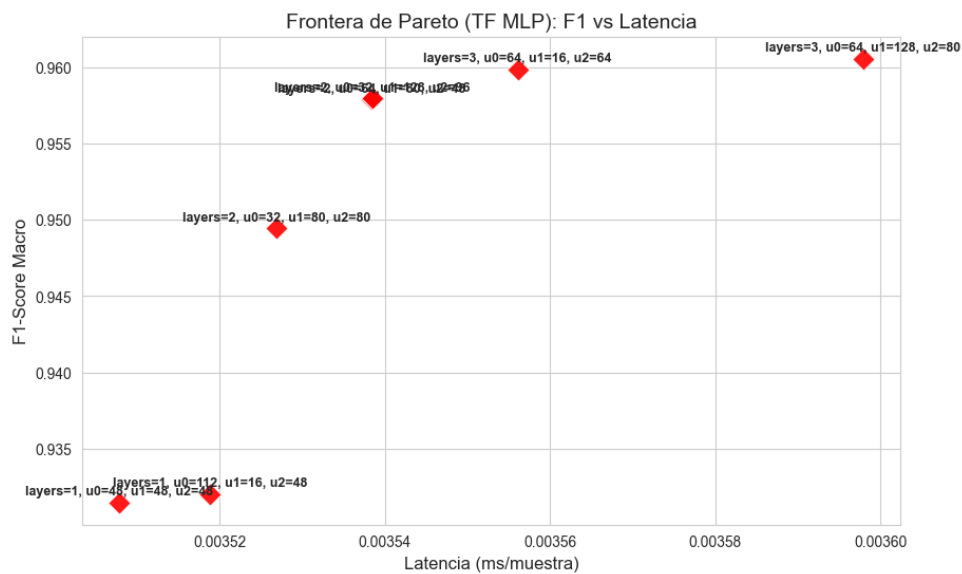


Figura 7.20. Frontera de Pareto - MLP (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (2, 32, 80, 0)
2. (2, 64, 80, 0)
3. (2, 32, 128, 0)
4. (3, 64, 16, 64)

Perfil	Capas	U. I0	U. I1	U. I2	F1	Accuracy	Lat. (ms)
Candidato 1	2	32	80	0	0.961973	0.9733	0.002257
Candidato 2	2	64	80	0	0.962048	0.973418	0.002328
Candidato 3	2	32	128	0	0.96124	0.972873	0.00237
Candidato 4	3	64	16	64	0.963034	0.97401	0.002268

Tabla 7.20. Comparativa final de modelos MLP en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto entre capacidad de detección y coste computacional, con un F1-Score de 0.963034 y una latencia de inferencia de 0.002268 ms, lo que lo posiciona como un modelo bastante bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

A.3.7 CNN-1D

La frontera de Pareto obtenida para CNN-1D con el dataset ToN-IoT es la siguiente:

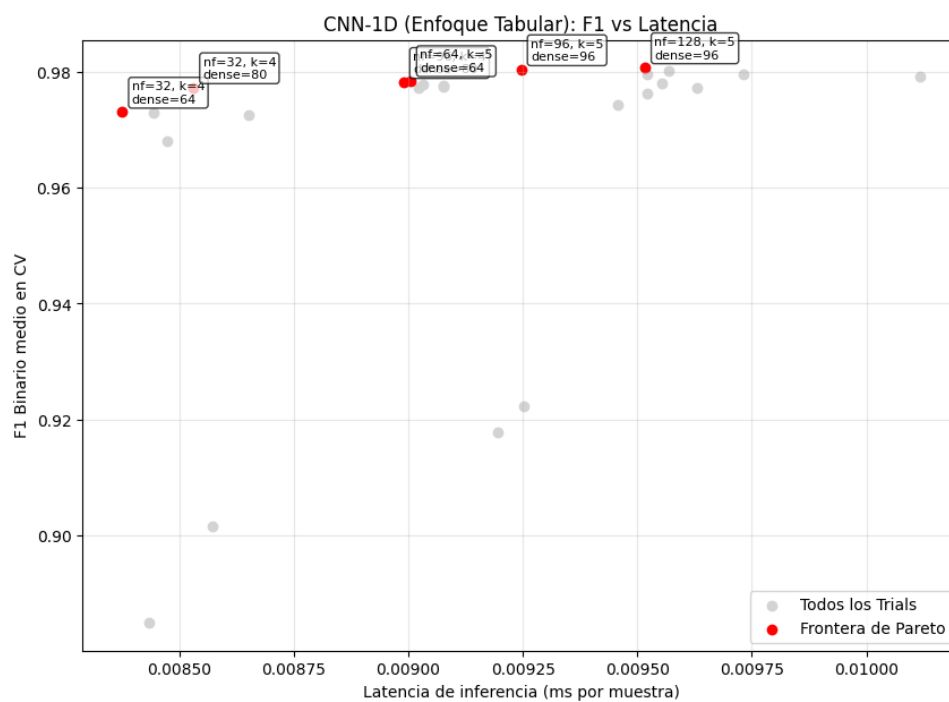


Figura 7.21. Frontera de Pareto - CNN-1D (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (32, 4, 64)
2. (32, 4, 80)
3. (96, 4, 80)
4. (64, 5, 64)
5. (96, 5, 96)

6. (128, 5, 96)

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	32	4	64	0.977894	0.96586	0.008596
Candidato 2	32	4	80	0.974993	0.96124	0.009421
Candidato 3	96	4	80	0.978447	0.966737	0.009012
Candidato 4	64	5	64	0.981183	0.970836	0.009133
Candidato 5	96	5	96	0.981769	0.97176	0.008953
Candidato 6	128	5	96	0.981696	0.971641	0.009564

Tabla 7.21. Comparativa final de modelos CNN-1D en el set de test (ToN-IoT)

El ganador es el **Candidato 5** que ofrece el rendimiento más alto (F1-Score de 0.981769) y una latencia de inferencia de 0.008953 ms.

Apéndice B. Manual de instalación, configuración y ejecución

Este apéndice describe los pasos necesarios para obtener el código fuente del proyecto, preparar el entorno de ejecución y repetir los experimentos presentados en la memoria. La explicación está pensada para una persona que parte de cero y que únicamente dispone del repositorio del TFG.

B.1 Contenido del repositorio

El repositorio se organiza en tres bloques principales:

- **MEMORIA_TFG/**: contiene la memoria en $\text{\LaTeX}$, las figuras utilizadas y los ficheros bibliográficos.
- **DATASETS/**: contiene el script de preprocesamiento en R y los conjuntos de datos reducidos empleados por las notebooks.
- **CODIGO/**: contiene las notebooks de entrenamiento, comparación de modelos, evaluación de consumo en CPU, ataques adversarios y los modelos entrenados en formato `joblib`.

Dentro de **CODIGO/** se distinguen las siguientes carpetas:

- **MODELOS_IT/**: entrenamiento y comparación de modelos sobre UNSW-NB15.
- **MODELOS_IOT/**: entrenamiento y comparación de modelos sobre IoT-23.
- **MODELOS_TON_IOT/**: entrenamiento y comparación de modelos sobre ToN-IoT.
- **ATAQUES_ADVERSARIOS/**: evaluación de robustez frente a ataques FGSM y PGD.
- **entrenar_propio_pc/**: notebooks utilizadas para medir el comportamiento de los modelos ya entrenados en CPU.
- **model/**: modelos finales guardados para cada *dataset*.

B.2 Requisitos previos

Para reproducir los resultados se recomienda utilizar un sistema Linux o macOS. También es posible ejecutar el proyecto en Windows, aunque en ese caso se recomienda hacerlo mediante WSL, ya que las rutas y comandos de este manual están escritos con sintaxis Unix.

El equipo debe tener instalado:

- Python 3.12 o una versión compatible de Python 3.
- R, necesario únicamente si se desea repetir el preprocesamiento desde los datos originales.
- Jupyter Notebook o JupyterLab para ejecutar las notebooks.
- Suficiente espacio en disco para los datos, modelos y entorno Python. En la versión usada para esta memoria, la carpeta `CODIGO/` ocupa varios GB porque incluye un entorno virtual y modelos ya entrenados.

Algunos entrenamientos pueden ejecutarse en CPU, pero se recomienda disponer de GPU para reducir tiempos en los modelos neuronales y en las pruebas más costosas. Las mediciones de latencia y consumo dependen del hardware, por lo que al repetir los experimentos en otro equipo pueden variar los tiempos absolutos. Lo esperable es que se mantengan las tendencias relativas entre modelos.

B.3 Obtención del código fuente

Si se dispone del repositorio en un servicio Git, el primer paso es clonarlo:

```
git clone <URL_DEL_REPOSITORIO> PLACI_TFG
cd PLACI_TFG
```

Si el código se entrega como un archivo comprimido, basta con descomprimirlo y abrir una terminal en la carpeta raíz del proyecto. La carpeta raíz es la que contiene, entre otros, los directorios `CODIGO/`, `DATASETS/` y `MEMORIA_TFG/`.

B.4 Preparación del entorno Python

El repositorio incluye un entorno virtual en `CODIGO/tfg_env/`. Si se trabaja en el mismo sistema para el que fue creado, puede activarse directamente:

```
source CODIGO/tfg_env/bin/activate
python --version
```

En el entorno utilizado para esta memoria la versión de Python era 3.12.3. Si el entorno incluido no funciona en otro equipo, se puede crear uno nuevo desde la raíz del repositorio:

```
python3 -m venv CODIGO/tfg_env
source CODIGO/tfg_env/bin/activate
pip install --upgrade pip
```

A continuación se instalan las dependencias principales:

```
pip install jupyter notebook jupyterlab ipykernel
pip install numpy pandas polars pyarrow scipy scikit-learn
pip install matplotlib seaborn plotly psutil joblib tqdm
pip install optuna xgboost lightgbm catboost tensorflow
pip install adversarial-robustness-toolbox
```

La última dependencia, `adversarial-robustness-toolbox`, es necesaria para las notebooks de ataques adversarios, donde se usa el paquete `art`. Si solo se desean repetir los entrenamientos y comparativas normales, esa dependencia no es imprescindible.

Para registrar el entorno como kernel de Jupyter:

```
python -m ipykernel install --user --name tfg_env --display-name "TFG IDS"
```

Después de este paso, al abrir una notebook debe seleccionarse el kernel TFG IDS.

B.5 Preparación de los datos

Las notebooks del proyecto trabajan con los datos reducidos situados en:

- `DATASETS/dataSets_Reducidos/nusw-nb15/datos_train_NUSW_redux.csv`
- `DATASETS/dataSets_Reducidos/nusw-nb15/datos_test_NUSW_redux.csv`
- `DATASETS/dataSets_Reducidos/iot-23/datos_IOT_23_preparado.csv`
- `DATASETS/dataSets_Reducidos/ton_iot/datos_TON_IoT_redux.csv`

Si estos ficheros ya están presentes, no es necesario ejecutar el preprocesamiento en R. En ese caso se puede pasar directamente a la ejecución de las notebooks.

Si se desea reproducir el proceso desde los datos originales, hay que colocar los ficheros originales en las rutas esperadas por `DATASETS/Preprocesamiento.r`. El script espera la siguiente estructura:

- DATASETS/dataSets_Raw/CIC-IDS-2017/
- DATASETS/NUSW-NB15/
- DATASETS/dataSets_Raw/TON_IoT/
- DATASETS/dataSets_Raw/IOT-23/

El script de R utiliza rutas absolutas del equipo de desarrollo. Si se ejecuta en otra máquina, deben substituirse esas rutas por rutas relativas al repositorio o por la ubicación local equivalente. Una vez corregidas, se ejecuta:

```
Rscript DATASETS/Preprocesamiento.r
```

Este paso genera los ficheros reducidos en DATASETS/dataSets_Reducidos/. Para IoT-23, la notebook CODIGO/MODELOS_IOT/rf_train_iot.ipynb también contiene celdas de preparación que construyen el fichero final datos_IOT_23_preparado.csv a partir de las capturas reducidas.

B.6 Ejecución de Jupyter

Desde la raíz del repositorio, con el entorno activado, se inicia Jupyter:

```
jupyter lab
```

o, si se prefiere la interfaz clásica:

```
jupyter notebook
```

Es importante abrir las notebooks desde la raíz del repositorio o respetar la estructura original de carpetas, ya que muchas rutas de datos se expresan de forma relativa, por ejemplo ../../DATASETS/...

B.7 Reproducción de los entrenamientos

Los experimentos de entrenamiento se organizan por *dataset*. En cada carpeta hay una notebook por familia de modelos. El procedimiento general es:

1. Abrir la notebook correspondiente.
2. Seleccionar el kernel TFG_IDS.
3. Ejecutar las celdas en orden desde el principio hasta el final.
4. Revisar los ficheros csv generados con los resultados de Optuna y las gráficas de frontera de Pareto.

5. Ejecutar la notebook de comparación del *dataset* para obtener las métricas finales agregadas.

Para UNSW-NB15 se usan las notebooks de CODIGO/MODELOS IT/:

- RF_train.ipynb
- XGBoost_train.ipynb
- LightGBM_train.ipynb
- CatBoost_train.ipynb
- svm_train.ipynb
- mlp_train.ipynb
- cnn1d_train.ipynb
- lstm_train.ipynb
- comparativaTodosModelos.ipynb

Para IoT-23 se usan las notebooks de CODIGO/MODELOS IOT/:

- rf_train_iot.ipynb
- xgboost_iot.ipynb
- lightgbm_train_iot.ipynb
- catboost_train_iot.ipynb
- svm_iot_train.ipynb
- mlp_iot_train.ipynb
- cnn_iot_train.ipynb
- lstm_iot_train.ipynb
- comparativaModelos.ipynb

Para ToN-IoT se usan las notebooks de CODIGO/MODELOS TON_IOT/:

- rf_ton_iot.ipynb
- xgboost_ton_iot.ipynb

- `lightgbm_ton_iot.ipynb`
- `catboost_ton_iot.ipynb`
- `svm_ton_iot.ipynb`
- `mlp_ton_iot.ipynb`
- `cnn_ton_iot.ipynb`
- `comparativaModelosTON.ipynb`

Las notebooks de entrenamiento realizan la búsqueda de hiperparámetros con Optuna, evalúan las combinaciones candidatas y guardan tablas de resultados en formato `csv`. Algunos ejemplos son:

- `rf_trials_results_cv.csv`
- `xgboost_iot23_trials_results_cv.csv`
- `catboost_ton_trials_results_cv.csv`

Los nombres exactos pueden variar ligeramente entre notebooks, pero se generan en la propia carpeta desde la que se ejecuta cada experimento.

B.8 Uso de modelos ya entrenados

Si no se desea repetir todo el entrenamiento, se pueden usar los modelos finales incluidos en `CODIGO/model/`. Las rutas son:

- `CODIGO/model/unswnb15/`: modelos entrenados para UNSW-NB15.
- `CODIGO/model/iot-23/`: modelos entrenados para IoT-23.
- `CODIGO/model/ton-iot/`: modelos entrenados para ToN-IoT.

Cada carpeta contiene modelos guardados con nombres como `rf_unsw.joblib`, `xgb_iot23.joblib`, `lgbm_ton_iot.joblib`, `mlp_iot23.joblib` o `cnn_ton_iot.joblib`. Estos ficheros permiten repetir las comparativas sin volver a entrenar.

Para medir los modelos ya guardados en CPU se pueden ejecutar:

- `CODIGO/entrenar_propio_pc/comparativa_cpu_unswnb15.ipynb`
- `CODIGO/entrenar_propio_pc/comparativa_cpu_iot23.ipynb`

- `CODIGO/entrenar_propio_pc/comparativa_cpu_toniot.ipynb`

Estas notebooks cargan los `joblib`, preparan los datos de test y calculan métricas como *accuracy*, F1, latencia de inferencia y consumo de recursos. Esta es la vía más rápida para comprobar los resultados finales sin repetir la búsqueda completa de hiperparámetros.

B.9 Reproducción de los ataques adversarios

La evaluación de ataques de evasión se encuentra en `CODIGO/ATAQUES_ADVERSARIOS/`. Para ejecutarla es necesario tener instaladas las dependencias de aprendizaje automático y la librería `adversarial-robustness-toolbox`.

Las notebooks principales son:

- `ataques-UNSW-NB15.ipynb`
- `ataques_IOT-23.ipynb`
- `ataques_TON-IOT.ipynb`

También existen versiones con prefijo `for_ataques`, usadas como notebooks auxiliares para preparar y comprobar la carga de modelos antes de ejecutar las pruebas completas.

El procedimiento recomendado es:

1. Comprobar que los modelos entrenados existen en `CODIGO/model/`.
2. Abrir la notebook del *dataset* que se quiere evaluar.
3. Ejecutar las celdas de carga de datos y modelos.
4. Ejecutar las celdas de generación de ejemplos adversarios FGSM y PGD.
5. Ejecutar las celdas de evaluación para comparar métricas limpias frente a métricas bajo ataque.

Los ataques se generan principalmente sobre modelos neuronales y después se evalúa la degradación en distintos clasificadores. Las métricas que deben compararse con las recogidas en la memoria son *accuracy*, F1 y *recall*. Las figuras finales incluidas en la memoria se encuentran en:

- `MEMORIA_TFG/images/ATAQUES_ADVERSARIOS/`
- `MEMORIA_TFG/images/ATAQUES_ADVERSARIOS_2/`

B.10 Compilación de la memoria

Para compilar la memoria hay que situarse en `MEMORIA_TFG/`. El documento principal es `main.tex`. La plantilla usa `fontspec`, por lo que debe compilarse con XeLaTeX o LuaLaTeX. Una secuencia habitual de compilación es:

```
cd MEMORIA_TFG
xelatex main.tex
bibtex main
xelatex main.tex
xelatex main.tex
```

La plantilla utiliza recursos locales de `template/`, imágenes de `images/` y la bibliografía situada en `sections/Referencias/references.bib`. Si el compilador informa de referencias o citas pendientes, se debe repetir `xelatex main.tex`.

B.11 Comprobaciones finales

Para comprobar que la reproducción se ha realizado correctamente, se recomienda verificar:

- Que las notebooks cargan los `csv` desde `DATASETS/dataSets_Reducidos/` sin errores de ruta.
- Que el kernel de Jupyter corresponde al entorno donde se instalaron las dependencias.
- Que las notebooks de comparación muestran métricas de F1 y *accuracy* similares a las tablas de la memoria.
- Que los tiempos de inferencia tienen el mismo orden relativo entre modelos, aunque no coincidan exactamente con los de la memoria.
- Que las notebooks de ataques muestran una degradación de prestaciones al aumentar la intensidad de FGSM o PGD.

Las diferencias pequeñas son normales debido a la aleatoriedad de algunos entrenamientos, a la versión concreta de las librerías y al hardware empleado. Si se desea reducir esa variabilidad, conviene mantener las semillas aleatorias fijadas en las notebooks y ejecutar todos los experimentos con las mismas versiones de Python y de las dependencias indicadas en este manual.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga